

A Decision Layer for Robot Object Search Under Unreliable Evidence, With a Checkable Provenance Record for Every Commitment

Kiran Nayudu DDS*

Independent researcher

Working paper v0.5 — September 2026

Abstract

A robot sent to find an object decides from late, partial, sometimes false evidence. We report a decision layer whose every commitment is one of three verdicts carrying a record a checker outside the engine re-derives: FOUND at a room, witnessed by a confirming look; NOT-FOUND, with a certificate that the robot’s own looks covered every room; or UNDECIDED. `robot_mind` is deterministic over an append-only, bi-temporal evidence ledger with typed statuses and runs no language model. Against an exact belief-space planner over a belief that reads the same relations, its four-action margin on the warehouse benchmark falls inside the interval and at two actions that planner leads; what it keeps there is zero wrong commitments, justified abstention and the record. On a scenario family designed without reading the engine it alone missed the exhaustive budget; the defect is fixed and re-measured at 1.000 on seeds held back for it. At a raised resolve bar the engine named a room for an absent object in 121/300 episodes; the certified verdict makes that **0/300**, at one false refusal a second look removes. Unchanged, one build is 30/30 correct and 0/30 fooled on staged decoys, 36/36 on hand-held-camera hunts with 18/18 true tips followed and 18/18 false ones debunked, 134/134 on ALFWorld’s unseen split, 95/96 in AI2-THOR and 83/96 under a real detector, 12/12 on DARPA SubT, 8/8 on a hospital floor, 20/20, 21/21 and 26/26 on ROS 2 integration checks, 1.75 actions per warehouse episode, and 12/12 every day across six days of nightly change. The kit, the baselines, the checker and the ledger exports are released: the checker re-validates 228,989 proof paths over 10,008 episodes with no engine code, and now re-derives each verdict and certificate. The belief arithmetic is withheld with the engine’s source, which ships as a demonstration build.

1 Introduction

A robot sent to find something decides from evidence that arrives late, in pieces and sometimes false: a camera sighting at low confidence, a badge event at a door, a report that the cart took it. The decision that matters is the commitment, “the object is in room *X*”, because a wrong one costs a trip, a mission or a safety case. Three kinds of agent make that decision today, and none can show afterwards why it committed. LLM agents reason fluently over the evidence but keep no separation between what was observed and what was

*Code and data availability: the benchmark kit (scenario generator, every scripted and belief-space baseline, the LLM comparison harness, the independent proof checker, and every engine trace and artifact this paper reports; Apache-2.0, data CC BY 4.0) and a demonstration build of the engine are released at <https://github.com/fourwheels2512/unknown-goals> with this paper’s deposit. The engine’s source is not released; its runs ship as checkable traces (see the reproducibility statement). Development disclosure: the system was designed and implemented by an LLM assistant under the author’s direction; the benchmarked artifact runs no language model (§6, item 7).

inferred. Learned navigation policies need demonstrations or reward and carry no provenance. Classical belief-space search keeps a belief but not the evidence behind it: no epistemic status, no supersession, no defence against a planted claim, no proof. This paper reports a decision layer that derives every commitment from the evidence, attaches a derivation record that a separate checker replays, and was graded against ground truth in every environment it entered, with one build throughout.

The task is object search, the kernel of object-goal navigation in embodied AI. A tool goes missing in a simulated warehouse, and evidence arrives as noisy, timestamped sensor claims (camera sightings, badge events, cart-tracker reports). The agent chooses physical checks, such as inspecting a location or scanning a cart, to find the object within an action budget. The benchmark abstracts perception and locomotion so that the search policy is the variable under test; §4.5 removes each abstraction in turn with the engine unchanged.

The engine, `robot_mind`, is a deterministic program with no language model at inference time. Claims enter an append-only, bi-temporal evidence ledger with typed epistemic statuses, so a hypothesis can never become an observation by scoring high and a stale report can never overwrite a newer one. Beliefs are views over the ledger, not mutable state. Every derived claim records the claims, rules and time constraints it rests on, and a checker outside the engine replays that record. The planner chooses each action for the information it buys, a fresh report is verified before it is trusted, and a refuted belief widens the search instead of ending it. From one integer seed the worlds, observations, choices and proofs reproduce byte for byte. The thesis is this: *an append-only evidence ledger with explicit epistemic statuses, plus a verify-then-redirect search policy over it, gives a low-latency, closed-domain object-search agent whose every decision carries a machine-checkable provenance trail, and that property, unlike accuracy, was supplied by none of the LLM agents evaluated here at any scale we tested.*

The provenance half of that thesis held everywhere it was tested, and the efficiency half now has a control that bounds it. On the warehouse benchmark the engine is 100% correct at 1.75 actions per episode. Where brute force cannot finish it keeps 98–100% of episodes at four and five actions, against 73–90% for an exact belief-space planner over a plain filter’s belief and 27–61% for scripted policies. Give that planner a belief that reads the same relations the engine reads and the four-action margin falls to 0.8 points, inside the interval, which locates the engine’s advantage in how it reads the evidence rather than in how it plans over it. Two hosted LLM agents reach the same accuracy at about 2,000 times the latency and with no trail a checker can replay. The same build then left the warehouse unchanged: 134/134 on ALFWorld’s unseen split, 95/96 fetches in AI2-THOR, 12/12 scored reports on DARPA SubT worlds, 8/8 on a hospital floor, and every target on a Mars terrain and a seabed. On that floor it was correct on 30/30 pre-registered decoy seeds and fooled on 0/30. It then crossed the robot’s seams with real components in the slot. An open-vocabulary detector fetched 83/96 with every commitment correct. Stock Nav2 with SLAM Toolbox in simulation passed 21/21 integration checks, and 26/26 with the detector in the loop. A hand-held phone camera in a lived-in house gave 36/36 hunts, 18/18 true tips followed and 18/18 false tips debunked, all pre-registered. A mind kept across six days of nightly change holds 12/12 every day. A nine-suite stress battery finds it correct on all 1,860 unseen episodes and never confidently wrong under 320 planted lies. One decision costs a third of a millisecond on a laptop core.

1.1 What was uncertain at the outset

Six of the eight technical uncertainties open at the outset were resolved in the engine’s favour, three of them only after a benchmark had exposed a failure and the fix had been built. Of the last two, the absent-object boundary is now closed by a verdict that was built and measured, and the cost-of-memory boundary still carries an objective with an acceptance test (§8). Two further uncertainties were opened by the controls of this round and are listed with them. None of this was assured when the work began. To our knowledge no prior object-search system attaches a replayable derivation to its commitments, and the closest work (§7) either stores memory without closing the hypothesise–act–verify loop or plans over beliefs without keeping

the evidence. Development was therefore organised around the ways the engine could fail. Tables 1 and 2 list them, how each was tested and what came out. The method was the same each time: state the hypothesis, build the mechanism, grade it against hidden truth, submit the result to adversarial review, fix, and re-verify every earlier battery byte for byte before keeping the fix.

The contributions are three claims and the apparatus that carries them.

1. **Every commitment is one of three verdicts, and a checker outside the engine re-derives it.** A verdict is FOUND at a room, witnessed by a confirming look; NOT-FOUND, carrying a coverage certificate over the rooms; or UNDECIDED. The certificate cites one empty-handed look of the robot’s own per room, none older than the last claim that placed the target, and the kit’s checker recomputes the room list, that claim and every cited look from the export’s own contents with no engine code. It rejects 29 forgery mutations, among them certificates whose provenance replays cleanly (§4.7, §9).
2. **Under a matched reading of the evidence, what the engine keeps is the abstention and the record rather than a better plan.** Against an exact belief-space planner over a belief that reads the same relations, the engine’s four-action margin falls inside the interval and at two actions the planner leads. What the engine retains there is zero wrong commitments, a justified abstention and the provenance trail (§4.1).
3. **The margin the engine does have is carried by one mechanism, and its counterpart in a Bayesian belief is one relation.** A leave-one-out ablation costs it more than a quarter of its four-action episodes without multi-anchor hypotheses, and it does not recover at the full budget. On the same grid, adding the departing-cart relation to a plain filter’s belief is worth 24 points, and every other relation in the ladder is worth at most 0.4 (§4.1).
4. the benchmark discipline (§3) and the kit: hidden-truth grading, a declared seed split per evaluation, a fairness contract for LLM agents, scripted and exact belief-space baselines under the identical episode contract, pre-registration wherever the world is not seeded, a scenario family designed under a reading restriction, and a kit in which every baseline regenerates byte for byte and every engine proof re-validates from its ledger export with no engine code, beside a research programme with an acceptance test per objective (§8).

The other evaluations in §4 are portability and integration certifications and are introduced as such: one unchanged build across ALFWorld, AI2-THOR, DARPA SubT worlds, a hospital floor, a Mars terrain, a seabed and a ROS 2 stack, the robot’s seams crossed with real components in the slot, a persistent mind at measured cost, and a nine-suite stress battery.

§2 describes the decision layer. §3 states how every number in the paper was produced. §4 reports the results in the order of the uncertainties. §5 discusses what they establish, §6 bounds them, §7 places the work beside its neighbours, §8 sets out the research that follows and §9 concludes. The kit and the artifacts are described after the conclusion. Appendix A is the ROS 2 integration note, Appendix B the development record and Appendix C a glossary of the acronyms used.

2 The decision layer

The engine is one deterministic Python application of ten modules (74 source files, inventoried in Appendix B), and Figure 1 is the loop it runs on every decision. This section walks that loop once, states the design rules it implements, and names the three mechanisms benchmarking added to it.

A claim arrives typed and stasured. Every input enters the SQLite evidence ledger as an append-only record carrying a subject, a predicate and an object, two clocks (event time and ingestion time), a source, a

uncertainty	how it was tested	outcome	where
1. Sufficiency. Can a compact deterministic engine, with no model in the loop, match LLM agents on temporal, identity and causal reasoning from unreliable sensors?	warehouse benchmark graded against hidden truth; head-to-head with Gemini 3.7 Flash and DeepSeek chat (August 2026) under the fairness contract, latency measured end to end through their APIs	yes: accuracy parity with two hosted LLM agents at three orders of magnitude lower latency, and a replayable evidence-to-decision trail neither agent produced. The multi-anchor hypotheses that secured it were found by benchmarking, after one false low-confidence sighting had severed the true carrier chain: 52%→92% at the hardest level	§4.1–4.3
2. Necessity. Does the evidence machinery buy anything a for-loop does not, and is the advantage the belief or the plan?	budget sweep against scripted policies, against an exact planner over the plain filter’s belief and against the same planner over a belief that reads the same relations the engine reads; a leave-one-out ablation of the engine’s own mechanisms	yes against every control below the exhaustive budget: 98–100% at four and five actions against 73–90% for the plain planner and 27–61% for the scripted policies, with no wrong commitment anywhere. Removing multi-anchor hypotheses costs over a quarter of the four-action episodes and is not recovered at the full budget. Against the matched planner the four-action margin is 0.8 points, inside the interval, and at two actions that planner leads: the advantage is the reading of the evidence, not the plan	§4.1
3. Deception. Does evidence hygiene survive an adversary that plants confident false claims?	the gaslight level (L6); floods of planted lies	yes: 40/40 under gaslight, and never confidently wrong under up to 320 planted lies per episode	§4.8
4. Lifelong memory. Is a memory that never forgets an asset in a changing world?	a persistent mind kept across days in a changing AI2-THOR world	yes, with two modules built for it: age-decayed evidence and refutation-driven search hold one mind at 12/12 every day across six days of nightly change, where before them it held day 1 and then collapsed	§4.6
5. Perception seam. Do commitments derived from a real detector’s raw output, with no instance identities and no association step, stay correct?	pre-registered hunts on hand-held phone footage of one house	yes: 36/36 pre-registered hunts, 18/18 true tips followed and 18/18 false tips debunked. Three earlier graded sittings diagnosed and fixed a resolve-on-sighting rule that is right wherever an inspection can confirm a sighting and wrong when the same clip returns the same detection	§4.5

Table 1: The technical uncertainties open at the outset, part one: how each was tested against hidden ground truth, what came out, and where it is reported. Table 2 carries the rest. Rows 4 and 5 record a failure a benchmark exposed and the fix that followed.

uncertainty	how it was tested	outcome	where
6. Navigation seam. Do the decisions survive a navigation stack's timeouts, aborts and drift?	stock Nav2 with SLAM Toolbox in simulation, then with the detector in the loop as well	yes: 21/21 integration checks on stock Nav2 with SLAM Toolbox and 26/26 with the detector in the loop; the four failed bring-up runs are on record with their fixes in the simulated body and the stack's configuration	§4.5
7. Cost of memory. Can per-decision cost stay bounded as the ledger grows without end?	a 300-day life; per-decision latency measured across three increments of incremental view maintenance	bounded in three increments, $\sim 7\times$, $4\times$ and $\sim 3\times$, every decision byte-identical; the remaining growth has a named cause and a bounded-cost objective with an acceptance test (O2, limitation 11)	§4.6
8. Absent object. Does the commitment discipline hold when the object is not in the building at all?	target removed from the world; raised resolve bar	found, then fixed and re-measured. The rule that closes an exhausted search named a room in 121/300 absent-object episodes at a raised bar, because the verdict vocabulary had no justified not-found. A NOT-FOUND verdict carrying a coverage certificate takes that to 0/300, issues a refusal in 119/300 and in 300/300 with the stale reports stripped, and costs one false refusal in 300 present-object episodes (O1, limitation 13)	§4.7
9. External benchmark under pressure. Does a reduced command allowance on ALFWorld separate the policies the standard allowance cannot?	the engine, a fixed alphabetical sweep and a seeded random order at 10, 15, 20, 30 and 50 commands on both splits	found, fixed and re-measured. It separates them, and on the unseen split against the engine until about 25 commands. Diagnosed: with no evidence every receptacle ties, and the acting policy settled that tie on the alphabetically last name while the sweep takes the first, which accounts for all 57 games the sweep wins at 20 commands. With the tie settled by the episode's own seed, and a class prior the mind learns from the training split out of its own confirmed finds, the unseen curve beats the fixed sweep at every allowance. The 134/134 at the standard allowance stands	§4.4
10. A distribution the rules were not designed on. Does any of this survive a scenario family designed without reading the engine?	a seven-room family with two-hop hand-offs, a confidently wrong log source and a skewed clock, pre-registered before any policy ran, on development seeds and then on validation seeds	found, fixed and re-measured. The engine was the only policy not to reach 1.000 at the exhaustive budget, 0.970 against 1.000, because rooms it had never looked in were not offered to its planner. With the coverage rule, 1.000 on the development seeds and 1.000 on validation seeds not run until the fix was final, 0 of 3,000 warehouse episodes changed. At four actions the matched planner still leads	§4.2

Table 2: The technical uncertainties, part two: rows 6 and 7 were open at the outset, row 8 is the boundary this round closed, and rows 9 and 10 were opened by the controls of this round. Table 1 carries rows 1–5. Rows 8, 9 and 10 each record a failure a benchmark exposed; for rows 8 and 10 the fix is built and re-measured in this paper.

confidence and an epistemic status. The status says what kind of thing the record is: OBSERVED for what a sensor saw, REPORTED for what somebody said, DERIVED for what the engine concluded, and seven more. A hypothesis therefore never becomes an observed fact by scoring high, and a status change appends history rather than touching the raw row, so a late-arriving stale claim cannot overwrite newer knowledge. One integer seed reproduces the worlds, observations, choices, proofs and learned-rule metrics of a whole campaign byte for byte.

Beliefs are views over that ledger rather than mutable state. Each belief is a deterministic function of ledger rows, normalized per question, and evidence ages: an old sighting and an old absence both fade, while validated rules never decay. Every derived claim records the claim ids, rule ids, assumptions and time constraints it rests on, and a checker outside the engine replays that record against the ledger. The benchmark reports the resulting validity rate, 1.000 in every run reported here. The same checker tests the episode’s verdict: a FOUND verdict must be witnessed by an observed sighting at the bar from something other than the reasoner, and a NOT-FOUND verdict’s coverage certificate is re-derived from the export’s own entities and claims rather than read back. It tests no belief value, so whether the room was the right one is graded against hidden truth instead.

Gaps and refutations both create work. A missing variable becomes ranked hypotheses and an information-seeking action, never a terminal “cannot determine”. Repeated failed checks eventually widen the search to places the evidence never pointed at, and a single absence does not, because one absence still reads as possible occlusion. Without that widening an object moved overnight is unfindable, which is the failure the dynamic-house protocol of §4.6 exposed before the mechanism was built. The planner then picks one action for the information it buys, over the belief the ledger defines.

The executor carries that action out and verifies its postconditions, so a fresh report is checked before it is trusted rather than after it has been acted on. Identity grounding is weighted and keeps ambiguity below a threshold instead of resolving it. The learned layer, rule induction and a contextual bandit, may reorder the candidates the planner already justifies and may never outvote evidence or create an observed fact. §4.8 describes the self-repair loop that audits the whole of it.

The language-model seam. A language model has one designed seam, a CognitiveProposer protocol, and the shipped implementation is a deterministic template with no model dependency. The seam allows hybrid operation: a model may translate natural requests into structured goals and propose up to two additional hypotheses per reasoning round. Proposals pass an engine-side verification gate and share a bounded belief mass (8%), so they can steer attention but never outvote evidence, touch the ledger, or alter proofs. No language model runs in any measurement in this paper.

The design rules, for reference. Seven rules state the loop above in the engine’s own terms.

1. **One canonical evidence store.** Claims are append-only records carrying subject, predicate and object, *two clocks* (event time and ingestion time), source, confidence and epistemic status. Status changes append history and raw rows are never touched, so late-arriving stale data cannot overwrite newer knowledge.
2. **Epistemic statuses.** OBSERVED / REPORTED / DERIVED / HYPOTHESIZED / PREDICTED / SIMULATED / VALIDATED_RULE / REJECTED / SUPERSEDED / CONTRADICTED. A hypothesis never becomes an observed fact by scoring high; only a new observation carries OBSERVED.
3. **Proof paths, independently checked.** Every derived claim records its input claim ids, rule ids, assumptions and time constraints. A separate checker replays every proof against the ledger: the operator must come from a declared table, every cited input must be on that episode’s ledger and have

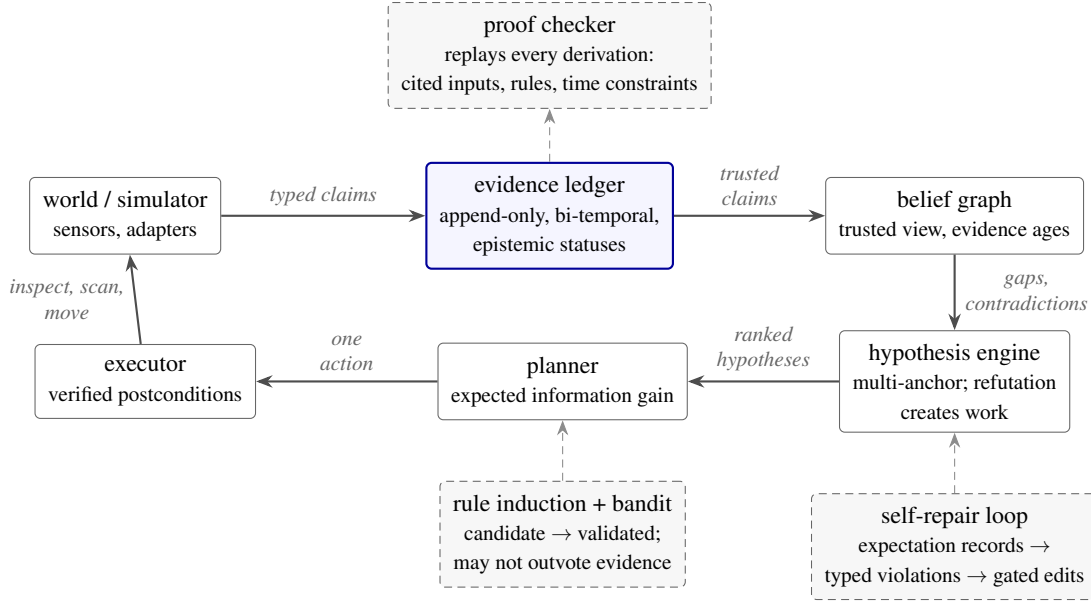


Figure 1: The reasoning loop. Solid boxes run on every decision; dashed satellites audit and tune it. Claims enter typed and statused, beliefs are views over the ledger rather than mutable state, and every derived claim can be replayed by the proof checker. The ledger and its belief-graph view together form an explicit, queryable world model.

reached it no later than the claim it supports, every rule must be one the episode knew, and every time constraint must be satisfiable on the cited claims’ intervals. The benchmark reports the validity rate (1.000 in all reported runs).

4. **Unknown creates work.** A missing variable becomes ranked hypotheses and an information-seeking action, never a terminal “cannot determine.”
5. **Explainable belief arithmetic.** Every belief is a deterministic function of ledger rows, normalized per question, and the engine’s developer output prints every number with its full breakdown (the demonstration build prints belief and status only). Evidence *ages*, symmetrically for support and contradictions, while validated rules (semantic memory) never decay. How support, contradiction, age and rule reliability combine, and the constants they combine under, are withheld from this version of the paper (see the reproducibility statement).
6. **Refutation creates work, exactly like ignorance.** Repeated failed checks eventually widen the search to places the evidence never pointed at; a single absence does not, because it still reads as possible occlusion. The trigger and the widening rule are withheld from this version of the paper. Without this mechanism an object moved overnight is unfindable; the dynamic-house protocol (§4.6) exposed that failure, and the mechanism was built in response.
7. **Determinism.** A single integer seed reproduces worlds, observations, choices, proofs and learned-rule metrics byte-for-byte.

What benchmarking added. Three mechanisms proved load-bearing and were *discovered through benchmarking*, not designed up front: **multi-anchor hypotheses** (the most recent sighting is not trusted blindly; adding them fixed a failure mode where one false low-confidence sighting severed the true carrier chain and

lifted hardest-level success 52%→92%); **transport handoff** (a container that leaves with plausible cargo changes what its parent hypothesis is worth: one object cannot be in two places); and **repetition decay with a policy value gate** (repeating an action without new evidence costs it value, and the learned policy may only choose among candidates the planner still justifies). How each of the three is computed is withheld from this version of the paper; what each does to the numbers is measured in §4.1.

Their measured shares are in §4.1, under the evaluation discipline the next section states.

3 Evaluation method

Every number in this paper was produced under one discipline: grading against hidden truth the agent never sees, a seed or world split declared per evaluation, and an adapter that may not read the answer. This section states that discipline once, so the results can be read without it.

Terms for verdicts, and what the checker certifies. In this paper *commitment* names one event, and every count that carries the central invariant counts that event. Table 3 fixes the verdict vocabulary; §4.7 reports each row’s rate under the one condition that exercises them all. Four thresholds recur. The *resolve bar* is the commitment confidence threshold on an observation: the sighting confidence at which the direct path commits (0.0 in the warehouse and simulator batteries, 0.9 in the real-pixels harness and in §4.7). The *claim gate* is the detection-score ingestion threshold (0.05): the detector confidence below which a detection never becomes a claim. The *confirmation confidence* (0.9) is what an inspection writes when it confirms a sighting. The *sighting bar* of §4.5 is the resolve bar under that harness’s name.

The engine’s vocabulary against the standard one. Table 4 maps every term this paper coins to its closest counterpart in POMDP planning, active perception, tracking or database provenance. The standard term is the one to read the results with; the short name is kept where the paper’s own tables and artifacts use it.

Scope: what the engine reasons over. The engine reasons over a structured interface by design, and the interface is the adapter contract of Appendix A. An entity registry gives every location, container, person and object an id and a type, and a fixed predicate vocabulary types every claim. No model is in the loop, so free text never becomes an entity. Two consequences are worth separating from the limitations. An entity an adapter first reports part-way through a hunt is registered and searched like any other: the hospital, SubT and ALFWorld adapters all do this, and it is how a robot meets an object nobody declared. Open-vocabulary perception is the detector’s job, on the seam §4.5 crosses, and the phone battery exercises it with a commodity detector and no instance identity at all. Natural-language goals are out of scope: a goal is a typed record, and §8 (O5, O6) states what would bring language and learned semantic priors inside the boundary without letting either create an observation.

The generator and the levels. A seeded generator produces warehouse scenarios in two separated layers. The *hidden truth* (actual object locations and causal movement chains) is seen only by the evaluator. The *observable evidence* (noisy, incomplete, sometimes contradictory claims) is all the agent sees. Five difficulty levels, L1 to L5, run from a direct sighting to contradictions, occlusion and wrong-branch recovery; the notation paragraph below names each. Truth branches are sampled. At L3 and above the cart transports the object in ~72% of episodes, the person keeps it in ~20%, and the absence report is spurious in ~8%. No fixed strategy is always right, and grading is against ground truth, not plausibility.

term	meaning	triggers an action	retractable	a wrong commitment if wrong
hypothesis	a ranked candidate holder or room in the belief’s support	yes: actions discriminate among them	yes	no
transient verdict	a resolved location held between two reasoning calls and re-examined at the next; it triggers the inspection that confirms or retracts it	yes	yes	no; reported separately (§4.7: 74/300 present-target episodes at bar 0.9; 799 firings in the absent arm)
committed verdict (FOUND)	the resolved location when the episode ends: in the warehouse and the simulators only after a confirming inspection; in the real-pixels harness on a sighting that clears the resolve bar, or by exhaustion once every zone has been inspected	yes	only by a later episode	yes: every “wrong commitment” count in this paper is this event
no verdict (UNDECIDED)	the episode ends with no resolved location: the budget is spent, or the resolving sighting aged past the direct path’s window	no	—	no; reported as a rate (§4.7)
NOT-FOUND	a refusal carrying a coverage certificate over the rooms: one empty-handed look of the robot’s own in every room the episode knows, none of them older than the last claim that placed the target. Measured in §4.7; the switch that issues it defaults to the behaviour every shipped number was measured under	yes	after a movement event touching the target	yes, if the object was present
proof path	the derivation record behind a verdict: the cited claims, rule ids and time constraints. The kit’s checker certifies <i>provenance integrity</i> (every cited claim and rule exists, was ingested no later than the claim it supports, and the time constraints hold), re-derives the <i>coverage certificate</i> behind a NOT-FOUND verdict from the export’s own entities and claims, and requires a FOUND verdict to be witnessed by an observed sighting at the bar from something other than the reasoner. It does not re-derive any belief value; <i>empirical correctness</i> is measured against hidden truth and never inferred from validity (§4.1)	—	—	—

Table 3: The verdict vocabulary. Every “zero wrong commitments” figure in this paper counts the third row; the second and fourth rows are reported as their own rates where they occur.

term in this paper	the standard term it names	literature
commitment / no verdict	a decision to accept rather than abstain, and the abstention itself: classification with a reject option, selective prediction	[10, 18]
resolve bar, sighting bar	the commitment confidence threshold on an observation, the accept/reject threshold of a selective classifier	[18]
claim gate	the detection-score ingestion threshold, the detector’s operating point	[9]
transient verdict	a commitment the next observation retracts: non-monotonic belief revision, dependency-directed retraction	[14, 16]
NOT-FOUND, coverage certificate	a certificate of non-existence; the planning analogue is an unsolvability certificate, and the proof-object form is a proof-carrying plan	[17, 21]
proof path	a derivation record: why-provenance, which inputs by which rules	[7, 20]
ledger	an append-only provenance store, bi-temporal in the database sense (valid time and transaction time)	[20, 34]
belief as a view	a belief state computed from the observation history rather than maintained recursively	[23]
the belief’s support (the paper’s “beam”)	the candidate set carrying mass; in tracking, the live hypotheses	[23, 40]
multi-anchor hypotheses	multiple-hypothesis data association, kept over past sightings rather than over sensor returns	[40]
refutation-driven widening	expansion of the belief’s support on negative evidence; negative information in tracking	[23, 40]
contradiction bookkeeping	conflict recording, as a truth-maintenance system keeps it: the record of which claims cannot hold together, and which was marked CONTRADICTED	[14, 16]
verify-then-redirect	verification before exploitation: an information-gathering action taken before a report is acted on	[12, 42]
the proposer (CognitiveProposer)	an LLM hypothesis proposer in a hybrid planner, behind a symbolic verifier. Not used in any measurement here	[2, 28]
metacognitive supervisor	a metacognitive controller over expectation violations; the autonomic MAPE-K loop	[13, 24]
exploration drive	an intrinsic exploration bonus, not a frontier map	—
decoy, staged twin	a same-class distractor in data association	[40]
closed ontology	a fixed entity registry and predicate vocabulary: the adapter contract of this paper	—

Table 4: Every term this paper coins, against its closest standard counterpart. The right-hand column is the literature the term should be read against, not a claim of equivalence: multi-anchor hypotheses are multiple-hypothesis tracking over evidence rather than over sensor returns, and the exploration drive is not the frontier-map mechanism of the open-vocabulary line. How the mechanisms in rows 10–12 compute their numbers is withheld (see the reproducibility statement).

Seed and world discipline. Table 5 lists the evaluations of §4 in the order they are reported and states, for each, which seeds or worlds were available during design and debugging, which were used only for reporting, and which were declared before they ran. Where a fix followed a first run on a reporting set, the result says so. The primary benchmark’s held-out and comparison sets are disjoint seeds from the same generator.

One evaluation needed a stronger rule than a seed split, and §4.2 states it. A second generator family was designed under a reading restriction: its author read the world model, the executor, the baselines and the evaluation method, and did not open the engine’s reasoning, planning, learning or configuration modules, so the family could not be shaped around the mechanisms it tests. The design and its predictions were written down before any policy ran on it. Its seeds are then split again: 70000–70099 are development seeds, on which a defect was diagnosed and a fix designed, and 70100–70199 are validation seeds, which were not run until the fix was final. Both are reported.

The fairness contract for LLM agents. All agents run under one fairness contract: identical evidence text, action menu, simulator, sensors, budget (8 actions) and success rule, graded against identical hidden truth. Temperature is 0, with one re-prompt permitted on unparseable output, over 48 episodes (12 seeds $\times$ levels 2–5). Local models run through Ollama [36, 37]. The API models are deepseek-chat [15] and gemini-3.7-flash [19], both accessed August 2026 (hosted models drift; the artifacts record the behavior measured at that date).

The embodied adapter contract. Every embodied evaluation shares one contract. The engine is byte-identical in all of them and decides every search step from typed claims on an initially empty ledger. A minimal adapter translates simulator perception into claims and engine actions into commands, and may not read target placements. Success is graded by the simulator’s own signals or by the benchmark’s published rule. Locomotion is teleport-abstracted, as ALFWorld’s `go to` is, and each adapter’s sensing model is stated with its result, so what the embodied numbers measure is the search policy rather than a full robotics stack.

Three further boundaries hold across all of them. There is no SLAM except in the Nav2 lanes, where SLAM Toolbox localizes the base while the frustum sensor’s vantage is read from physics. Sensing is simulator metadata except in three places: the AI2-THOR detector battery and the camera lane use tracker stand-ins for instance association, and the phone battery has a real camera and no association step. MoveIt 2 was not run, and its interface is the grasp-pose hand-off, a `PoseStamped` in the goal frame. AI2-THOR’s placement randomization is not bit-stable across runs, so those episodes are artifact-logged rather than seed-reproducible. Two further simulators were not run: Habitat’s headless builds need an EGL device that WSL2 does not expose, and OmniGibson’s Isaac Sim installation was not completed. `docs/EMBODIED.md` records the state of each and the commands that resume them.

Which figure is the headline. Unless stated otherwise a step figure is the shipped build at the eight-action budget with its learned rule loaded. The headline efficiency figure is the held-out battery: 300 episodes, seeds 10000–10074, levels 2–5, learned layer off, 100% correct at 1.75 actions. Every other step figure names its seed family, levels, build and learned-layer state where it appears.

Notation, level names and version tags. Success rates carry Wilson 95% intervals (the score interval for a binomial proportion). Benchmark levels: L1 direct sighting; L2 person-carrier bridge; L3 person-and-cart transport chain; L4 distractors, decoys and stale claims; L5 contradictions, occlusion and wrong-branch recovery; L6 the adversarial planted-report level (“gaslight”). Version tags name engine milestones: v0.2.0 is the config version of the rule set after the temporal probes (limitation 4); v0.3 added evidence ageing and the persistent-mind modules; v0.4 closed the metacognitive loop (§4.8); v0.5 is the build reported here. The engine is written `robot_mind` throughout, and Appendix C defines the acronyms.

evaluation	development (design, debugging)	reporting only	declared before running
warehouse engine, learned layer	seeds 0–249 (rules designed on this distribution; rule mined here)	held-out 10000–10074 $\times$ L2–5, 300 episodes per arm	—
warehouse baselines, LLM contract, budget sweep	— (engine changes after the first campaign came from hand-built temporal probes; re-run figures reported beside originals)	seeds 20000–20011 $\times$ L2–6	—
ALFWorld	one belief-hygiene fix after the first run; first run kept in history	valid_unseen 134 games; random and sweep baselines on both splits	external benchmark
AI2-THOR, oracle sensing	seeds 7, 11, 23, 42, 99, 256 (adapter fix loop)	seeds 137, 314 (no fix diagnosed on them)	—
DARPA SubT	bridge fixed on tunnel_01 first, then the other worlds	4 worlds $\times$ seeds 7, 11, 23	external worlds and scoring rule
hospital fetch, tips, decoys	sensor-model fix diagnosed on the seed-7 smoke; battery re-run	seeds 7, 11, 23, 42, 99, 137, 256, 314 $\times$ 3 policies; decoys on the six false-tip seeds	tip-coin addendum seeds 3, 5, 6, 8 chosen by coin alone before running; the 30-seed decoy power run pre-registered and run to completion
AI2-THOR, real detector	seeds 7, 11, 23 (synonym table, gate, confirmation ordering)	—	seeds 42, 99, 137, 256, 314 declared and launched on commit 8b4a83f before any ran: 52/60, 52/52 correct; two start-up crashes rerun as fresh draws, crash records shipped
ROS 2 stack, Nav2, camera lane	four diagnostic Nav2 runs, resolved in the simulated body and in configuration; one CPU smoke of the camera projection on teleported views	staged truths bay_3/1/4/2 + one tip, three lanes (demo driver, Nav2, Nav2 + YOLO-World)	—
AI2-THOR, lifelong	habit spot chosen by sweep; first curve shipped	the reported 30-episode curve, 4-episode wrong-habit run	—
stress battery, self-repair exam	v1 sabotage roster screened for detectability	sealed seeds and hidden-truth grading outside the loop’s reach	exam v2 jitters every field before sabotage
held-out generator family	seeds 70000–70099 (the re-look defect was diagnosed and its fix designed here)	validation seeds 70100–70199, not run until the fix was final	the family’s design, and nine predictions about it, written before any policy ran; its author did not read the engine’s reasoning, planning, learning or configuration code
ALFWorld under reduced command allowances	valid_seen at every allowance, re-runnable	valid_unseen, one run per cell, never re-run	the allowance grid declared before the unseen runs

Table 5: What was available to whom, per evaluation. “Reporting only” sets were not used to take design decisions; exceptions are stated in the development column and at the result.

4 Results

The subsections below take the uncertainties of Tables 1 and 2 in order, each opening with the answer it reached and the control that bounds it. §4.4 is the exception. It reports what the same build did in worlds it was not built for, which no uncertainty anticipated until the reduced command allowances of row 9.

4.1 Does the evidence machinery matter?

Under a budget brute force cannot exhaust, what carries the engine’s margin is the way its belief reads the evidence, not the plan over it. The controls are an exact belief-space planner over a plain filter’s belief, the same planner over a belief that reads the same relations the engine reads, four scripted policies, and the engine with one of its own mechanisms removed at a time. Uncertainties 1 and 2 asked whether a compact deterministic engine is sufficient for this task and whether its evidence machinery buys anything a for-loop does not; on this benchmark the answer to both is yes, and Table 9 is where the necessity claim is settled and bounded.

Held-out result and the learned layer. The engine finds every target in 300 held-out episodes per arm. Its learned layer cuts actions to success by 16% (Table 6). The layer is a transport rule mined from the engine’s own episodes and a contextual bandit over action types, evaluated frozen. The rule deploys at its measured reliability of 0.71, and proof paths cite it at that value. Two caveats apply. The proof checker validates provenance only: cited claims and rules exist in the ledger and the time constraints hold on replay. It does not re-derive conclusions, and a valid trail never says the inputs were true. The miner’s labels are simulator ground truth; a physical deployment would need a labeling process.

metric	baseline	learned layer
task success	1.000	1.000
avg. actions to success	1.75	1.47 (−16%)
L5 actions	2.33	1.88 (−19%)
proof validity	1.000	1.000

Table 6: The learned layer keeps success and proof validity at 1.000 and cuts actions to success by 16% overall and 19% on L5. 300 held-out episodes per arm, seeds 10000–10074 $\times$ levels 2–5, disjoint from the seeds the rules were designed on.

Scripted baselines at the full budget. At the full budget success does not discriminate. Every policy in Table 7 reaches 60/60, because eight actions cover the six locations and two carts, as on ALFWorld (§4.4); steps and the cost of deception do. A plain Bayesian belief filter, with argmax-posterior selection and none of the engine’s machinery, spends barely fewer steps than the random walk, and the engine’s 1.75 is $2.5\times$ fewer than the filter’s, so what buys the efficiency is not belief updating in general. On the gaslight level the two policies that read the evidence naively chase the planted tip, the evidence-ignoring sweep is unlucky in its fixed order, and the random order pays the least of the scripted five because it cannot be deceived. The engine pays 3.25: it verifies the tip once and redirects.

Below the exhaustive budget. Where brute force cannot finish, the benchmark discriminates, and the engine keeps 98.4% of episodes at four actions where the scripted policies keep between a quarter and a half. Table 8 and Figure 2 cut the action budget from eight to two for every policy (experiments/budget_sweep.py)

policy	correct (L2–6)	avg. steps	gaslight L6 steps
sweep (ignores evidence)	60/60	5.88	6.83
random order	60/60	4.92	3.92
last-seen-first	60/60	5.97	6.92
belief filter (no ledger/statuses/proofs)	60/60	4.80 (4.40 on L2–5)	6.42
belief-space planner, exact (filter’s belief + lookahead)	60/60	3.13 (2.60 on L2–5)	5.25
engine	60/60	2.05 (L2–6; 1.75 on L2–5)	3.25

Table 7: Scripted baselines under the identical episode contract, 12 seeds per level on levels 2–6, with the same generator, executor, 8-action budget, success rule and hidden-truth grading as the LLM comparison (`experiments/scripted_baselines.py`; the policies are defined in Appendix B). Every policy reaches 100% because the budget covers the six locations and two carts, so the two right-hand columns are the discriminating ones.

and carry every cell and interval. No policy commits wrongly in any of the 18,000 episodes, because the success rule requires a confirming inspection.

The strongest scripted competitor is an exact belief-space planner under the same contract (`experiments/belief_planner.py`). It maximizes the probability of finding the target within the remaining budget, looking ahead over what each check could reveal (solved to optimality; recursion and sensor constants in Appendix B). It starts from the belief filter’s own reading of the evidence, with the true sensor model. So it is optimal for the POMDP the filter’s model defines, and lookahead is the only thing it adds. Lookahead adds a lot and is not enough: it closes about half of the filter’s gap to the engine and none of the rest. What remains is the choice of belief the lookahead runs over, and the next two paragraphs measure that choice from both sides.

Each mechanism’s share. A leave-one-out ablation on the shipped build measures what the three mechanisms of §2 contribute (`experiments/ablation_loo.py`; `data/artifacts/ablation_loo.json`). Each is switched off in turn and the engine runs the same 3,000 episodes as the sweep, learned rule loaded and every proof checked. The shipped arm reproduces every cell of Table 8. Multi-anchor hypotheses carry the margin, and the engine does not recover at the full budget without them: 78.8% at eight actions, where every scripted policy reaches 100%. Repetition decay carries a second, disjoint share, and both arms’ intervals are disjoint from the shipped arm’s at every budget from three upward. Transport hand-off’s share is not measurable on this family: the arm without it sits inside the shipped arm’s interval at every budget (Table 9). No arm commits wrongly in any of its 3,000 episodes; every point of margin lost is a no-verdict.

The same evidence, read the same way. The exact planner above is optimal over a weaker reading of the evidence than the engine’s, so it cannot separate a better plan from a better reading. A second exact planner closes that gap. It runs the same dynamic programme over a belief that reads the relations the engine reads: every past sighting as a live alternative, a carrier’s movement only when the record postdates the association, an absence discounting only what was in the room before it, and a cart that leaves a carrier’s room as evidence that the object may have gone with it (`experiments/matched_belief_planner.py`; its plain arm reproduces the published planner’s sweep cell for cell and episode for episode). Against that control the engine’s 25-point margin at four actions falls to 0.8 points, inside the interval: 9 episodes go to the engine alone and 5 to the planner, McNemar $p = 0.42$. At two actions the matched planner leads, 61 episodes to 15, $p = 1 \times 10^{-7}$. The cumulative ladder in Table 9 attributes the whole of that gain to one reading: adding the departing-cart relation is worth 24.0 points at four actions, while the carrier reading is worth 0.0 and the absence reading 0.4, and keeping every sighting as a live alternative costs 10.4 points on its own and is not

separable from the cart relation. So the margin over the published planner was the belief’s reading of the evidence, not the plan. The engine’s own ablation agrees from the other side: removing its transport hand-off costs it 0.4 points, because it reaches the cart by other routes, and the plain filter has no such route at all.

Why a horizon plan does not help. A budget-aware lookahead inside the engine was built, measured and left off. Under this action model what one look can reveal and what another can reveal never overlap, so the probability of finding the object within k looks is additive over the looks taken: every ordering of the best k carries the same value, and no horizon plan can improve the first look. Measured, it is never better and is worse in the middle: the warehouse falls from 98.4% to 96.6% at four actions with the horizon plan on, and spends more actions doing it (data/artifacts/budget_sweep_lookahead.json). This is a statement about the problem rather than about one implementation, and it is the reason the engine’s remaining margin does not live in deeper planning.

What is left at two and three actions. At the tightest budgets the engine is behind the matched planner, and what it gives up there is coverage, not correctness. At two actions it holds no verdict in 152 of 500 episodes and commits wrongly in none, while the planner arms simply run out of actions with a guess already made. The two ways of not answering are graded the same here and are not the same thing, which is what §4.7 separates.

Under a budget that brute force cannot exhaust, the machinery is necessary: without multi-anchor hypotheses the engine loses more than a quarter of its episodes and does not recover at the full budget. What it is not is a better plan. A robot with a finite battery operates in the budget-limited regime, and in that regime the reading of the evidence is what pays.

budget	engine	planner (exact)	belief filter	random	last-seen	sweep
2	69.6 ± 4.1 (1.63)	40.6 ± 4.3 (1.67)	40.6 ± 4.3 (1.67)	23.8 ± 3.7 (1.88)	11.0 ± 2.8 (1.96)	14.0 ± 3.1 (1.93)
3	89.8 ± 2.7 (1.93)	52.8 ± 4.4 (2.26)	42.0 ± 4.3 (2.26)	34.2 ± 4.2 (2.64)	19.6 ± 3.5 (2.85)	21.0 ± 3.6 (2.79)
4	98.4 ± 1.2 (2.03)	73.4 ± 3.9 (2.74)	42.6 ± 4.3 (2.84)	48.2 ± 4.4 (3.30)	27.4 ± 3.9 (3.65)	28.6 ± 4.0 (3.58)
5	100 [99.2, 100] (2.05)	90.0 ± 2.6 (3.00)	42.6 ± 4.3 (3.42)	61.0 ± 4.3 (3.81)	34.0 ± 4.2 (4.38)	35.6 ± 4.2 (4.30)
6	100 [99.2, 100] (2.05)	97.8 ± 1.4 (3.10)	44.2 ± 4.4 (3.99)	72.2 ± 3.9 (4.20)	42.6 ± 4.3 (5.04)	42.6 ± 4.3 (4.94)
8 (shipped)	100 (2.05)	100 (3.12)	100 (4.80)	100 (4.62)	100 (5.88)	100 (5.78)

Table 8: Correct-commitment rate in % with the Wilson 95% half-width (mean actions in parentheses), 500 episodes per cell (100 seeds $\times$ levels 2–6, seeds 20000–20099), as the action budget shrinks; at 100% the interval is one-sided and is printed as its bounds. The planner is the exact finite-horizon belief-space planner of §4.1 (the filter’s reading of the evidence, the true sensor model, optimal lookahead). Zero wrong commitments in every cell. The engine’s step counts differ by build and by seed family, and Appendix B tabulates the three builds side by side. data/artifacts/budget_sweep_100.json, belief_planner_sweep.json; the 12-seed runs that match Table 7 at budget 8 are budget_sweep.json and belief_planner_sweep_12.json.

Confidence and counterfactuals. The engine’s stated beliefs are calibrated, and every answer is attributable to named claims. Calibration against hidden truth gives ECE 0.047 on levels 1–5 (Figure 3); the gaslight cell, the Brier scores and the shape of the miscalibration are in Appendix B. The robot-mind whatif command re-computes beliefs with a claim subset removed: on gaslight episodes the planted tip accounts for all of the belief in the fake location, and removing it leaves the true chain’s ranking unchanged.

Necessity is settled on this generator and its scope is now stated: the machinery is necessary against every control raced here, and against the matched control what remains at the headline budget is inside the interval. Two questions follow. §4.2 asks whether any of it survives a family designed without reading the engine, and §4.3 asks what the same evidence buys against an agent that reasons in language.

budget	engine	engine, no multi-anchor	exact planner, plain belief	exact planner, matched belief	exact planner, matched, informed
<i>correct-commitment rate</i> [Wilson 95%], 500 episodes per cell					
2	.696 [.65,.74]	.600 [.56,.64]	.406 [.36,.45]	.734 [.69,.77]	.788 [.75,.82]
3	.898 [.87,.92]	.700 [.66,.74]	.528 [.48,.57]	.910 [.88,.93]	.920 [.89,.94]
4	.984 [.97,.99]	.728 [.69,.77]	.734 [.69,.77]	.976 [.96,.99]	.972 [.95,.98]
5	1.000 [.99,1.0]	.740 [.70,.78]	.900 [.87,.92]	.998 [.99,1.0]	1.000 [.99,1.0]
6	1.000 [.99,1.0]	.752 [.71,.79]	.978 [.96,.99]	1.000 [.99,1.0]	1.000 [.99,1.0]
8	1.000 [.99,1.0]	.788 [.75,.82]	1.000 [.99,1.0]	1.000 [.99,1.0]	1.000 [.99,1.0]
<i>mean actions spent</i>					
2	1.63	—	1.67	1.53	1.48
4	2.03	—	2.74	1.89	1.77
8	2.05	—	3.12	1.91	1.80
<i>cumulative reading, at four actions: each rung adds one relation to the plain belief</i>					
plain belief			.734		
+ the matched frame, with the plain reading of every relation			.836	+10.2	
+ every past sighting kept as a live alternative			.732	−10.4	
+ the carrier’s movement read in time order			.732	0.0	
+ an absence discounting only what preceded it			.736	+0.4	
+ a departing cart read as transport			.976	+24.0	
+ an inspection revealing the carts parked there			.964	−1.2	

Table 9: The necessity claim with its matched control. Same grid as Table 8 (seeds 20000–20099, levels 2–6, 500 episodes per cell, hidden-truth grading). “engine, no multi-anchor” is the leave-one-out arm of data/artifacts/ablation_loo.json; the arm without repetition decay reads .806/.876/.890 at three, four and five actions and the arm without transport hand-off .890/.980/.998, inside the shipped arm’s interval throughout. The three planner columns are the exact dynamic programme of §4.1 over three beliefs: the plain filter’s, a belief that reads the same relations the engine reads, and the same with the generator’s own transport rate supplied as a prior (matched_planner_sweep.json). The plain column reproduces belief_planner_sweep.json cell for cell. The last panel is the cumulative ladder at four actions on the same 500 episodes per cell, in points of correct rate. Wrong commitments: 0 in every cell of every arm.

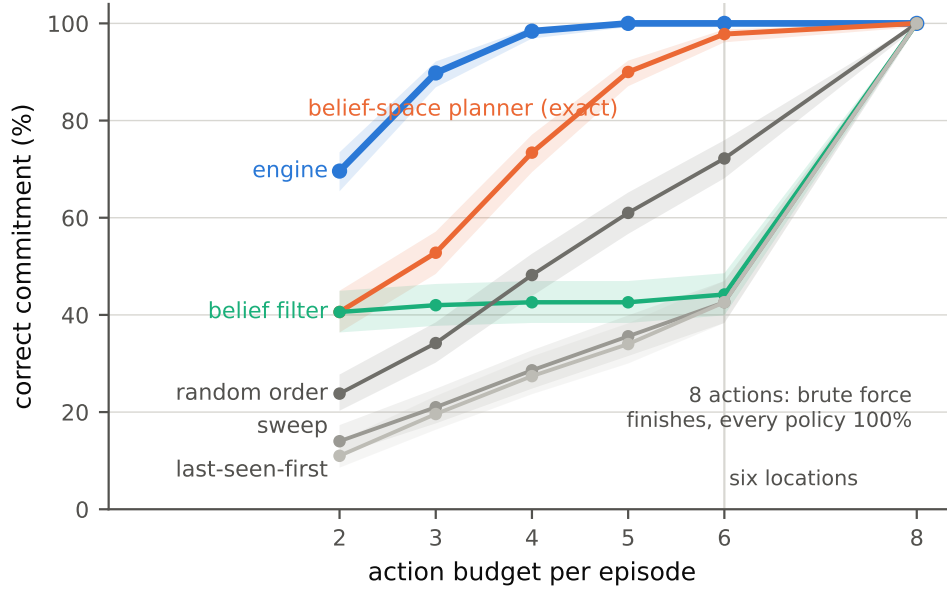


Figure 2: The benchmark below the exhaustive budget. Correct-commitment rate against action budget, 500 episodes per point, bands are Wilson 95% intervals. Brute force finishes at eight actions and every policy reaches 100%; below six, the scripted policies lose most episodes while the engine keeps them. The exact belief-space planner, optimal lookahead over the filter’s own belief, closes about half of the filter’s gap to the engine and none of the rest. The belief filter’s plateau is the cost of argmax-posterior without transport handoff; the random walk’s slope is plain coverage.

4.2 A family designed without reading the engine

On a scenario family designed under a reading restriction, the engine led the three scripted walks at every budget, trailed the belief-based planners between two and six actions, and was the only policy that did not finish at the exhaustive budget. That failure named a defect which is now fixed and re-measured on seeds held back for the purpose. The controls are the same seven policies as §4.1, including both exact planners. Every number in this subsection is in Table 10.

The family. Seven rooms on a corridor with three spurs, so travel costs real time; three containers, two of which travel and one of which never moves; three or four people and eight to twelve objects from name pools the warehouse family does not share. Six levels add, in turn, a two-person relay, two plausible carriers at once, objects that end inside containers, a machine log that is confident and wrong at a stated rate beside a hand-held recorder whose clock runs six minutes slow, a level on which the object never moves and every movement report is about something else, and a level that combines them. The exhaustive budget is ten actions, seven rooms and three containers, and every level is solvable at ten. The design, the branch rates and nine predictions about how the policies would order were written down before any policy ran (docs/HELDOUT_GENERATOR.md); their author read the world model, the executor, the baselines and the evaluation method and did not open the engine’s reasoning, planning, learning or configuration code. Of the nine predictions, three held, three held in part, two failed and one was not assessed. One of the two that failed was the prediction of the final ordering of the policies.

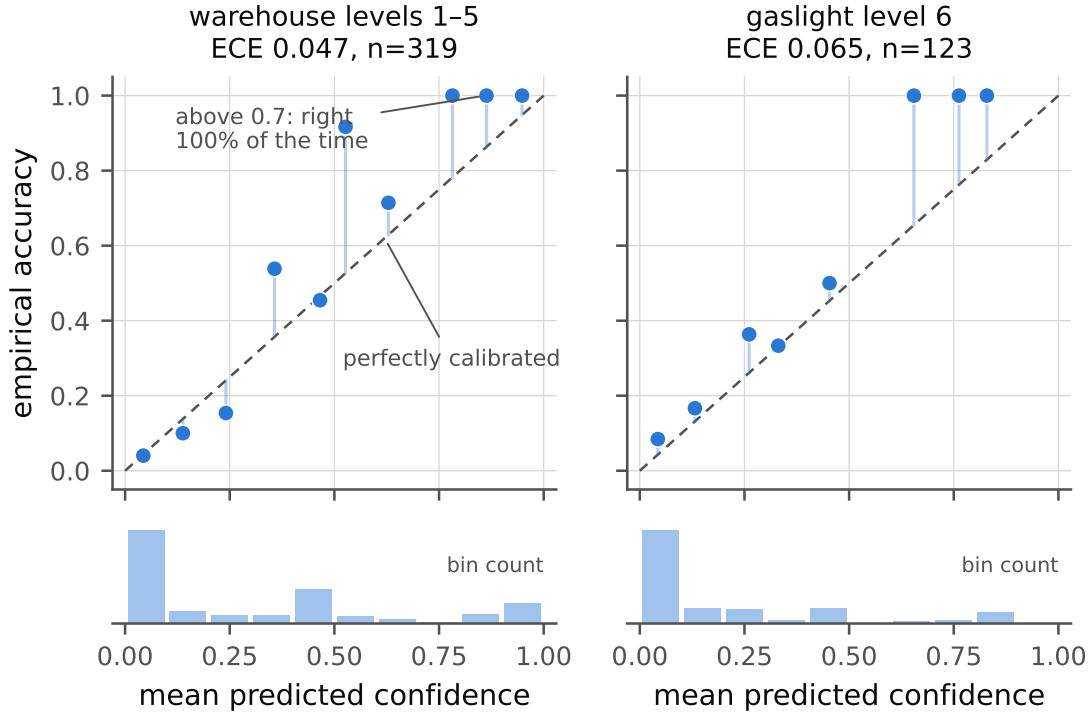


Figure 3: Reliability diagrams with bin counts, from data/artifacts/calibration.json. Dots above the diagonal are underconfident. The engine stays conservative under gaslighting: high confidence is never wrong; the miscalibration that exists sits in the low-confidence bins.

Found: the engine was the only policy that did not finish. At the exhaustive budget the engine reached 0.970 on the development seeds against 1.000 for all six other policies, and the intervals are disjoint. At four actions it reached 0.880, behind the matched planner’s 0.908 and the plain exact planner’s 0.898 and well ahead of the scripted policies. At the exhaustive budget the whole of the shortfall was one level and one branch: the level whose machine log is confidently wrong, on the episodes where it is wrong. On that level at four actions the engine scored 0.50 against the plain filter’s 1.00. On those episodes the engine looked in the same room twice while rooms it had never entered were never offered to its planner at all: 146 second looks across 600 episodes at the exhaustive budget, and a mean of 4.2 distinct rooms of seven on the episodes it failed.

Fixed: the robot’s own looks decide when to widen. The diagnosis was that the rule which widens a refuted search asks a question about a decayed score, and a score that decays can also come back, so the widening could fire and then un-fire with nothing learned in between. The rule now asks a question about the ledger: once every lead the evidence gave the robot has been checked by a look of the robot’s own, every place it has never looked in becomes a candidate and stays one until it is looked in. A report is not a look and neither is somebody else’s sensor. Being a fact about the ledger rather than about a number, it cannot un-fire, and one look per lead is enough, so the robot never has to look somewhere twice to earn the right to look somewhere new. The change ships behind a switch whose default reproduces every number measured before it.

Re-measured, on seeds held back. With the rule on, the engine reaches 1.000 at the exhaustive budget on the development seeds and 1.000 on validation seeds 70100–70199, which were not run until the fix was final. At four actions on the validation seeds it reaches 0.888, against the matched planner’s 0.908 and the plain exact planner’s 0.890. Second looks over the development range fall from 146 to 11 at the exhaustive budget. On the warehouse family the rule changes nothing: 0 of 3,000 episodes differ, action sequence included. The cost is stated once and it is real: on the relay level the fix loses episodes at the middle budgets, 0.95 to 0.89 at four actions on the validation seeds, because the room a second carrier walks to reaches the planner only through the movement model and its lead is diluted once unvisited rooms compete for the same actions. Both are back to 1.00 by eight actions.

What stays broken, and what the family does not test. The level with the slow clock keeps one episode class the engine cannot solve by reasoning: a carrier’s movement record is stamped six minutes before the association it should follow, so the chain to the right room is never built and the room is reached by coverage instead. That is a property of the data and the ordering rule that rejects a badge record from before a hand-off, and it is not fixed. Across 37,800 episodes on this family no policy committed wrongly, and that figure is weaker than it looks: this simulator writes a confirming sighting only where the object really is, so no policy *can* commit to a wrong room here. The family grades finding, not commitment discipline, and the batteries that grade commitment discipline are §4.5 and §4.7.

seeds	budget	engine		exact planner	matched	belief		
		shipped	coverage fix	plain belief	planner	filter	sweep	random
development	4	.880	.905	.898	.908	.810	.453	.405
70000–70099	10	.970	1.000	1.000	1.000	1.000	1.000	1.000
validation	4	.875	.888	.890	.908	.795	.440	.427
70100–70199	10	.965	1.000	1.000	1.000	1.000	1.000	1.000
<i>the level whose machine log is confidently wrong, development seeds</i>								
level 4	4	.50	.67	1.00	.49	1.00	.57	.44
level 4	10	.82	1.00	1.00	1.00	1.00	1.00	1.00
<i>the relay level, validation seeds: the cost of the fix</i>								
level 1	4	.95	.89					
level 1	8	1.00	1.00					

Table 10: The held-out family. Correct rate under hidden-truth grading, 600 episodes per cell over six levels and 100 seeds. Wilson 95% intervals at the exhaustive budget: engine as shipped .970 [.953, .981] on the development seeds and .965 [.947, .977] on the validation seeds, against 1.000 [.994, 1.000] for every other column, disjoint in both ranges. The coverage fix defaults to the shipped behaviour and changes 0 of 3,000 warehouse episodes. Wrong commitments: 0 in every cell of every arm, 37,800 episodes in the first sweep alone, a figure this simulator cannot falsify (see the text). data/artifacts/heldout_sweep.json, heldout_sweep_fix.json; docs/HELDOUT_GENERATOR.md.

4.3 Against LLM agents

Uncertainty 1 asked whether the engine could match LLM agents on temporal, identity and causal reasoning from unreliable sensors. Over 48 episodes under the fairness contract of §3 it does, at about three orders of magnitude lower latency, and it is the only agent in the comparison that leaves a trail a checker can replay.

The engine’s ~5 ms is the campaign’s per-episode wall-clock on an unpinned machine, and Table 21 gives the pinned-core profile beside it. The LLM figures are end-to-end wall-clock through the provider’s

agent	correct (L2–5)	avg. steps [†]	gaslight L6	wall-clock/ep	proofs
robot_mind engine, session 1	48/48	1.75	40/40	~5 ms	100% checked
qwen2.5-7B basic (local), session 1	40/48	3.73	–	~11 s	none
qwen3-8B thinking (local), session 1	45/48	2.07	–	~94 s	none
deepseek-chat (API, strong), session 1	48/48	1.94	–	~9 s	none
robot_mind engine, session 2	48/48	1.83	12/12	~5 ms	100% checked
qwen2.5-7B strong (local), session 2	39/48	3.62	–	~11.7 s	none
gemini-3.7-flash (API, strong), session 2	48/48	1.62	12/12	~10 s	none

Table 11: Head-to-head under the fairness contract (Figure 4). Each LLM row sits beside the engine figure of its own measurement session, on the same 48 episodes. The two session-2 LLM rows were re-measured because their session-1 artifacts were not retained; qwen-strong reproduced its session-1 figures unchanged and gemini reproduced 48/48 with slightly different step and latency figures. The engine’s step count differs between the sessions because the session-2 build carries its learned rule. [†]LLM step averages are over successful episodes only (over all 48 the qwen-basic figure is 4.44); the engine’s coincides because it never fails. Gaslight cells: the engine’s 40/40 is its 40-episode L6 battery; its 12/12 and Gemini’s 12/12 are the 12 head-to-head seeds, where the engine takes 3.25 steps to Gemini’s 2.50. Wilson 95% intervals: 48/48 [0.93, 1.0], 45/48 [0.83, 0.98], 40/48 [0.70, 0.91].

API, with network, queueing and the tool loop included. The ratio is therefore about $2,000\times$ per episode, not a hardware-normalised inference benchmark.

The strong scaffold coaches the 7B model in the engine’s own strategy, to control for prompting. It does not change the result: the difference is one episode, inside binomial noise at $n = 48$. Knowing the strategy is not the bottleneck at this model scale; holding calibrated belief state across steps is.

Between 8B and the two hosted models the measured gap vanishes. Both hosted models reach 48/48, and Gemini stays perfect under the L6 gaslighting campaign, at fewer steps than the engine. By the same noise standard the best local model’s shortfall is not significant at $n = 48$ (Fisher exact $p = 0.24$). The precise claim is that no accuracy gap is *detectable* for the two hosted agents at this sample size. The step difference is the engine’s verify-before-trusting discipline: one inspection that disproves the planted tip, recorded on the ledger. Gemini ignores the tip on judgment, correctly, and cannot afterwards show why that was safe.

The gaslight campaign (L6) adds an adversary who knows the true final location and plants a confident fake sighting steering away from it, with further moves listed in Appendix B. Over 40 seeded episodes the engine goes **40/40** with 100% proof validity, at about 1.2 more actions than it spends undeceived on the same seeds. The average moves with the seed set (Appendix B), and this 40-episode run is regenerate-to-verify. Fresh reports are verified before being trusted. In the ledger, conflicting sightings of one object are marked CONTRADICTED by a fixed rule rather than averaged, and multi-anchor hypothesis generation keeps the true carrier chain alive. Deception raises the action count and leaves accuracy untouched.

In these measurements the accuracy gap closes for the two hosted agents. The latency gap and the absence of a replayable trail remain (§5). Those 48 episodes come from one generator and one set of worlds. The next question is whether the same build travels.

4.4 One build, unchanged, in other worlds

One build left the warehouse with nothing changed and was correct wherever it committed, on an external text benchmark and in six simulators; Table 13 carries every count. The baselines in each are an evidence-ignoring sweep and a seeded random order under identical adapter mechanics. Most of these evaluations are portability and integration certifications and are introduced as such: under uniform priors and an always-present target they grade completeness and correct commitment, and a complete searcher that commits only

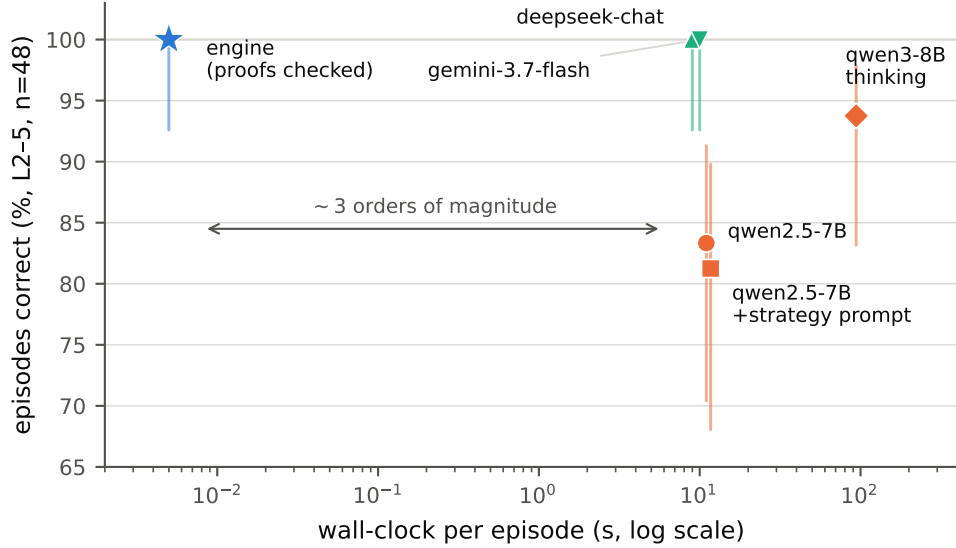


Figure 4: The capability ladder of Table 11: accuracy against per-episode wall-clock, with Wilson 95% intervals. Local 7–8B models trade latency for accuracy and still fall short; the two hosted API models match the engine’s accuracy at $\sim 2,000\times$ its latency. Only the engine’s answers carry machine-checked proofs.

on sightings passes them by construction. Two do more. The planted-tip and staged-decoy rounds make a wrong commitment possible, and the reduced command allowances of ALFWorld separate the policies where the standard allowance does not.

ALFWorld’s unseen split. At the standard command allowance the engine solves every unseen task, and so does almost everything else: a seeded random walk under identical adapter mechanics scores 133 of the same 134. The warehouse and its levels are worlds we built, so v0.4 also entered ALFWorld [46], the text-world twin of ALFRED [45]. It has 134 unseen household tasks, a standard 50-step budget, and success judged by the game. The adapter turns template text into claims and engine actions into the game’s admissible-command menu, and hand-codes six task recipes (the fixed clean/heat/cool/examine/place mechanics). Which receptacle to try next, and when to give up, are the engine’s calls. Development used the train/seen splits. The first run scored **129/134** one-shot. Tracing the seen-split two-object losses exposed an engine defect, a stale class-level sighting that resurrected already-checked rooms (Appendix B gives the mechanism). The fix is belief hygiene, and it was developed and validated on the seen split only. The unseen split was then evaluated a second time, a second run of the test set: **134/134**. That matches the only other verified perfect score, AutoRefine’s, with no language model running at inference time and milliseconds per decision (Table 12, lower panel). The engine’s edge over a memoryless random walk at this allowance is +1 game unseen and +4 seen, consistent in direction and bounded by a benchmark at ceiling.

ALFWorld under a reduced command allowance. The standard allowance does not separate the policies; a reduced one does, and on the unseen split it separates against the engine until about 25 commands. The upper panel of Table 12 runs all three policies at 10, 15, 20, 30 and 50 commands on both splits, one run per unseen cell. On the seen split the engine beats the fixed sweep at 20, 30 and 50 commands and nothing separates it from the random walk at any allowance. On the unseen split the sweep beats the engine at 10, 15 and 20, by 34 games at 20, and the random walk beats it at 20; the engine leads at 30.

That result has one cause and it was found. With no evidence about a household it has never seen, every receptacle carries the same score, so the choice falls to whatever settles a tie, and the acting policy settled it on the alphabetically last name while the scripted sweep takes the first. 108 of the 134 unseen hunts walked the receptacle list backwards. Under the forward order, with the engine’s own machinery otherwise untouched, every one of the 57 games the sweep wins at 20 commands and the engine does not come in at or under 20 commands, so that single inversion accounts for all of them. Two facts bound how much the allowance curves can carry. The unseen split is four households holding 134 task instances, against 76 households on the seen split, so the curves are four households’ worth of evidence about search order. And the direction in which a tie is settled is arbitrary in both policies: forward alphabetical is not better in principle, it is a different arbitrary order that happens to suit where ALFWorld puts things. What the engine brings that neither walk does is visible in the same traces: it never walks to a place twice before it finds the thing, it re-treads 8.2% of its walks over a whole game against the sweep’s 13.7% and the random order’s 11.8%, and its second hunt in a two-object task costs 3.94 commands against 5.11 and 10.37. That bookkeeping is what carries it past both at 30 commands. It does not make it cheaper: over the games each policy wins at the standard allowance, the random walk on the unseen split averages 18.38 commands against the engine’s 19.51, so search cost does not favour the engine on this benchmark either. The 134/134 at the standard allowance stands.

Fixed, and re-measured at every allowance. Two changes were built and measured, and Table 12 is their acceptance test. A tie between equally supported places is now settled by the episode’s own seed rather than by an identifier, so the order of the names carries no information. Beside it, a class prior the mind learns from the training split out of its own confirmed finds, advisory and consulted only where there is no evidence, on the template of the lifelong-mind result of §4.6. One mind played the 200 training-split games once and recorded 258 finds the game itself confirmed over 44 kinds of thing, never an answer key and never either evaluation split, and was then loaded read-only. Both are default-preserving switches, and each split was run once per allowance under each. On the unseen split the seeded settlement alone recovers most of the deficit, and with the class prior as well the engine beats the fixed sweep at every allowance, by 28, 35 and 16 games at 10, 15 and 20 commands. The knee between 20 and 30 commands is gone in every task kind. The cost is stated once: the seeded settlement gives back five games at the standard allowance on the seen split, 138 to 133, and the class prior recovers one of them. Neither switch is the shipped default in this build. The defect was found by reading traces of the games the engine lost on the unseen split, and the class prior was trained on the training split alone, in one pass, from finds the robot made and the game confirmed.

A fourth arm settles what a name is worth. Run with the forward alphabetical order the sweep takes, and the engine’s own machinery otherwise untouched, the engine is the worst arm on the seen split at every allowance above ten, losing to the shipped settings by 19, 20 and 13 games at 20, 30 and 50 commands. The two splits disagree about that order for the reason the household counts give, so the measurement, not the assertion, is what says a name is not evidence.

Six simulated environments. Table 13 lists every embodied evaluation with its sensing abstraction, its result and its scope, and Figure 5 plots each row with its Wilson interval.

What is real in each battery. The abstractions are stated per battery rather than in one blanket sentence, because they differ along six axes and no battery is abstract on all of them (Table 14). Three readings are worth taking from it. No real drive base runs anywhere, and no battery uses a real multi-object tracker; the two axes a deployed robot is hardest on are the two this work stands in for. Under Nav2 the localization is real and the sensor’s vantage is not: SLAM Toolbox localizes the robot and Nav2 plans on its map, while the frustum and the camera lane both look from the body’s physics pose, so measured drift and SLAM error are reported beside the result and do not move the sensor. And the real-pixels row is the one place where the

<i>games won at a reduced command allowance</i>						
split	policy	10	15	20	30	50
valid_unseen ($n = 134$)	engine, as shipped	22	46	68	127	134
	+ a seeded settlement	46	77	96	121	134
	+ the class prior too	76	101	118	128	134
	fixed sweep	48	66	102	120	129
	random order	30	56	83	115	133
valid_seen ($n = 140$)	engine, as shipped	34	68	85	116	138
	+ a seeded settlement	48	72	91	118	133
	+ the class prior too	85	100	110	126	134
	fixed sweep	33	54	63	96	124
	random order	36	66	83	115	134
	forward alphabetical	38	59	66	96	125
<i>published unseen-split scores, verified against the cited papers</i>						
system	model at inference		attempts	score		
AutoRefine [35]	GPT-5.6-terra		single	134/134		
robot_mind (this paper)	none		single	134/134		
AutoManual [8]	GPT-4-turbo		single	97.4%		
Reflexion [44]	GPT-4		12 trials	97%		
ReAct [35, Table 1]	current backbone		single	79.9%		
ReAct [55, Table 3]	PaLM-540B		best of six	71%		
BUTLER [46]	none		best of eight	37%		

Table 12: ALFWorld. Upper panel: data/artifacts/alfworld_budget_sweep.json for the shipped, sweep and random rows and alfworld_budget_sweep_fix.json for the rest. Fifty commands is the benchmark’s standard allowance. Unseen cells were run once and never re-run; the unseen engine and random cells at 50 are quoted from the committed standard-allowance artifacts. Paired exact McNemar on the same games, unseen split: as shipped the sweep beats the engine at 10, 15 and 20 ($p = 0.0007, 0.019, 0.0002$) and the random order at 20 ($p = 0.036$), and the engine beats the random order at 30 ($p = 0.008$); with the seeded settlement nothing separates the engine from the sweep at any allowance, and with the class prior as well the engine beats it at 10, 15 and 20 ($p = 5.0 \times 10^{-4}, 5.1 \times 10^{-6}, 7.0 \times 10^{-3}$). Seen split: as shipped the engine beats the sweep at 20, 30 and 50 ($p = 0.005, 0.005, 0.001$) and nothing separates it from the random order ($p \geq 0.29$ at every allowance); the forward alphabetical arm loses to it at 20, 30 and 50 ($p = 0.013, 0.0037, 0.0010$). Lower panel: BUTLER learns the task mechanics this adapter hand-codes, so that row scopes the search policy only; ReAct’s 57% average over its six seeds is the figure behind its 71% best-of-six.

simulator / task	n	sensing abstraction	result	scope
AI2-THOR fetch	96 ep.	visibility flags + NBV poses	95/96, prec. 95/95	placements not bit-stable
AI2-THOR, real detector	96 ep.	YOLO-World on 1080 ² frames	83/96, prec. 83/83	8 thin/small-object recall, 4 mislabels, 1 sealed in a box; prec. by construction
AI2-THOR, lifelong mind	30 ep.	visibility flags, one persistent mind	95 $\rightarrow$ 10 actions on habit episodes	one house, oracle sensing, habit spot selected
SubT artifact search	12 ep.	section-granular state read	12/12 scored	9 distinct searches
Hospital fetch	8 ep.	6 m live state read	8/8 correct	depth 1–21 varies by draw
Hospital planted tips	12 ep.	6 m live state read	12/12; true tip $\rightarrow$ 1.0 visits	false-tip edge inside noise
Hospital staged decoy	6 ep.	6 m live state read	6/6, 0 fooled	class-match policies fooled 4–5/6
Hospital staged decoy, 30-seed power run	30 ep.	6 m live state read	30/30, 0/30 fooled	pre-registered; class-match policies fooled 16/30 and 26/30
Mars rover terrain (Space ROS demo on Jazzy)	12 ep.	logical 8 m mast sensor, 169 stations	4/4 per arm (engine, sweep, random)	uniform priors: completeness only
Seabed survey (Project DAVE)	12 ep.	sonar range–confidence curve, 70 stations	4/4 per arm	uniform priors: completeness only
Gazebo box hunt	4 ep.	logical camera	4/4 correct	4-bay world
ROS 2 topic demo, diff-drive	4 ep.	logical-camera frustum over bridged poses	4/4 correct	driven over / cmd_vel, no teleports
ROS 2 packaged stack, demo driver	4 + 1 tip ep.	frustum at the odometry pose	4/4 + tip redirected; 20/20 checks	odom drift to 27 m; MoveIt not run
ROS 2 stack on Nav2 + SLAM Toolbox	4 + 1 tip ep.	lidar SLAM localizes; frustum from physics pose	4/4 + tip redirected; 21/21 checks	same sequences; drift ≤ 0.10 m
ROS 2 stack on Nav2 + YOLO-World	4 + 1 tip ep.	RGB camera, YOLO-World, pose-projection association	4/4 + tip redirected; 26/26 checks	finds: same sequences; tip reordered by one nav timeout; 0 wrong bindings
Real pixels: phone camera in a house, 4th sitting	36 + 36 tip ep.	hand-held 1080p clips, YOLO-World per-zone maximum, no association	36/36; tips 18/18 true, 18/18 false	one house, camera carried by hand, zones declared; sweep 24/36, random 27/36

Table 13: Embodied evaluations: one unchanged engine, per-adaptor abstractions stated. Locomotion is teleport-abstracted except in the ROS 2 rows, where a diff-drive base is driven with no teleports, and absent in the real-pixels row, where a person carried the camera; none of these numbers is comparable to full-stack embodied leaderboards (ObjectNav, ALFRED vision track, SubT systems track). The plain SubT and hospital protocols grade search completeness and correct commitment only (limitation 10); the planted-tip variant below carries the discriminating embodied evidence.

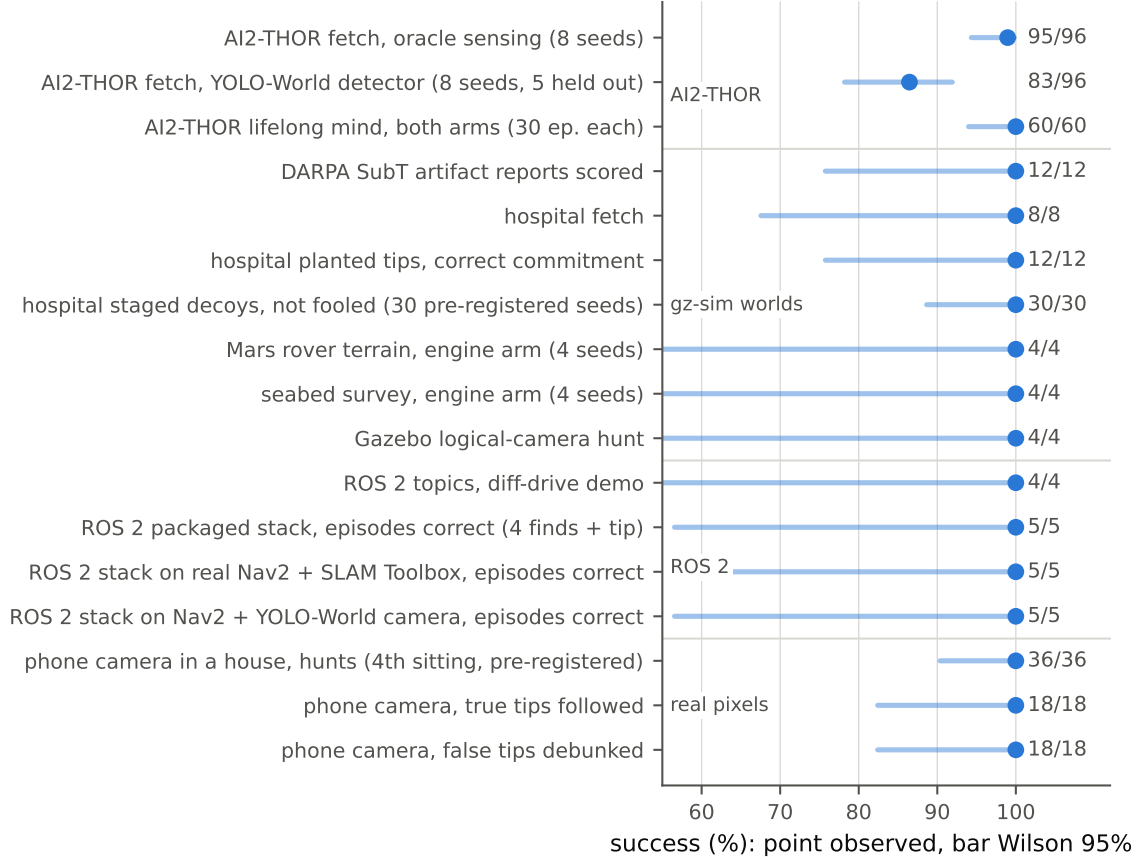


Figure 5: One unchanged engine across every embodied evaluation in this paper. Points are observed success, bars are Wilson 95% intervals; k/n at right. The detector row is the eight-seed battery of §4.5, five of the seeds held out; the ROS 2 rows are the checked episodes of each lane (four staged finds and the false-tip episode), the typed-client check counts (20/20, 21/21, 26/26) being in the text; the last three rows are the pre-registered fourth sitting of the real-pixels battery, a hand-held phone camera in a house.

battery is *harder* than every simulator row on an axis: it has no instance association at all, so a hallucinated detection can create a false sighting and drive a wrong commitment, and that is what the battery grades. Limitation 12 states the boundary this table draws: no physical robot has been run.

AI2-THOR find-and-fetch. Twelve unseen iTHOR scenes [25] span all four room types, with placements shuffled per episode and success graded by the simulator’s `PickupObject` signal. Vantages come from an occlusion-tested next-best-view planner [12, 42]. Eight seeds give **95/96** fetched, with every commitment matching ground truth. The one miss, a plate on a side table on seed 314, exhausted the budget with nothing wrongly committed: a perception-coverage residue, not a reasoning error. Precision is 100% by construction, since a commitment fires only on a sighting and oracle visibility flags make every sighting true.

DARPA SubT Virtual artifact search. The `osrf/subt` circuit practice worlds [11] run under DARPA’s scoring rule: a report scores iff the artifact class is right and the position is within 5 m of truth. The engine gets the tile list as bare locations and hunts a seeded target class per episode. Four worlds (two tunnel, one urban, one cave) and three seeds give **12/12** reports scored. Sensing is section-granular and reads simulator state, so

battery	locomotion	viewpoint	detection	identity	mapping	physics
ALFWorld text	n.a.: a go to command	n.a.: no camera, no pose	abstracted: objects parsed from the game's text	abstracted: the game's own noun phrase	abstracted: receptacle list, no coordinates	none: a live TextWorld game
AI2-THOR fetch	abstracted: teleport	abstracted: precomputed NBV list	abstracted: the simulator's visibility flag	abstracted: the simulator's instance id	abstracted: receptacle graph	real : Unity PhysX
AI2-THOR + YOLO-World	abstracted: teleport	abstracted: same NBV list	real : YOLO-World on 1080 ² renders	abstracted: IoU against the simulator's instance masks	abstracted: receptacle graph	real : Unity PhysX
AI2-THOR lifelong	abstracted: teleport	abstracted: same NBV list	abstracted: visibility flag	abstracted: instance ids, plus the robot's own confirmed pickups	abstracted: receptacle graph	real : Unity PhysX
DARPA SubT	abstracted: teleport	none: a proximity read of exact poses	abstracted: proximity read at a fixed confidence	abstracted: the scene service's model name	abstracted: section graph parsed from DARPA's world	real : gz-sim
AWS hospital	abstracted: teleport	abstracted: a 6 m bubble at the teleported pose	abstracted: proximity read at a fixed confidence	abstracted: the simulator's instance ids	abstracted: furniture spots parsed from the world	not determined: the world file is outside this repository
Mars, seabed	none: placement only	sim pose, read back and verified	abstracted: a declared footprint, a declared sonar curve	abstracted: the simulator's instance name	abstracted: a declared station grid	real : gz-sim
Gazebo box hunt	abstracted: teleport	sim pose, commanded	abstracted: logical-camera frustum over physics state	abstracted: model names	abstracted: four declared bays	real : gz-sim
ROS 2 demo-driver lanes	simulated diff-drive over /cmd_vel, no teleports	odometry	abstracted: frustum over bridged poses	abstracted: model names	odometry, declared bays	real : gz-sim
ROS 2 on Nav2 + SLAM	real stack : Nav2 drives the base	sim pose: the sensor looks from physics	abstracted: frustum over physics poses	abstracted: model names	real : SLAM Toolbox	real : gz-sim
ROS 2 on Nav2 + YOLO-World	real stack : Nav2 drives the base	sim pose plus a fixed camera mount	real : YOLO-World on 640 × 480 renders	abstracted: IoU against the entity's projected physics box	real : SLAM Toolbox	real : gz-sim
Real pixels: phone in a house	none: a person carried the camera	abstracted: declared zones, no pose of any kind	real : YOLO-World on real 1080p footage	none : no association step exists	abstracted: six declared zones	none: video replayed from disk

Table 14: What is real and what stands in, per battery and per axis. **real** names a real component doing the work. *sim pose* is the simulator's own pose supplied to the adapter rather than estimated by the robot. *abstracted* is a teleport, a precomputed vantage list, a frustum or proximity test over physics state, a label or instance id from the simulator, or a declared graph of rooms, bays, zones or stations. *not determined* means this repository does not fix the answer. Every cell is cited to its adapter, harness, node or world file in docs/EMBODIED.md and the adapters' own docstrings. Rows 2–12 all launch a live simulator process and read its state back; the phone row is replay of pre-recorded video with no simulator at all.

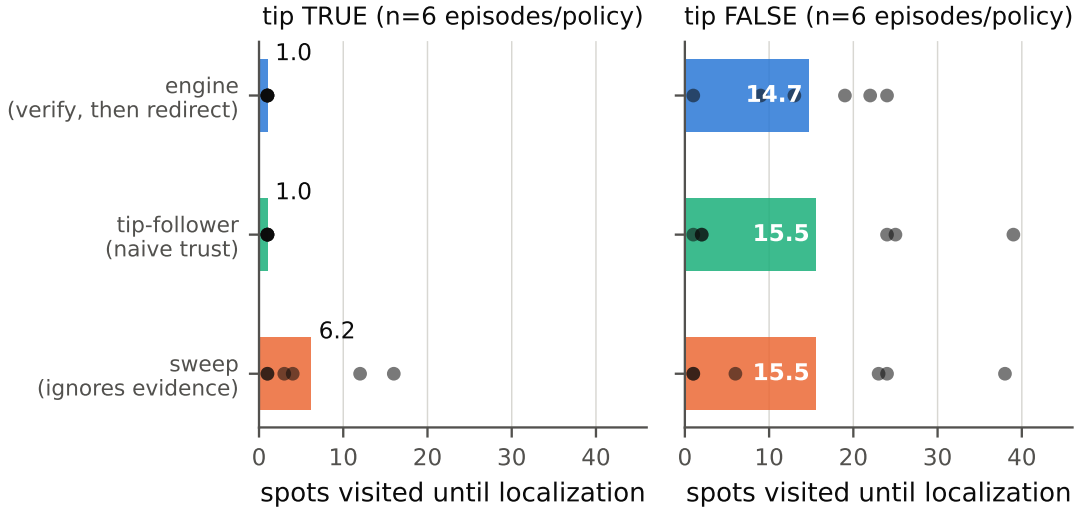


Figure 6: The planted-tip hospital protocol: mean visits per policy with per-episode draws as dots (12 seeds per policy; 6 true, 6 false tips). A true tip collapses the engine’s search to one visit; a false tip costs one verification and a redirect. No policy ever commits wrongly, and only the engine can show *why* the tip was disbelieved.

the position half of the rule is inherited from the sensor and the graded quantity is the search. Duplicate-class seeds are deterministic replays, and targets are always present, so this battery certifies portability rather than discriminating among policies.

Hospital fetch and planted tips. The AWS RoboMaker hospital world has, to our knowledge, no prior published fetch benchmark. Per episode the evaluator relocates one unique-instance item beside a seeded furniture spot, and the engine, told only the class, must find it among 102 real hospital furniture locations. Eight seeds give **8/8** correct. The planted-tip variant adds an anonymous REPORTED claim to the ledger naming a spot, true with seeded probability one half. It runs the engine, an evidence-ignoring sorted sweep and a naive tip-follower on twelve seeds, six true and six false tips. All three find every target, so visits are the discriminating quantity (Table 15, Figure 6). A true tip collapses the engine’s search to one visit, matching the tip-follower and six times fewer than the sweep. With a false tip it checks the spot once, records the absence and redirects. No policy committed to a reported-but-unverified location anywhere in the campaign, and the false-tip visit counts do not separate the policies at this sample size: one debunked tip is too little evidence structure for belief arithmetic to beat a fixed fallback order.

Staged decoys. The decoy round makes wrong commitment possible. For each false-tip seed a same-class look-alike is staged beside the lied-about spot and the briefing names the specific asset id. A fourth decider joins: a tip-follower that checks instance identity but carries no belief machinery. The pre-registered thirty-seed run gives the engine **30/30** correct and **0/30** fooled, at the same false-tip cost it pays without a decoy (Table 15). The identity-checking control is never fooled either, so what separates safe from fooled is commitment discipline rather than belief machinery, and the two class-committing policies are fooled in most of their episodes. On the six seeds shared with the first round the engine’s visit count reproduced on four and moved on two (Appendix B gives the counts and the likeliest cause).

policy	planted tip, visits		decoy, 6 seeds		decoy, 30 seeds	
	true	false	correct	fooled	correct	fooled
engine	1.00	14.67	6/6	0/6	30/30	0/30
tip-follower, checks instance identity	—	—	6/6	0/6	30/30	—
tip-follower, class only	1.00	15.50	—	5/6	—	26/30
evidence-ignoring sweep	6.17	15.50	—	4/6	—	16/30

Table 15: The two hospital rounds in which a wrong commitment is possible. Visits are means over the six true-tip and six false-tip seeds; the identity-checking control joins only for the decoy round. The engine’s decoy visit counts are 14.7 over the six seeds and 11.7 over the pre-registered thirty, which is its false-tip cost without a decoy. Wilson 95% on 0/30 fooled is $[0.00, 0.11]$; on 0/6 it is $[0.00, 0.39]$, which is why the thirty-seed run was run. The false-tip visit counts do not separate the policies (paired sign-flip permutation over the six false-tip seeds, $p = 0.91$ against the sweep and $p = 1.0$ against the tip-follower). `hospital_tips_twin.json`, `hospital_tips_twin_30.json`.

Planetary and underwater analogues. The same adapter contract was carried to the Space ROS Mars rover demonstration’s Curiosity terrain (169 stations) and to Project DAVE’s ocean-objects seabed (70 stations). Engine, sweep and random-order arms share executor, budget and sensor. Every arm finds every target, with zero wrong commitments in either world (Table 13; Appendix B gives the visit counts and the intervals). As in the plain SubT and hospital protocols (limitation 10), uniform priors and an always-present target grade completeness and correct commitment only. A random order can beat the engine by luck at $n = 4$, so no efficiency claim is made. The batteries show transfer of the search machinery, unchanged, to planetary and underwater station maps.

Transfer holds under the adapter contract. What the contract still abstracts away is the robot itself, and the next subsection puts real components in its place.

4.5 The robot’s seams

With a real detector on the perception seam and a real navigation stack on the navigation seam, every commitment the engine made was correct, and the engine binary was unchanged in every run below. The controls are the same batteries with the simulator’s own oracle in the slot, and, on the real-pixels clips, a zone-by-zone sweep and a seeded random order reading the same detections. Uncertainties 5 and 6 asked whether commitments derived from a real detector’s raw output stay correct, and whether the decisions survive a navigation stack’s timeouts, aborts and drift. A deployed robot reaches the engine through three seams. Perception turns pixels into typed claims. Navigation turns “inspect here” into motion in a map. Time matters because a mind that runs for weeks is not the mind that runs for one episode. This subsection crosses the first two with a real component in each slot, and §4.6 crosses the third. YOLO-World stood in for AI2-THOR’s visibility oracle. Stock Nav2 with SLAM Toolbox in simulation stood in for the demo driver. A last battery films a house with a hand-carried phone, with no simulator anywhere in sensing.

A real detector in AI2-THOR. The same twelve scenes and all eight seeds ran with the simulator’s visibility flags replaced by YOLO-World [9] on the rendered frame at 1080^2 . The detector is prompted once with the 62 iTHOR pickupable types and never told the target. A detection is bound to the rendered instance’s segmentation box at $\text{IoU} \geq 0.35$ through a fixed table of eleven visual-synonym groups. It enters the ledger at the detector’s own confidence once it passes a 0.05 claim gate. This association stands in for a tracker. The detector supplies localization and a coarse label, the simulator’s instance segmentation supplies identity, and the association can only accept detector output, never add to it. Five of the eight seeds were declared

seed	fetches	commitments correct	median actions	misses, each probed
7 (dev.)	12/12	12/12	15.5	—
11 (dev.)	11/12	11/11	61	fork inside a garbage can
23 (dev.)	8/12	8/8	13	two credit cards lying flat, keychain inside a garbage can, candle on a tank (mislabel)
42	11/12	11/11	20	fork inside a garbage can
99	12/12	12/12	25	—
137	11/12	11/11	19	butter knife inside a garbage can
256	8/12	8/8	17	butter knife on a dining table (mislabel, “Pen”), CD sealed inside a box on a bed, alarm clock on a shelf (never named), plunger on a shelf (mislabel)
314	10/12	10/10	19.5	pepper shaker on a dining table (mislabel, “Soap-Bottle” 0.92), credit card on a chair
dev. (3)	31/36	31/31	—	
held out (5)	52/60	52/52	19.5	
all (8)	83/96	83/83	19	

Table 16: The real-perception battery per seed: twelve scenes each, a target class drawn per seed, YOLO-World on 1080² frames. Seeds 7, 11 and 23 were the development seeds (synonym table, claim gate and confirmation ordering diagnosed on their misses); the five below them were declared before they ran and touched by no fix. Every commitment was correct; every miss was probed to the detector and reported not-found after the 250-action budget. Seeds 42 and 137 each had one scene rerun as a fresh draw after a simulator start-up crash. Wilson 95% on the recall: 83/96 [0.78, 0.92], the held-out seeds alone 52/60 [0.76, 0.93]. Mean actions per fetch is 30 against a median of 19, the gap being the hard cases of Figure 7(C).

before any ran and held out from every fix; the engine binary was unchanged throughout, and the adapter-side changes are listed in Appendix B and in docs/EMBODIED.md.

The held-out seeds fetched **52/60 with 52/52 commitments correct**, and all eight seeds together stand at 83/96 with 83/83 correct (Table 16, Figure 7). Association is gated by the simulator’s instance boxes, so a wrong-instance commitment cannot occur here: precision is by construction, and what 83/83 checks is the seam end to end. The search itself is measured by recall and by actions per fetch. Every miss was probed to the detector and reported not-found after the full budget, and every one of the thirteen is a perception failure rather than a reasoning error: eight sit below the detector’s recall from every reachable vantage, four are mislabels localized accurately and named as something else, and one, a CD sealed inside a box, was never rendered. The 480² first campaign, two start-up crash reruns, one adapter defect, each miss with its detector scores, and the operating point are in Appendix B.

ROS 2, lane one: the packaged node with the demo driver. The engine is packaged as a managed ROS 2 lifecycle node with a FindObject action and a nav2_msgs/NavigateToPose client, so Nav2 replaces the demo driver one for one. A typed certification client (an integration-test suite run against the live stack; the repository’s name for it is kept) passes 20/20 checks. Lifecycle gates hold. Four staged finds succeed with proof paths and grasp poses, and a false tip injected mid-episode names a bay already inspected and found empty; the engine re-verifies it once, abandons it, and finds the truth (Table 17). The demo driver’s stepwise commands let wheel odometry drift by tens of metres within the session, so this lane grades arrival and sensing in the odometry frame.

Lane two: stock Nav2 with SLAM Toolbox in simulation. The same stack then ran with navigation served by stock Nav2 in simulation and localization by SLAM Toolbox over a roof lidar (Appendix A

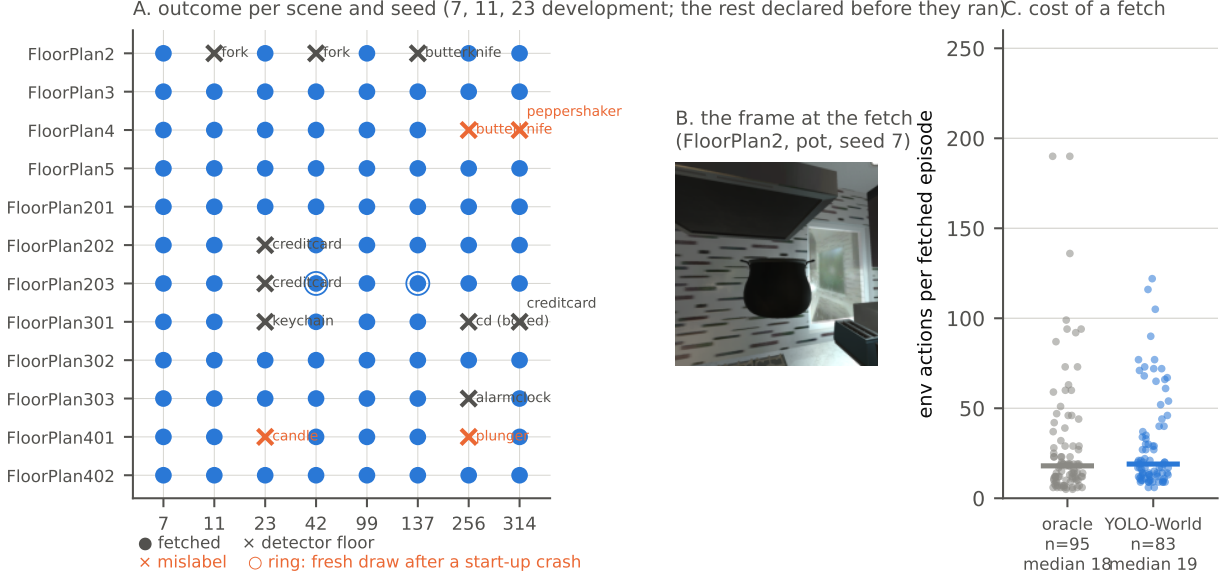


Figure 7: The real-perception battery. (A) Outcome per scene and seed, the target class drawn afresh per seed: filled circles are fetches; crosses are misses labelled with their object, gray for the detector’s recall floor on thin and small objects, orange for a mislabel; the CD marked boxed sat inside a box and was never rendered. (B) The 1080² render as YOLO-World saw it at the moment of a fetch (the detector’s boxes are drawn in the run animations of the artifact bundle). (C) Actions per fetched episode with oracle sensing (eight seeds, 95 fetches) and with the detector (83 fetches), one point per episode, bars at the medians (18 and 19): the detector costs actions on the hard cases, never commitments.

inventories the stack). The demo driver was not launched, and the cognition node changed only its goal frame. Result: **21/21 checks**. The four finds and the tip episode reproduced the same inspection sequences the engine chose against the demo driver, with localization accurate to within a tenth of a metre at every arrival (Table 17). Almost all the wall time per inspection is driving; the engine’s decision inside it takes a third of a millisecond (§4.8), and the Nav2 parameters are in Appendix B. A `NavigateToPose` goal that is rejected, aborts, or exceeds the node’s 120 s result timeout returns to the engine as a failed action. It costs one unit of visit budget and records no claim, and the engine re-plans from the unchanged ledger. The tip episode contains one such timeout, on the re-verification leg: five navigation attempts for four inspections.

Lane three: YOLO-World in the Sense slot under Nav2. The third set of checks keeps the Nav2 stack and swaps the Sense slot. A roof RGB camera (640 × 480, 1.4 rad) feeds a `yolo_sense_node` that runs YOLO-World with a fixed generic vocabulary that never names an entity. Each detection is bound to an entity by projecting that entity’s physics pose through the camera intrinsics at $\text{IoU} \geq 0.35$. This pose-projection association is the tracker stand-in, as the instance boxes were in AI2-THOR. The target and three distractors are 0.6 m cardboard cubes with one geometry and one texture. The target stands beside a distractor in three of the four finds and in the tip episode, so identity comes from geometry alone, which is where a wrong-instance binding becomes possible. One adapter-side rule was needed, of the same kind as the AI2-THOR confirmation gate. It adds an inspection-confirmation claim only when the engine’s own verdict resolves the target at the bay the robot stands at. The sensor node and the rule are specified in Appendix B.

Result: **26/26 checks**. The four finds were correct with the same inspection sequences as under the logical sensor. The tip episode verified and abandoned the tip and found the truth; a navigation timeout

episode	truth	inspection sequence (identical under both)	demo driver (odom frame)		Nav2 1.3 + SLAM Toolbox (map frame)		
			wall (s)	odom drift (m)	wall (s)	odom drift (m)	SLAM err. (m)
0	bay_3	bay_4, bay_3	23.4	0.76	49.9	0.032	0.049
1	bay_1	bay_4, bay_3, bay_2, bay_1	60.1	17.44	147.5	0.099	0.040
2	bay_4	bay_4	18.1	9.58	40.2	0.064	0.043
3	bay_2	bay_4, bay_3, bay_2	31.9	27.03	69.8	0.007	0.028
false tip: bay_4	bay_2	bay_4, bay_3, bay_4 (tip re-verified), bay_2	56.2	8.81	246.1 [†]	0.028	0.082

Table 17: One cognition node checked against two navigation backends (data/artifacts/embodied/ros2_stack_cert.json, ros2_stack_cert_nav2.json). Every episode is correct with a proof path and a grasp pose under both, and the engine’s inspection sequence is the same under both; under uniform priors that sequence is the fixed sweep bay_4, bay_3, bay_2, bay_1 cut at the truth, so the tip episode is the evidence-driven lane. Wall time is per FindObject goal, bring-up excluded. Odometry drift is the wheel-odometry pose against the physics pose at each arrival, cumulative across the session: the demo driver’s stepwise commands slip the sphere-wheeled body, DWB’s 0.5 m/s² ramps do not. SLAM error is SLAM Toolbox’s map-frame estimate against physics. Under Nav2 the frustum sensor looks from the physics pose, so neither localization column affects grading. [†]Includes one 120 s NavigateToPose result timeout on the re-verification leg, returned to the engine as a failed action (five navigation attempts for four inspections).

on its first leg reordered its inspections without changing any decision, and the reordering and its cause are in Appendix B. The detector could not separate the target from its look-alikes on the score alone, naming the target at 0.12–0.60 and the look-alikes at 0.32–0.65, and two same-mesh boxes shared the frame at an inspection in four of the five episodes. Every detection was nevertheless bound to its own projected entity, with **0 wrong-instance sightings**, and each confirmation followed the engine’s verdict (ros2_stack_cert_nav2_yolo.json; Figure 8). Drift and SLAM error stayed within a tenth of a metre, and the detector’s per-frame cost is in Appendix B.

Real pixels: a phone camera in a house. The batteries above still let the simulator tell the adapter which entity a detection belonged to, and this one removes that too. The owner filmed six declared zones of a lived-in house (a sofa, a desk, a kitchen island, a marble table, a blue bin, a navy bench) with a hand-held phone, placing six object classes over six rounds per sitting from a seed committed as a manifest before filming. An inspection decodes one zone’s clip, runs YOLO-World on every tenth frame, and enters the per-zone maximum in the ledger as an OBSERVED sighting at the detector’s own number. There is no instance id, no IoU gate and no association step, so a wrong commitment is possible, and that is what the battery grades. The filming procedure, the recording sheet and the sense model are in docs/PHONE_PROTOCOL.md, and the engine is briefed with the target class and the six zone names, nothing else. The baselines are a zone-by-zone sweep and a seeded random order that commit on the first accepted sighting, and a tip variant reports one zone at 0.75 per episode, half of them true, with a tip-follower as a third arm.

The phone adapter sets a sighting bar of 0.9, the loop’s own confirmation confidence, leaving the engine default unchanged and every native suite byte-identical. Below the bar a sighting is a hypothesis, the search goes on, and an exhausted search commits to its uniquely best-supported zone. The bar and the sense model were chosen on the graded footage of earlier sittings, then pre-registered with the manifest. Three graded sittings preceded the pre-registered fourth, and the sitting-by-sitting record, with the three fixes between them, is in Appendix B.

The fourth sitting was one run, with nothing changed after seeing it. The engine commits to the right

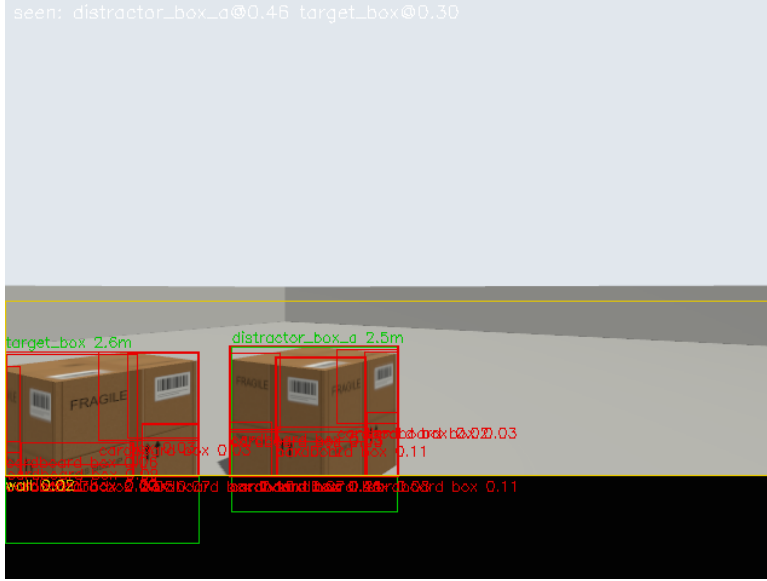


Figure 8: The camera lane, episode 1, the inspection of bay_1 that settled the find: the target box (left, staged 1.0 m beside the bay centre) and distractor_box_a (right) are the same cube and the same texture. Green boxes are the entities’ physics poses projected through the camera intrinsics, red boxes YOLO-World’s detections; the sensor answered target_box at 0.30 and distractor_box_a at 0.46, each bound to its own projected box by IoU, and the engine’s verdict on that sighting confirmed the find. One of the twelve frames the check run keeps.

zone in **36/36** hunts, follows **18/18** true tips and debunks **18/18** false ones, and commits at a false tip’s zone in none of them. It pays for that with twice the inspections of a sweep that stops at its first accepted sighting, and the sweep is wrong in a third of its hunts (Table 18). Seventeen of the 36 true sightings were under the bar, the tumbler in every round; those hunts ended by exhaustion after every zone had been inspected rather than on arrival, and none was decided wrongly.

policy	finds	tip hunts correct		committed at a false tip’s zone	inspections per hunt
		true	false		
engine	36/36	18/18	18/18	0/18	5.03
zone-by-zone sweep	24/36	11/18	13/18	—	2.5
seeded random order	27/36	—	—	—	—
tip-follower	—	18/18	13/18	—	—

Table 18: The pre-registered fourth real-pixels sitting, every policy on identical pixels through the same seam: one house, one hand-held camera, six declared zones, six object classes over six rounds, no instance identity and no association step. The engine’s Wilson lower bound on 36/36 is 0.90 and every class is 6/6. Seventeen of the 36 true sightings sat below the resolve bar, the tumbler in every round at 0.83–0.89. docs/PHONE_PROTOCOL.md carries the manifest, the filming procedure and the sense model.

One was close. In round 19 the phone on the bin read 0.750 and the tumbler on the sofa read “cellphone” at 0.752, so an argmax over the detector would have named the sofa. The engine’s belief after the room was searched stood at 0.51 to 0.49 for the bin, which it returned to and confirmed. That episode’s false tip pointed at the sofa, and the engine still committed to the bin, where the sweep and the tip-follower did not.

On identical pixels, and with a commodity detector’s raw output through the same seam, the search discipline and not the detector separates 36/36 from the sweep’s 24/36. This is one house, one camera and one sitting.

Both seams held with a real component in the slot. The third seam is time, and it is what the next subsection measures.

4.6 A mind that persists

A memory that ages, beside a search that widens on refutation, keeps one mind at **12/12 every day** across six days of nightly change; a memory that only accumulates does not. The control is the same mind with those two modules removed, on the same houses and the same seeds, and Table 19 carries every count in this subsection. Uncertainty 4 asked whether a memory that never forgets is an asset in a changing world, and the answer is that it is a liability until it can forget.

Transfer and persistence. Two modules separate a mind that cannot enter an unseen room from one that clears a house it has never seen: an exploration drive, then ageing memory. The upper block of Table 19 is the ladder. Entering a procedurally generated house with an empty ledger fails almost completely before the exploration drive, and re-taking the same house daily needs the memory modules on top of it.

The dynamic house. The dynamic-house protocol keeps one mind across twelve houses, six days each, with 1–2 items moved nightly. Without the memory modules that mind holds day 1 and then collapses (`woz_dynamic.json`). Per-episode budgets, refutation-driven widening and age-decayed evidence restore **12/12 every day**, and under realistic 8-hour nights a patrolling mind holds most of that while a drive-less one erodes.

Habits kept across episodes. Every result in this paper is a cold start. The opt-in lifelong mode (default off, so every number byte-reproduces) persists the robot’s own confirmed fetches and mines habitual-location rules from them through the standard candidate/provisional/validated lifecycle. The rules are advisory blind-search priors that an observation always overrides. The habit world is one house with oracle sensing, 30 episodes, and a mug staged on a habitual counter with seeded probability 0.75. A measured sweep chose that counter as the receptacle the cold-start search reaches latest among those it finds within budget (the floor it never finds). So 95 is the worst-case cold cost of a fetchable placement, and the gain is an upper bound. The lifelong mind is indistinguishable from a cold start for twenty episodes. Once the habit validates on held-out fetches, every visit to the habitual counter costs **10 actions instead of 95**, with no loss of success and no wrong commitment in either arm. A deliberately wrong habit, four episodes off the counter, costs actions and never costs a commitment. The learning is the engine’s own: no demonstrations, no labels, no evaluator truth. Two protocol facts that bound the reading, and a first curve that validated and saved nothing, are in Appendix B.

The cost of memory. Keeping a mind costs time per decision, and three increments of incremental view maintenance have paid most of it back. Caching the append-only ledger and linearising a contradiction scan came first, for about $4\times$ per simulated day. Increment one then made a 300-day life about $7\times$ faster, increment two a further $4\times$, and increment three a further $2.5\text{--}3.4\times$ over five interleaved paired repetitions. Every run is byte-identical in its decisions (Figure 9). The profiling, the mechanism and the equivalence check behind each increment, with the per-increment figures, are in `docs/STRESS_LATENCY_NOTES.md`.

The maintained views now cost, per decision, $O(\Delta)$ for the belief graph and effective statuses and $O(\Delta \cdot k)$ for contradiction detection. Here Δ counts the claims appended or re-trusted since the last check and k the claims about one subject. A filtered read matching m claims costs $O(m)$ and the room-visit view over R rooms

<i>transfer ladder</i> , 25 seeds per rung: success rate			
rung	before the drives	drive and fallback only	+ memory modules
home world, find one item	—	—	0.96
home world, find all five	—	—	0.88
home world, find, collect and brew in order	—	—	0.88
procedurally generated house W2, empty ledger	0.20	0.88	1.00
procedurally generated house W3, empty ledger	0.00	0.68	0.96
re-taking one house daily	0.25	0.875	1.00
<i>dynamic house</i> , 12 houses $\times$ 6 days, 1–2 items moved nightly: finds of 12			
arm	day 1	days 2–6	source
before the memory modules, drives off	12	4, 1, 0, 2, 0	woz_dynamic
before the memory modules, drives on	12	6, 5, 4, 2, 3	woz_dynamic
with the memory modules	12	12 every day	woz_dynamic
8-hour nights, no exploration drive	12	down to 4 by day 6	night480
8-hour nights, patrolling	12	10 at day 6	night480
<i>habit world</i> , one house, oracle sensing, 30 episodes			
arm	success	wrong	actions at the habit
cold start	30/30	0	95
validated habit (precision 0.80, held out 0.73)	30/30	0	10
a deliberately wrong habit, 4 episodes	4/4	0	+1, −13, +7, −15

Table 19: The persistent mind, in one place. The ladder’s three arms are the engine before the exploration drive and fallback plans existed, with them, and with the v0.3 memory modules (age-decayed evidence and refutation-driven widening) on top. Staleness-sized patrols cut patrol spend from 5.8 to 2.6 actions per day at identical success. Zero wrong commitments in every row. The lifelong mode is off by default, so every other number in this paper is a cold start.

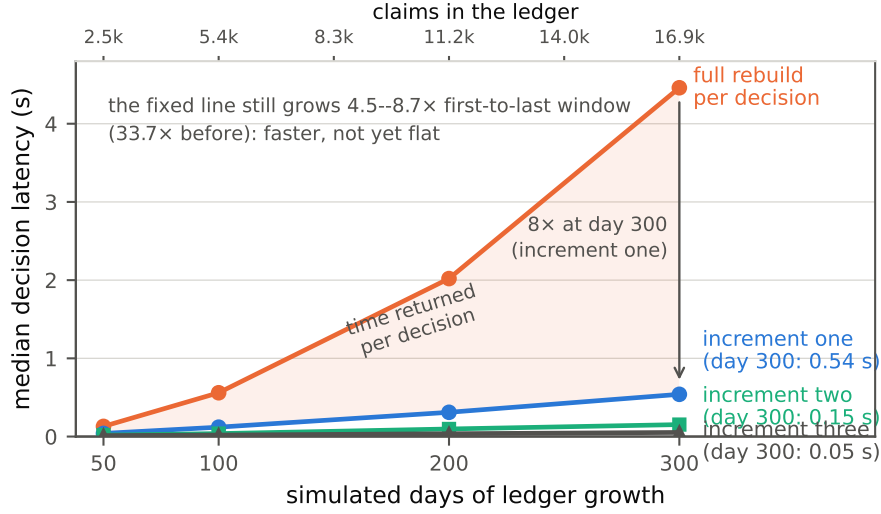


Figure 9: The cost of lifelong memory across three increments of incremental view maintenance (300-day soak; every run byte-identical in its decisions). The full-rebuild and increment-one curves are one session’s paired runs; the increment-two curve is a later session’s run whose own paired before-run reproduced the increment-one curve to within machine-load noise; the increment-three curve is the harness pair of a third session, bracketed by five paired repetitions (`s4_soak_ivm3_reps.json`). The remaining growth has a named cause: the drives’ and meta-controller’s contradicted-claim search and the per-subject sighting walk still walk full claim lists. The upper axis gives the ledger’s claim count at each checkpoint, identical in every run; the marked 8× is increment one’s day-300 ratio (4.46 → 0.54 s), the band it spans being the time that increment returned per decision.

costs $O(R)$. Two operations still walk the whole ledger of N claims. They are the contradicted-location-claim search in the drive manager and the meta-controller, and the per-subject sighting walk over a history of order N . Per-day cost therefore still grows about linearly with N on this workload (Table 21), which is the boundary O_2 in §8 sets out to close. The chain of per-checkpoint figures across the four sessions’ curves in Figure 9 is cross-session, so it is indicative rather than paired.

Every battery to this point places the target somewhere in the world. The next one does not.

4.7 The absent target

Requiring a coverage certificate before the engine will name a place takes wrong verdicts on an absent object from 121/300 to **0/300** at eight actions and from 79/300 to **0/300** at three. The control is the same battery under the shipped verdict rule, on the same seeds, and Table 20 carries every count. Uncertainty 8 asked whether the commitment discipline holds when the object is not in the building at all; the discipline held in every battery above, and every one of them put the target somewhere in the world. This battery takes it away deliberately.

Controls, target present. With the target in the world the engine finds it in 300/300 episodes at each bar, with and without the deployed learned rule, and shrinking the action budget breaks that figure, so it is falsifiable (Table 20). But it measures search success, not commitment correctness: the control confirms early and never spends its budget. A length-matched control therefore suppresses confirmation, spends all eight actions with the object present, and scores the final verdict against live world truth. At bar 0.9 it holds a

verdict in 196/300 episodes, all from the direct path, and none names a wrong room. The rest end with no verdict: their sighting aged past the direct path’s freshness window, and the exhaustion path stays gated off once a sighting has cleared the bar.

Found: an exhausted search names a room. The engine has two ways to commit. The direct path commits on a fresh, uncontradicted room sighting whose confidence reaches the resolve bar. The exhaustion path opens only when that bar is above zero. It fires once no sighting reaches the bar and every room is either named by a hypothesis or already observed (sighted or found empty). It then commits to the uniquely best-supported candidate rather than return no verdict. At bar 0.9, with the target nowhere and its stale in-building reports left in place, that rule names a place for the missing object in **121/300** episodes. The split is 43, 43 and 35 at levels 3, 4, 5, all naming rooms rather than containers, and the count is 126/300 on a disjoint seed range and unchanged with the learned rule removed. The rule fires 799 times across the arm’s 3,000 reasoning calls; all but the 121 are retracted by the inspection they trigger, which finds the room empty. World truth reads *unknown* in 300/300 episodes and the seed streams are byte-identical to the controls (Appendix B), so absent and present worlds are the same worlds.

The same rule, seen from the other side. The failure is not absence alone. The exhaustion rule also fires with the object present, whenever the candidate rooms are refuted before the object is seen. At bar 0.9 it fires transiently in 74/300 control episodes and names a room that is wrong at that moment. The inspection it triggers retracts it, and every such episode still ends on the right room. Under a three-action budget there is no time for the retraction: the rule decides 24/120 control episodes and is wrong in 19 of them. So the boundary is the exhaustion rule at a raised bar whenever the candidates run out before a sighting clears it, and absence is the condition under which that always happens. The one published run of the rule at a raised bar with the target present is the fourth real-pixels sitting (§4.5), where the hunts that ended by exhaustion all ended on the right zone at a budget that always allows a re-inspection.

Fixed: a refusal that carries its own proof. The verdict vocabulary gains a third terminal state. Where the engine would have named its best-supported candidate it can instead decline, and the refusal carries a coverage certificate: one derived claim on the ledger that cites, for every room the episode knows and has not excluded as unreachable, one empty-handed look of the robot’s own, none of them older than the last claim that placed the target anywhere. The rules are worth stating exactly, because they are what a checker re-derives. A report is not a look: a third party’s “not here” is evidence, and it cannot clear a room. Neither is somebody else’s sensor, even when it is filed OBSERVED; only an absence one of the robot’s own actions returned counts, and the engine and both checkers enforce that identically. A certificate that cites a subset of the rooms, cites a second-hand report, or ignores a later sighting is invalid. The coverage question is asked at the end of every reasoning call that names no place, at any bar, so it is not a special case of the exhausted search.

Re-measured. Under the certified verdict no episode of the absent battery names a wrong place, at either budget (Table 20). A certificate is issued in 119/300 episodes at eight actions; every one of the remaining open episodes still had one to three rooms it had never looked in, which is a budget limit rather than a rule limit. With the object’s own stale reports stripped as well, so that the robot searches on nothing at all, it sweeps the building, finds it empty and says so in 300/300 episodes, and stops before the budget is out. With the coverage fix of §4.2 also on, so that rooms never looked in are always on the planner’s menu, the evidence-kept arm reaches 297/300 at the raised bar and 299/300 at the warehouse default bar. On the present-object arms the three-action wrong verdicts fall from 60/300 to 0/300, and on the paper’s own 40-seed arm from 19/120 to 0/120, while correct finds rise from 176 to 178.

What it costs. One false NOT-FOUND in 300 present-object episodes at eight actions, and the certificate that produced it is faithful. In that episode the robot looked in the room the object was in, the inspection missed it, and the certificate cites that miss: the certificate certifies coverage, not perception. A second look at the favoured room before declaring absence removes it, and does better than remove it, because the second look finds the object and the episode ends on the right room: false refusals over the whole certified matrix fall from 2 of 2,400 present-object episodes to 0. The price is certificates the budget cannot pay for. At eight actions the count falls from 119 to 101 on the evidence-kept arm, every certificate lost is an episode that stopped still owing the second look, and the look costs one extra action per episode. Give the hunt twelve actions with the coverage fix and the second look together and every objective line holds at once: a certificate in 300/300 on both absent arms, 0 wrong verdicts and 0 false refusals.

On real pixels, nothing changes, and one thing is not yet reachable. The two real-pixels replays score 29/36 and 35/36 under the certified verdict, identical row by row to the shipped rule. The coverage question is asked at the end of every one of those hunts and answers “no” every time, for two reasons. No hunt reads all six zones empty: after synonym folding the detector fires for the target class in at least one zone of every clip set, and the most any hunt clears is five of six. And the phone adapter does not yet mark which of its observations one of the robot’s own actions produced, so under the own-look rule none of its absences can clear a zone at all. A justified refusal is therefore unreachable on the real-pixels path until each adapter stamps its own looks. That is one line per adapter and it is not done.

What is settled and what is not. The missing capability is built and measured, and it is not yet the shipped default. The certificate is a proof object that a checker re-derives from the ledger export with no engine code, which is the first half of objective O1; the statistical half, a conformal bound on the false-refusal rate, is open (§8). The 121/300 was re-derived by a classifier that does not use the harness’s reporting code, and the two diagnostic arms of the sweep, with their bar-0.0 readings and the wiring check that no verdict is invented from nothing, are in Appendix B. Worlds are synthetic warehouses; only the resolve bar is shared with the real-pixels battery.

The boundary now carries a verdict with a proof behind it. What the engine does under attack, and what it costs, are the last measurements.

4.8 Robustness and cost

Uncertainty 3 asked whether evidence hygiene survives an adversary who plants confident false claims, and the engine’s price per decision had to be measured beside it. Nine suites built to break the engine leave all 1,860 unseen episodes correct and never confidently wrong under 320 planted lies per episode, and one decision costs a third of a millisecond on a laptop core.

Stress battery. The nine suites attack different surfaces. Seed fuzzing ran 1,860 episodes (310 unseen and degenerate seeds $\times$ 6 levels) and property fuzzing threw 3,400 random claim/belief soups at the ledger. Other suites fed it malformed and hostile input, ran a 300-day dynamic-house soak and checked determinism across parallel processes under varied hash seeds. Gaslighting planted up to 320 lies per episode, and crash recovery hard-killed the engine after 73 committed claims, all durable. Resource limits scored a wide belief beam in 0.07 s, validated a 3,000-deep proof chain in 0.28 s and queried a 40,000-claim ledger in 0.38 s. The ninth is an unscreened self-repair sabotage sweep (120 random corruptions). The artifact JSONs ship with the kit; the full protocol notes stay with the engine.

Every one of those unseen episodes was correct, each under 32 ms in the worst case, and byte-identical across four processes. Under the floods the engine never reported success at a fabricated location. It was

arm	actions	verdict rule	wrong	NOT-FOUND	no verdict	correct find
absent: its stale reports left in place	8	shipped	121	0	179	0
	8	certified	0	119	181	0
	8	certified, coverage fix on	0	297	3	0
	3	shipped	79	0	221	0
	3	certified	0	0	300	0
absent: its own reports stripped too	8	shipped	0	0	300	0
	8	certified	0	300	0	0
control: target present, confirmation on	8	shipped	0	0	0	300
	8	certified	0	1	0	299
	8	certified, second look on	0	0	0	300
	3	shipped	60	0	47	176
	3	certified	0	0	122	178
length-matched control: confirmation suppressed	8	shipped	0	0	104	—
	8	certified	0	1	107	—
<i>twelve actions, coverage fix and second look together</i>						
absent, reports kept	12	certified	0	300	0	0
absent, reports stripped	12	certified	0	300	0	0
both present-object arms	12	certified	0	0	—	—

Table 20: Verdict accounting at bar 0.9, 300 episodes per cell, levels 3–5, seeds 0–99, in the rows of the Table 3 vocabulary. “shipped” is the rule every other number in this paper was measured under, and it reproduces `absent_target.json` row for row. “certified” requires a coverage certificate before the engine will decline, and the three-action present-object row corresponds to the paper’s own 40-seed arm going from 19/120 wrong to 0/120. The exhaustion rule fires 799 times across the absent arm’s 3,000 reasoning calls; 678 of those are retracted by the inspection they trigger and 121 are still held when the budget ends. The same rule fires transiently in 74/300 present-object episodes, always retracted. The single false NOT-FOUND is one episode seen in both present-object arms, and the second look removes it. Search success on the present-target control as the budget shrinks, over 40 seeds: 120/120 at eight actions, then 74, 53 and 23 of 120 at three, two and one, which is what makes the 300/300 falsifiable. Every switch here defaults to the shipped rule. `absent_target_certify.json`, `absent_target_certify_fix.json`, `absent_target_certify_dc.json`.

100% correct through 160 lies per episode and 93% at 320, not-found in the rest, and the degradation is graded rather than sudden. The ledger survived hard kills and concurrent access. Across the 120 config sabotages the self-repair loop never crashed, never adopted a regression and never adopted its placebo.

The battery found five defect classes (one at two code sites), all of one kind: correct reasoning, ungraceful failure on hostile or corrupt input. An unbounded timestamp crashed the ledger, a corrupt saved-config file crashed its loader, and a bad command-line level printed a raw stack trace. A tampered config value outside $[0, 1]$ pushed a computed claim confidence out of range at two claim-construction sites. Each fix refuses or clamps at the typed boundary: the suite of that day (133 tests) green, replays byte-identical, ALFWorld re-run from scratch at 134/134. A sixth reported failure was a defect in the test itself, since corrected. Per-suite figures, the defects and the sabotage sweep are in `docs/STRESS_LATENCY_NOTES.md`.

Latency. Table 21 collects this paper’s latency figures, each from a different instrument, and rows are not comparable without their basis.

quantity	value	basis
one decision, fresh ledger	0.33–0.34 ms median; p95 0.70–0.77 ms; p99 < 1.1 ms	1,025 decisions over 360 held-out episodes, one pinned laptop core, near-idle machine
one episode, fresh ledger	2.8–3.1 ms median; 4.8–5.4 ms p95	the same run
one episode, head-to-head campaign	~5 ms wall-clock	Table 11’s session, unpinned, beside the LLM harness
one episode, hosted LLM agent	~9–12 s	end to end through the provider’s API, network, queueing and tool loop included; local 7–8B models 11–94 s
one simulated day at day 300 of the soak, after increment three	0.040–0.081 s (0.157–0.248 s before)	five interleaved paired repetitions, 33–59% background load
first-to-last-window growth of that cost	4.5–8.7× after (10.5–15.9× before), against 6.6× in claims	the same five repetitions; the least stable number in the battery

Table 21: The latency quantities of this paper on one page, each with the instrument that produced it.

One decision on a fresh ledger takes a third of a millisecond on one pinned laptop core, over 1,025 decisions across 360 held-out episodes, and the adversarial level costs under half a millisecond (Table 21, Figure 10). Peak resident memory is 53–60 MiB, the runtime dependency closure 14 MiB and the wheel 170 KiB. These are laptop-CPU figures. No embedded or ARM board was measured (`docs/EDGE.md`), and the profile’s run details are in `docs/STRESS_LATENCY_NOTES.md`.

Self-repair. v0.4 closes a metacognitive loop with no language model and no human inside it. Each module’s implicit predictions become settled expectation records, and systematic gaps become typed violations that name the implicated constants. Each violation scopes a finite menu of one-edit repairs, which race on the failing scenarios and are adopted only if every guard suite holds and held-out validation does not regress. Each cycle also races one placebo, a mutation of a non-implicated field, which is never adopted and always recorded. Its one departure from the Darwin Gödel Machine is a typed deterministic repair vocabulary in place of that system’s LLM mutator [58], and the exam is fault injection with sealed grading. Two sealed fault-injection exams recover every detectable single-field sabotage at 98–100% with zero false repairs and zero placebo adoptions; one no-archive lane fails by construction; the exams are in Appendix B. Limitation 9 states the four ceilings the loop does not cross.

Failure modes, mitigations, status. Table 22 gathers the failure modes met in this paper with the mitigation and measured status of each.

§5 reads these results together.

failure mode	mitigation in the engine	status
hostile or corrupt input (malformed claims, oversized fields, corrupt saved config)	refuse or clamp at the typed boundary	five classes found by the battery, fixed, behaviour-preserving
planted false sightings at scale	verify before trust; contradiction and absence bookkeeping	100% correct through 160 lies/episode, 93% at 320; never confidently wrong
very old evidence loses its recency ordering	an ordering-preserving treatment, default off	measured; 0 of 360 re-ranks (limitation 4)
detector recall floor (thin, tiny objects)	a missing detection cannot create a sighting; not-found after budget	8 (plus one target sealed inside a box, never rendered) of 96 episodes; 0 wrong commitments
detector mislabel (a jar candle read as a cup)	synonym groups, added only after two independent spawns	4 of 96, reported not-found; a Candle/Cup synonym enters on a second independent spawn
a learned habit that is wrong	rules advisory only, held-out validation before deployment	−15 to +7 actions, 0 wrong commitments ($n = 4$)
latency growth with lifetime memory	incremental view maintenance	$\sim 7\times$, then $4\times$, then $\sim 3\times$; 4.5–8.7 $\times$ growth remains (limitation 11)
a sabotaged configuration constant	expectation monitor, guarded one-edit repair, placebo control	recovers pre-screened single-field faults; one lane fails by construction (limitation 9)
sensing from drifted wheel odometry under a map-frame navigator	vantage from the localized pose (physics, in sim); drift and SLAM error recorded per arrival	Nav2 run 1 wrong at 6.7 m drift; 21/21 run drift ≤ 0.099 m
SLAM starving the navigator (scan queue overflow, stale map→odom)	configuration: sparser graph, longer transform tolerance	Nav2 run 4 answered not-found, no wrong commitment; run 5 21/21
a navigation goal that aborts or times out	failed action at the executor seam: one unit of budget consumed, no claim recorded, re-plan from the unchanged ledger	one 120 s timeout in the 21/21 run's tip episode (5 attempts, 4 inspections); run 4: 8 attempts, not-found, no wrong commitment
wrong-instance binding under a real tracker	re-inspection and absence bookkeeping (the verify loop)	exercised in the camera lane: 4 episodes with two same-mesh boxes in view, every detection bound to its own projected entity, 0 wrong-instance sightings, 0 wrong commitments; the verify loop stays the mitigation for a tracker's binding error (limitation 12)

Table 22: Failure modes met in this paper, the engine's mitigation, and the measured status of each. Every row with a number points at a shipped artifact.

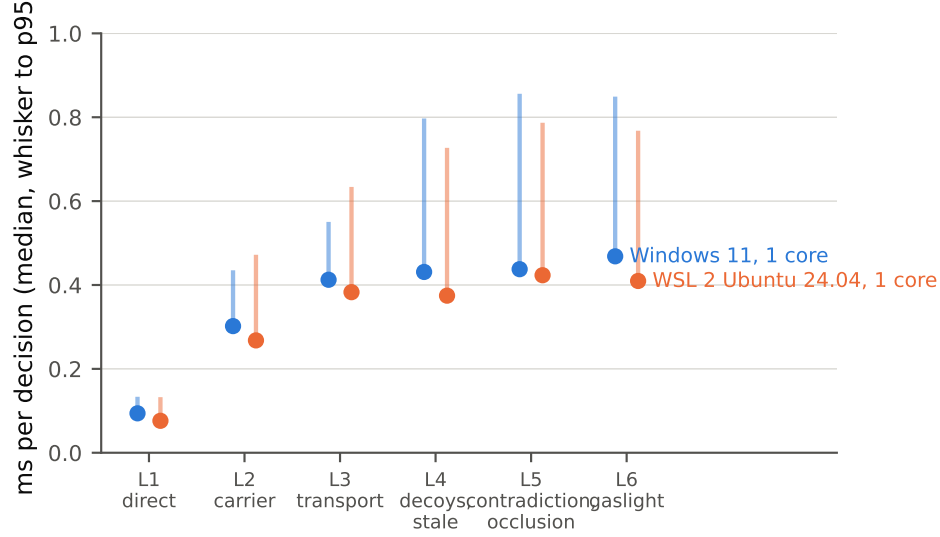


Figure 10: Per-decision latency by benchmark level on one pinned laptop core (Intel i9-11900H): median as the point, whisker to the p95, Windows 11 and WSL 2 Ubuntu 24.04. The adversarial level costs under half a millisecond; the two platforms agree to within a few percent.

5 Discussion

The results establish three things. A deterministic decision layer reaches accuracy parity with two hosted LLM agents on identical evidence. It adds four properties neither of them supplied here: about $2,000\times$ lower per-episode latency (5 ms locally against 9–12 s end to end through the API), byte-identical replay, no per-call model fee, and a machine-checkable derivation record behind every answer. Below the exhaustive budget the machinery is necessary as well as efficient against every control raced here, because an exact belief-space planner over a plain filter’s belief closes about half of that filter’s gap to the engine and none of the rest, and because removing one of the engine’s own mechanisms costs more than a quarter of its four-action episodes. And one build crossed the perception and navigation seams with real components in the slot, and stayed correct wherever it committed.

Does explicit provenance improve the tradeoff? Under matched perception, identity handling and prior knowledge, the answer this paper can support is narrower than the machinery is. Search cost: no. Give an exact planner a belief that reads the relations the engine reads and it matches the engine at four actions to within 0.8 points, leads at two by 9 points, and finishes every budget in fewer actions. Correct findings: no separation at the headline budget, and at two actions the matched planner is ahead. Justified abstention: yes, and it is the one leg that separates. At two actions the engine’s shortfall is 152 episodes on which it declines to name a place and none on which it names a wrong one, where the planner arms have already guessed; with the target absent from the building the same discipline, given a certificate to stand on, takes wrong verdicts from 121/300 to 0/300 and issues a refusal in 119/300 that a checker re-derives, at the price of one false refusal in 300 present-object episodes. So explicit provenance does not buy a cheaper search. What it buys is a verdict a third party can audit, and the ability to decline with a reason rather than by running out of budget. That is a smaller claim than the 25-point margin over the published planner suggested, and it is the one the controls support.

What the seams do and do not establish. The decision layer is not the hard part of a robot, and this paper does not claim it is. Table 14 states per battery what is real and what stands in, and the honest summary is that no real drive base and no real multi-object tracker appear anywhere in it. What the seam experiments establish is narrower and is worth separating from what they do not. Substituting a real open-vocabulary detector for a simulator’s visibility oracle left every commitment correct and cost actions rather than correctness. Substituting a real navigation stack, with its timeouts, aborts and drift, left the engine’s inspection sequences unchanged and turned a failed goal into one unit of spent budget and no claim. Removing instance identity entirely, on real footage of a real house, left the commitments correct where a detector argmax would not have been. None of that measures perception under occlusion, SLAM drift that moves a sensor’s vantage, or a tracker’s binding errors, and §8 states which objective closes each.

What the LLM comparison means is bounded by what it measured. On these 48 episodes no accuracy gap is detectable for the two hosted agents, so what separates them from the engine is structural rather than a difference in reasoning quality: latency, cost per call, offline operation and the trail. Against trained robotics stacks the comparison is one of requirements rather than benchmarks. The one learned component inside the decision loop is a drop-in detector on the perception seam (§4.5). The classical belief-space line is raced directly as an exact planner, and the open-vocabulary line supplies front-ends for that detector slot rather than rivals at this layer.

The deployment properties follow from the design rather than from any one benchmark. A deployment is briefed rather than trained: typed goals over a closed ontology, no demonstrations, no data collection and no fleet. Operation is offline and private, every run replays byte-identically, and every decision is contestable afterwards from its ledger. The engine fits inside a control loop at a third of a millisecond per fresh-ledger decision on one laptop core (§4.8). That holds for a fresh ledger per task or for consolidation between tasks; an unbounded lifelong ledger’s cost grows until the remaining view-maintenance increments land (limitation 11). Hospital logistics and SubT-style search, the two settings modelled here, need exactly those properties.

Replayable decisions, offline operation and determinism are evidence an assurance case or a post-incident audit can use. Test, evaluation, verification and validation (TEVV) processes and safety standards such as ISO 13482 [22] ask for evidence of that kind. This paper claims conformity with none of them. The Machinery Regulation (EU) 2023/1230, applicable from January 2027, reserves mandatory third-party assessment of safety software for systems “with a fully or partially self-evolving behaviour using machine learning approaches” and exempts “software incapable of learning or evolving” (Recital 55). The decision layer as shipped, with every learned element off by default, is of the second kind, which is an observation about the text rather than legal advice, and it holds only while learning stays outside the safety function.

6 Limitations and threats to validity

Read as threats to validity, items 1, 2, 4, 5, 10, 16 and 17 bound *external* validity, and items 3 and 8 and the detector run’s by-construction precision bound *construct* validity. Items 6, 9, 11, 13 and 15 are measured ceilings, item 12 bounds the evidence, and item 18 states which results are behind switches that are off by default. Items 7 and 14 are disclosures, of provenance and of the audits’ independence.

1. **Home-field advantage.** The engine’s rules were designed for the world distribution the generator samples; the LLM sees it cold. The result shows how grounded competence is built for a domain, not domain-general superiority.
2. **Bounded world, one task family.** Six locations, LOCATE goals, a closed ontology. The larger embodied worlds (102 hospital furniture spots, the SubT tile lists) run under the same ontology and the same goal type (§4.4).

3. **The LLM comparison is bounded by our scaffolds.** The two hosted LLM agents already match the engine’s accuracy here, and stronger agentic scaffolds could lower their step counts further. The harness accepts any Ollama/OpenAI-compatible endpoint and publishes per-episode data, so the comparison is open to extension.
4. **Hand-set constants remain.** Priors, reward weights, the evidence half-life and the staleness horizon live in one versioned config. The half-life was chosen a priori, not fitted to the dynamic-house results it was tested on. An autonomous optimizer with mutation power over every numeric constant found no improving mutation and rolled itself back (`data/artifacts/evolved_config.json`). The hand-set defaults are therefore also the optimizer-validated ones, a statement about the search, not about optimality. The current rule set, v0.2.0, followed temporal probes that exposed two defects after the first campaign (Appendix B). Original-campaign and post-fix figures are both reported (1.75 and 1.80 actions on L2–5). One belief-arithmetic choice is measured rather than settled: how evidence that has aged all the way down is ordered, where the shipped treatment loses the recency ordering between two very old sightings. A design study ran three treatments on the full benchmark: an ordering-preserving variant re-ranks the top hypothesis in 0 of 360 episodes and moves ECE by -0.008 to $+0.007$ across the four calibration cells, while a decay-to-prior variant re-ranks 35 of 360 and degrades shipped-protocol calibration by 0.038 (the ECE cells and the re-rank counts are from the study’s artifact and log, which stay with the engine). Both exist as a default-off engine option; every number in this paper is from the shipped treatment and byte-reproduces with the option in place.
5. **The dynamic house is one change model.** Items move uniformly at random; modules that would learn movement structure are untested.
6. **Consolidation is measured neutral here.** Nightly consolidation compresses repeated-observation runs into summary claims, with the originals superseded, never deleted. It is behaviour-neutral in the dynamic house: `woz_dynamic_reflect.json` equals `woz_dynamic_full_decay.json` in every daily outcome and action count, because passive perception already deduplicates re-sightings. Its benefit under denser sensor streams is expected, not claimed.
7. **Development disclosure: an LLM built this system.** The engine contains no language model at runtime, and every benchmarked decision is deterministic symbolic computation. The system itself was designed, implemented and debugged by an LLM assistant (Anthropic’s Claude) under the author’s direction. Claims of the form “no LLM anywhere” are claims about the *artifact*, never about its development process.
8. **No error bars beyond binomial intervals.** Every table reports raw counts from single deterministic runs; Wilson 95% intervals for the key proportions are in the captions and in Appendix B, and Table 27 collects every one of them. The decoy protocol is directional, not powered: 0 of 6 fooled has a Wilson upper bound of 0.39, which is why the pre-registered 30-seed run was run (§4.4; launch log in Appendix B). Seed-family variance is unquantified and single-machine, with one visible instance in Appendix B, and one-episode deltas are labelled as inside binomial noise, in the LLM comparison of §4.3 too.
9. **Self-repair has four ceilings no published system removes.** The loop can only fix defects expressible in its mutation space and visible to its expectation monitor. It cannot invent a missing representation, fix a wrong objective, repair its own diagnostic machinery, or see failures it holds no expectations about.
10. **The embodied evaluations grade search, not robotics.** Locomotion is teleport-abstracted except under the ROS 2 stack, and sensing reads simulator metadata or physics state except in the real-detector

battery, whose instance association is a tracker stand-in (Appendix B). Under uniform priors the plain SubT and hospital protocols grade search completeness and correct commitment only. The planted-tip variant (§4.4) discriminates on true-tip exploitation, and its staged-decoy round measures commitment discipline directly: engine 0/6 fooled, class-matching policies fooled in 4–5 of 6. A full-stack robot could fail where these adapters succeed. Mapping, control and vision are outside what these evaluations measure.

11. **Latency grows with lifelong memory.** Three increments of incremental view maintenance have cut a 300-day soak $\sim 7\times$, then $4\times$, then $2.5\text{--}3.4\times$, with byte-identical results (docs/STRESS_LATENCY_NOTES.md). Latency still grows $4.5\text{--}8.7\times$ from the first window to the last, against $6.6\times$ in claims. Constant latency over an unbounded lifetime needs the remaining increments, an ordered contradicted-claim index and an event-time-sorted per-subject sighting index. Correctness is unaffected throughout.
12. **Scope of evidence: the decision layer, in simulation.** No physical robot has been run: the work was conducted without lab hardware, and the evaluations isolate evidence-grounded search and decision auditing from locomotion, mapping, manipulation and perception (Appendix B). The two real components ran separately and together (26/26 checks); the claim is that the engine survives each substitution and their combination, not that a tracker’s binding errors were measured. The real-pixels battery’s 36 hunts are one house, one camera and one sitting (Wilson lower bound 0.90), and one was decided by a belief margin of 0.02. Physical deployment follows from the ROS 2 blueprint of Appendix A, and the repository, seeds and artifacts let an independent group reproduce every number before that step (see the reproducibility statement).
13. **The justified not-found verdict is built and measured, and three things bound it.** §4.7 measures the boundary it closes: under the shipped rule 121/300 phantom verdicts for an absent object at the raised bar of 0.9, and 19/120 wrong under a three-action budget, against 0/300 and 0/120 under the certified verdict. The certificate inherits the sensor’s recall. It certifies that the robot looked in every room, never that it would have seen the object had it been there, and the one false refusal in 300 present-object episodes is exactly that failure, cited faithfully by the certificate that produced it. A refusal is also unreachable on any adapter that does not yet mark which observations its own actions returned, which is every real-pixels path today. And the statistical half of objective O1, a conformal bound on the false-refusal rate, does not exist (§8).
14. **Every audit was a separate session of the same assistant.** The engine, the benchmarks, the adapters, the red-team reviews, the pre-submission audit and this paper share one author and one LLM assistant; where the paper calls a review “separate” or “outside” it means a fresh session of that assistant with its own charter, never an outside group. The artifacts and the kit are shipped so that this can change, and no outside group has yet regenerated the kit’s numbers; §8 makes that regeneration a gate on every objective.
15. **The matched belief was written after reading the generator.** The exact planner that closes the engine’s margin in §4.1 reads relations its author found by reading the generator and the executor, and one of those relations exists only because this benchmark has carts. It is therefore a strong control on the question “is the advantage the belief or the plan?” and a weak one on “would this belief generalise?”. The engine never sees the generator. §4.2 is the control for the whole arrangement: a family designed without reading the engine, on which the matched planner still leads at four actions.
16. **The held-out family cannot falsify a wrong commitment.** Its simulator writes a confirming sighting only where the object really is, so no policy can commit to a wrong room on it. The 37,800 episodes

with zero wrong commitments there test whether a policy finds the thing, not whether it commits safely; the batteries that test commitment safety are the staged decoys, the real-pixels sittings and the absent-target battery.

17. **ALFWorld’s unseen split is four households.** Its 134 task instances live in one kitchen, one bedroom, one bathroom and one living room, against 76 distinct households on the seen split. The reduced-allowance curves of §4.4 are therefore four households’ worth of evidence about search order, and the seen and unseen splits disagree about the fixed alphabetical sweep for a reason that is one household’s furniture. The 134/134 at the standard allowance is a ceiling result and is unaffected.
18. **Three of this round’s results are behind switches that are off by default.** The certified verdict of §4.7, the coverage rule of §4.2 and the second look before declaring absence are each a configuration field whose default reproduces the behaviour every other number in this paper was measured under, and each is reported as measured rather than as shipped. The budget-aware lookahead of §4.1 is likewise off, and is reported because it does not help. Making any of them a default is a re-baseline of every battery, and none has been taken.

7 Related work

The neighbouring literatures supply every part this decision layer is built from, and none of them supplies the certificate. This section places each beside the work: object search first, then the certificate and provenance lines, then agent memory and the embodied benchmarks.

Object search with a certificate. We found no object-search or object-goal navigation system that attaches a replayable derivation record, a completeness certificate or a justified refusal to its search commitments. In the systems we examined, a published “found” or “not found” is a threshold on a detector score or an embedding similarity. Machine-checkable proofs exist elsewhere in planning and are the parents of the trail here (the paragraph on certificates and provenance below), so the claim concerns search commitments only.

The open-vocabulary line is where perception and exploration are strongest. VLFM builds language-grounded frontier maps [56], OK-Robot does open-knowledge pick-and-drop [29], and WildOS searches unstructured outdoor terrain onboard [43]. SCOUT distils relational knowledge into lightweight scorers, keeping the language model out of inference, and ships the SymSearch benchmark [32]. R2F scores ray frontiers with embeddings [4]. These are front-ends for the detector slot of §4.5, not rivals at the decision layer. OSG Navigator casts the task as a POMDP over open scene graphs [30]. The confidence-calibrated evidence arbitration of Zhao et al. [59] is nearest in spirit to evidence-driven decision-making, without a ledger or a proof. The classical belief-space line is the baseline family for the external head-to-head of §8. It comprises pomdp_py with its multi-object search domain [60], multi-resolution search in 3D [61], and correlational search with a real detector in AI2-THOR [62].

The nearest published guarantees attach elsewhere. KnowNo gives an LLM planner a conformal abstention with a statistical bound on task completion [41]. Conformalised semantic maps give a user-set mission-completion rate for reaching a *known* goal while avoiding semantic hazards [48]. Neither certifies where an unknown object is or that it is absent. Both are templates for the statistical layer of O1 in §8. TRACE states the auditability premise this engine is built on, that investigators must be able to reconstruct why a robot decided, scored by traceability metrics [50]. Here the trail is a proof path a checker replays, and the decisions it explains are graded against hidden truth.

Certificates, provenance and refusal in adjacent fields. Proof-carrying plans give a resource logic in which a plan is a function whose pre- and post-conditions are types, formalised in Agda so planner output

becomes proofs [21]. Unsolvability certificates certify that *no* plan exists [17]. They are the nearest published analogue of the NOT-FOUND certificate O1 specifies, and O1 inherits their proof obligation: a checkable argument that the search space was covered. Luckcuck et al. [31] survey formal specification and verification of autonomous robots. That line certifies properties of the controller or the plan, where the trail records the derivation of one commitment, so the two are complementary.

The trail descends from data provenance. Why-provenance [7] and provenance semirings [20] are the formal parents of “which inputs, by which rules”. ECHO brings a bi-temporal, revision-typed memory with checkable provenance to language agents [34]. None of these selects a robot’s next action or is graded against hidden truth. Refusal has a statistical home in classification with a reject option [10] and selective prediction [18]. Both trade coverage for a bounded error rate, the construction O1’s conformal layer and KnowNo share. Epistemic planning [6] reasons formally about what an agent knows and does not know, the closest formalism to a justified “not found”. Tru-POMDP builds particle beliefs from LLM-generated hypothesis trees and plans over an open-ended POMDP [49], so hypothesis structure inside belief-space planning is not new. It lacks the evidence ledger, the typed statuses and the replayable record. LLM+P hands language-described planning problems to a classical planner [28], the LLM-to-formal line beside which the hybrid mode of §4.3 belongs.

Agent memory systems and cognitive architectures. Cognitive architectures [3, 26] pioneered mind-from-modules over production rules. The engine ships a small fixed rule set plus induced rules that pass a validation lifecycle, and no production system. Truth-maintenance systems [14, 16] keep justifications for beliefs. The ledger’s status and supersede machinery descends from them, and proof paths are justification records with time constraints and replay added. Multiple-hypothesis tracking [40] keeps competing tracks alive. Multi-anchor hypotheses are that idea over evidence rather than sensor returns. Belief-space planning [23, 53] chooses actions under state uncertainty with a model and lookahead, without evidence ledgers, proof paths or rule induction. It is the nearest principled competitor, where we choose by information gain over the belief the ledger defines. §4.1 races it under the identical contract as an exact finite-horizon planner over the filter’s belief and the true sensor model, the optimal policy at this size. It closes about half of the filter’s gap to the engine and none of the rest.

LLM-agent memory systems give a language model a store to consult: text, or a graph of facts, sometimes with time. MemGPT [33], Zep’s temporal knowledge graph [39], and reflection and manual-building agents [8, 35, 44] are the examples. Zep’s data model is the closest to the ledger (bi-temporal facts, provenance, supersede rather than delete), but it stores memory rather than closing a hypothesize–act–verify–learn loop. In all of them a model that reasons in language consults the memory and makes the decision, and nothing certifies the path from memory entry to action. They handle open-ended language this engine cannot. Belief-graph agents for text games [1] learn the state tracking the ledger performs symbolically. LLM-grounded task planning [2, 38, 47] attacks the same embodied problems from the opposite direction, a language model planning over grounded affordances or scene graphs. It is the natural opponent for a future embodied fairness contract. Neuro-symbolic agent research argues for the hybrid this system instantiates in closed-domain, auditable form. The self-repair loop draws on expectation-violation metacognition [13], MAPE-K [24], the Darwin Gödel Machine [58], Recuris [57], PAST-Bench [54] and, more distantly, generate-and-validate program repair [27].

What is ours is the combination and its guarantees. The ledger is the only world model. Statuses are typed, so a hypothesis cannot become an observation by scoring high. Every action’s inputs are replayable, the run is deterministic from one seed, and no language model runs at decision time. The trade is a closed ontology, structured input only, and bounded worlds.

Embodied evaluation. AI2-THOR [25] and ALFRED [45] supply the interactive scenes. Object-goal navigation [5] and semantic object search study the same task with learned perception and locomotion, which the evaluations here abstract away to isolate the search policy. The next-best-view planner is the classic occlusion-tested set-cover formulation [12, 42]. The DARPA Subterranean Challenge [11] contributes the worlds and the scoring rule. Full SubT systems [51] solved SLAM, locomotion and communication jointly, none of which is claimed here. We found no prior published fetch benchmark on the AWS RoboMaker hospital world. The contribution is the working synthesis plus a benchmark methodology. That methodology is hidden-truth grading with a fairness contract for LLM comparison, a sealed fault-injection exam for self-repair, and embodied evaluations, with stated abstractions, in which only the engine varies. To our knowledge these have not been published together.

8 Proposed research and development

The results establish the first half of the innovation of §1.1: a commitment, “the object is in room X ”, that carries a machine-checkable derivation record, at accuracy parity with two hosted LLM agents and with the best published ALFWorld score. The second half is a refusal, “it is not here, and here is the proof I looked properly”. Its proof object now exists and is measured (§4.7); what does not exist is a statistical bound on how often that refusal is wrong. Two boundaries were measured rather than resolved. §4.7 located the first, where an exhausted search named a room instead of declining, and it becomes O1. §6 states the second, a per-decision cost that still grows with lifetime memory, and it becomes O2. The remaining objectives carry the work to a real tracker, to a physical robot and to external benchmarks. Each objective names its challenge, its management, and a falsifiable acceptance test on a committed artifact. The method is the one every result here used: hypothesis, mechanism, hidden-truth grading (pre-registered where the world is not seeded), adversarial review, fix, then byte-for-byte re-verification of every earlier battery before the fix is kept.

O1. Certified refusal: the proof object done, the bound open. *Done:* the verdict type now runs `{FOUND( $X$ , proof), NOT-FOUND(coverage certificate), UNDECIDED(budget)}`, with a refusal state in `GoalStatus`. Exhaustiveness is defined over the location registry rather than the live candidate set, and only an absence one of the robot’s own actions returned counts as coverage, never a reported tip and never another sensor’s reading. The certificate is a proof object that the kit’s checker re-derives from the ledger export with no engine code. §4.7 reports what it moved and what it cost, and Appendix B the forgeries it rejects. Of the acceptance test set out before it was built, two lines are met at the eight-action budget and all three are met at twelve: 0/300 wrong on the absent battery against 121/300, 0/120 wrong under a three-action budget against 19/120, and a checked NOT-FOUND in at least 250/300, which the evidence-kept arm reaches only once the coverage rule of §4.2 is on as well. One present-target regression gate moved: 300/300 became 299/300 at the raised bar, and the second look before declaring absence restores it to 300/300. *Open:* the statistical layer. Above the proof, conformal prediction on the present-target arms calibrates the refusal threshold, the KnowNo construction [41] moved from plan options to room coverage, so the false-refusal rate carries a user-set bound as well as a certificate. *Challenge:* a criterion strict enough to refuse correctly may be too strict to find present objects within budget, which is exactly what the certificate count buys and the second look pays for. Sensors cannot prove absence: eight of the 96 AI2-THOR episodes sat below the detector’s recall floor, and the one false refusal measured here is a missed inspection of the right room. *Management:* every present-target battery stays a regression gate. The declared recall is itself a claim with provenance, so the checker can reject an unsupported one. The certificate carries its window, and any later movement event touching the target invalidates it. The bound assumes exchangeable calibration and deployment episodes, which nightly object movement violates, so it holds per movement window and is re-calibrated when a

window closes. O1 can pass this synthetic test and still fail in a deployment whose declared recall is wrong, so O3 precedes any real-world claim for O1. *Acceptance for the open half*: a user-set false-refusal bound held on the present-target arms at the stated rate, with the certificate unchanged and every shipped number byte-identical when the layer is off.

O2. Bounded per-decision cost over an unbounded lifetime. *Task*: the two remaining maintained views, an ordered contradicted-claim index kept under status events and an event-time-sorted per-subject sighting index. If the slope is still not flat, a multi-tier ledger: a hot window of raw claims plus consolidated summaries that keep the proof provenance of every folded claim. *Challenge*: consolidation may change decisions or orphan proofs, and timing on a shared machine is noisy. *Management*: the standing equivalence gates, record-dump SHA-256 and differential tests against the previous implementation on every call. Any changed decision is reported with both proof paths. Every increment is measured as at least five interleaved paired repetitions, led by load-independent counts. *Acceptance*: on a 1,200-day soak, first-to-last-window growth at most $2\times$ (today $4.5\text{--}8.7\times$ over 300 days), with decisions byte-identical or every difference reported.

O3. Identity without instance ids. *Task*: replace the physics-pose association stand-in of §4.5 with a real multi-object tracker with re-identification, whose identity claims enter the ledger with confidence and typed status. The real-pixels protocol gains same-class decoys placed by manifest. *Challenge*: a tracker’s binding errors are a new source of false sightings the ledger has never seen. *Management*: identity claims enter at REPORTED grade until an inspection confirms them, so a mis-binding can cost actions but not a commitment. The decoy protocol is pre-registered before the clips exist, as every sitting has been. *Acceptance*: a pre-registered sitting with same-class decoys in every zone, 0 wrong commitments, finds not below the previous sitting, and the close-call margin reported per hunt.

O4. The physical robot. *Task*: the packaged ROS 2 stack on a TurtleBot3-class platform under ROS 2 Jazzy, with the detector in a hardware Sense node. The cognition node’s coordinate discard is corrected so the planner’s cost term is live. The 21 integration checks are repeated on hardware in a mapped room, with drift and SLAM error recorded at every arrival. *Challenge*: real navigation fails in ways the simulated body did not, and detector inference costs 130–240 ms per frame on the laptop. *Management*: the executor seam already treats an aborted or timed-out goal as one unit of budget with no claim recorded and re-plans from the unchanged ledger. The four failed simulated runs before the 21/21 result are the checklist. Decision latency is measured apart from stack latency, so the ~ 5 ms figure is tested on its own terms. *Acceptance*: 21/21 on hardware, the same inspection sequences as simulation where the world matches, a false tip injected over ROS abandoned, and any wrong commitment reported with its ledger.

O5. Semantic priors without training data. *Task*: an advisory prior over where an object class tends to be, drawn from an open commonsense source or the mind’s own confirmed finds. It enters at the planner’s weight hook, which today returns 1.0 for every room unless a learned habit fires, and never overrides an observation or the exhaustion invariant. *Management*: priors enter as HYPOTHESIZED-status weights, off by default, on the template of the lifelong-mind result where a wrong habit cost -15 to $+7$ actions and no commitment. *Acceptance*: a pre-registered reduction of at least 10% in mean actions on both the 96-episode AI2-THOR battery and the 36-hunt real-pixels battery, with 0 wrong commitments. Results must be byte-identical to the baseline when the prior is off, and the threshold is set here, before the prior exists.

O6. External validity. *Task*: a pre-registered head-to-head on a third-party object-search benchmark against a third-party opponent. The candidate is the pomdp_py Multi-Object Search domain with its POUCT planner and false-positive dial, the object-oriented POMDP of Wandzel et al. [52] as implemented in Zheng

and Tellex [60] and extended in Zheng et al. [61]. The comparison is on that family’s native metrics, not a forced success-rate table. For the open-vocabulary line the benchmark is SCOUT’s SymSearch [32], whose reference system also runs no language model at inference time, so the comparison isolates the decision layer. *Management*: one world and one setting first, under the benchmark-iteration protocol. The adapter may not read the target’s placement, as no adapter here may. *Acceptance*: on the pomdp_py domain, the engine’s success rate within the Wilson 95% interval of the POUCT planner’s at matched budget and false-positive rate. That arm also requires 0 wrong commitments and every proof path checked. On SymSearch, the engine’s success rate reported beside the reference system’s under the benchmark’s own protocol. Both are committed before the paper cites them, whichever way they fall. The first arm is done. It is the held-out generator family of §4.2: a different location count, carrier count, report reliability and set of delays, designed under a reading restriction and pre-registered before any policy ran. Against the threshold set for it before it existed, success within five points of the home family at every budget, the result splits. At each family’s own exhaustive budget the gate is met, 1.000 against 0.970 as shipped and against 1.000 with the coverage rule. At matched action counts it is not: the held-out family is harder, and at three and four actions the engine falls more than five points short of its home figures. The threshold did not anticipate a family whose exhaustive budget is ten actions rather than eight, and the split is reported as measured. The zero-wrong gate held, in the weak sense limitation 16 states. What remains of O6 is a third-party opponent on a third-party benchmark.

Mechanism ablations and the matched control, done. The leave-one-out control that O6 needed is in §4.1: on the shipped build, multi-anchor hypotheses and repetition decay each carry a disjoint share of the sub-exhaustive margin, and transport hand-off’s share is within noise on the home family. The matched belief-space planner in the same subsection is the other half of that control and reads the opposite way round: the relation whose removal costs the engine nothing is the relation whose addition is worth 24 points to a plain filter, because the engine reaches the same conclusion by other routes and the filter has none. Running the leave-one-out arms on the held-out family is the outstanding piece, since a mechanism idle on the home family need not be idle on one with more carriers.

Sequencing and completion criterion. O1 comes first, since it needs no hardware and is the missing half of the innovation. O2 runs in parallel, O3 on the next real-pixels sitting, O4 when the platform is in hand, and O5 and O6 after O1. The programme is complete when O1, O2 and O4 meet their acceptance tests on committed artifacts that an outside group has regenerated. That means certified commitment and certified refusal, at bounded lifelong cost, on a physical robot. The settings that benefit are those where a wrong commitment is a safety or mission event and an audit trail is required. Hospital and laboratory logistics, warehouse and tool-crib search, disaster-response and defence-adjacent search, and planetary and underwater survey are each already modelled by a battery above. The component that benefits them is the decision layer alone. It consumes an existing stack’s detections and navigation through the ROS 2 seams of Appendix A, and is briefed through typed goals rather than trained.

9 Conclusion

An append-only, bi-temporal evidence ledger with typed statuses, replayable proof paths and a verify-then-redirect search policy is sufficient for a correct, efficient and auditable closed-domain object-search agent. On the warehouse benchmark it is 100% correct at 1.75 actions and abandons planted lies at 3.25, and under budgets brute force cannot exhaust it holds 98–100% at four and five actions where an exact belief-space planner over a plain filter’s belief reaches 73–90%. Give that planner a belief that reads the same relations and the margin at four actions is inside the interval: what the engine keeps under that control is zero wrong commitments, a justified refusal and the record behind both. No policy, the engine included, commits wrongly

in the sweep’s 18,000 episodes, because success requires a confirming inspection. Two hosted LLM agents match its accuracy and not its ~ 5 ms episodes, its determinism or its proofs. Unchanged, the same build scores 134/134 on ALFWorld, fetches 95/96 in AI2-THOR and scores 12/12 on SubT worlds. It passes 21/21 integration checks on stock Nav2 with SLAM Toolbox in simulation, and commits correctly in 36/36 hunts on a hand-held phone camera’s footage of a house. A mind kept across six days of nightly change holds 12/12 every day. Every one of those commitments carries a derivation record that a checker outside the engine replays from a shipped ledger export. §8 sets out the research and development that follows, with an acceptance test per objective.

Reproducibility and data availability

Three things are released. The *benchmark kit* (footnote on the title page) holds the scenario generator, the scripted and exact belief-space baselines of §4.1, the LLM comparison harness, the ROS 2 message definitions, a proof checker, a decision replayer and one verify command. A *demonstration build* of the engine ships beside it, so a reader can run the engine on scenarios of their own. Every artifact this paper reports ships too. The engine’s *source* is not released, including the computations withheld in §2. The consequence is worth stating plainly: the method is not reproducible from this paper, and the numbers and the proofs are. The contributions are therefore stated as measured behaviour, not as mechanism.

Per-episode ledger exports ship for every battery whose commitments the kit’s checker re-validates with no engine code importable: the warehouse held-out battery (`battery_final`), the budget sweeps at 12 and at 100 seeds, calibration, the six head-to-head runs, the absent-target battery, the Wizard-of-Oz house batteries (`woz_test`, `woz_selfimprove`, `woz_dynamic`, `woz_dynamic_night480`) and the unseen-seed audit. That is 10,008 engine episodes and 228,989 proof paths. The remaining evaluations ship as per-episode artifacts rather than ledger exports: ALFWorld, AI2-THOR (whose placements are not bit-stable), SubT, the hospital battery, Mars and seabed, the ROS 2 lanes, the phone battery, the held-out family, the stress battery, the self-repair exams and the edge profile.

What the checker certifies, and what it does not, is worth stating precisely. For every derived claim it requires a proof whose operator comes from a declared table, whose cited inputs are on that episode’s ledger and reached it no later than the claim they support, whose rules the episode knew, and whose time constraints are satisfiable on the cited claims’ intervals. All 228,989 shipped proof paths re-validate under those checks, across the 16 exports and 10,008 episodes the kit ships, with 0 ingestion-order violations. For every episode whose export records a verdict it requires a FOUND verdict to be witnessed by an observed sighting at the resolve bar from something other than the reasoner, and a NOT-FOUND verdict’s certificate to be re-derivable: the room list recomputed from the export’s own LOCATION entities minus the certificate’s declared exclusions, the last claim that placed the target recomputed from the export’s own claims, and one cited first-hand empty-handed look per room no older than it. A certificate that covers a subset of the rooms, leans on a second-hand report or ignores a later sighting fails, and it fails even when its own constraint list has been rewritten to look sound, because nothing in the re-derivation reads that list. Twenty-nine forgery mutations are rejected on that basis. Two limits are measured rather than claimed away. The shipped exports predate the verdict fields, so the commitment and certificate checks apply to the exports made for §4.7 and not to the ten thousand older episodes, which are counted as not checkable rather than passed in silence. And 79.3% of the shipped derived proofs assert no time constraint at all, because 148,136 of them cite a single input and 31,338 cite none, which is a limit of the constraint vocabulary and not of the checker. Above all that, the checker certifies no belief value: how much a hypothesis was worth, and whether the room was the right one, are the engine’s arithmetic and the hidden truth respectively, and neither is inferred from proof validity.

Every configuration switch introduced for the results of this round defaults to the behaviour under which

every other number in this paper was measured, and every shipped number reproduces with those switches in place: the warehouse budget sweep reproduces cell for cell and episode for episode, the held-out family’s shipped arm reproduces its committed artifact row for row, and the two real-pixels replays are unchanged. The artifacts added for this round are the matched belief-space planner’s sweep, the held-out family’s two sweeps, the certified-verdict battery in its three arms, the ALFWorld allowance sweep with the two measured switches beside it (`alfworld_budget_sweep_fix.json`) and the warehouse grid they leave unchanged (`budget_sweep_ties.json`), and the measured budget-aware lookahead, all under `data/artifacts/` with a provenance block naming the commit, the command line and the clock.

A few figures in this paper come from earlier builds and are labelled where they appear. The head-to-head’s original measurement session and the dynamic-house collapse counts are that build’s record and are artifact-logged; the shipped build’s own figures are the ones the kit regenerates. Every number regenerates from integer seeds except the AI2-THOR episodes, which are artifact-logged because the simulator’s placement randomization is not bit-stable. Summary evaluation JSONs ship in `data/artifacts/`, and `docs/EMBODIED.md` carries per-episode tables, stand-up notes and reproduction commands for the embodied evaluations. The real-pixels battery ships its pre-registered manifests, per-episode artifacts and protocol with every sitting’s result (`docs/PHONE_PROTOCOL.md`). Its clips are available on the same request basis as the bundle. `scripts/reproduce.py` runs the native suites in one command, compares every regenerated number with the committed artifact, and restores the committed files. Its report ends with one gaslight episode traced end to end and its counterfactual. These engine-side commands run in the author’s environment. `docs/REPRODUCTION.md` carries the per-suite command block, the regenerate-to-verify list and the artifact bundle’s contents with its manifest and SHA-256 sums. Appendix A is the integration note for a ROS 2 stack. A pre-submission audit recomputed every artifact-backed number from the 2026-09-01 shipped JSONs, and all agreed (the soak growth ratios in §4.8 and the staleness re-rank counts in limitation 4 are quoted from the author’s stress-test and design notes rather than from a shipped JSON); the regenerate-to-verify figures are listed in `docs/REPRODUCTION.md`.

Broader impact statement

The engine is an auditable alternative to LLM decision-making for search-and-localization tasks. Its proof paths and counterfactual queries are designed to make automated decisions contestable, which we view as a positive contribution to accountability in deployed autonomy. Two points of scope apply. One evaluation uses the DARPA Subterranean Challenge’s virtual worlds. Subterranean search is dual-use (disaster response and military reconnaissance alike), but nothing here advances the hard parts of fielding such systems: perception, locomotion and communications. We evaluate a search policy on public competition worlds under public scoring. The hospital-logistics benchmark involves no patient data or clinical decisions. It is a fetch task on a synthetic floor. The engine’s determinism and offline operation reduce data-exfiltration and prompt-injection surfaces relative to LLM-agent deployments. Its guarantees hold at its typed input boundary, whose failure modes §4.8 stress-tests and reports.

Acknowledgments

The system, its benchmarks and this paper were produced by an LLM assistant, Anthropic’s Claude, under the author’s direction. The author supplied goals, evaluation standards and review (limitation 7), has reviewed the paper, and takes full responsibility for its content and claims. The audits and red-team reviews cited throughout were internal: separate sessions of the same assistant, whose findings, including defects in earlier results, are preserved in the repository.

Funding and competing interests

This work received no external funding. The author conducted it as an independent researcher. The author is the owner and chief executive of ModelBrew.ai, which had no role in this work and whose products are unrelated to it.

A Dropping the engine into a ROS 2 stack

This appendix is for a robotics engineer who wants to know what the engine assumes, what an adapter has to supply, and what a minimal integration looks like. The ROS 2 package in `ros2/` is the blueprint; the simulator adapters in `experiments/embodied/` are the same contract without ROS.

What the engine assumes. Three things, and nothing about the platform. (1) A closed ontology: the entities the robot can reason about (locations, containers, persons, objects) are registered by id and type before or during an episode; the engine never invents one from free text. (2) Typed claims: every input is a claim $\langle \text{subject, predicate, object, confidence, source, event time, ingestion time} \rangle$ with a predicate from a fixed list (`LOCATED_AT`, `ABSENT_FROM`, `INSIDE`, `NEAR`, `ENTERED`, `MOVED_TO`, ...). Raw sensor data never enters; a perception module’s job is to emit claims at the confidence it actually has. (3) A small action vocabulary the platform can execute: inspect a location, scan a container, pick up, and the sense call that returns what is visible from the current pose. The engine decides which of these to do next, when a sighting settles the find, and when to give up; it does not drive, map, grasp, or see.

What an adapter supplies. A location map (ids and, on a robot, poses); a *sense* function that returns what is visible from the current pose (in the packaged node, entity ids, the sensor’s confidence in each, and the nearest registered location; the node mints a `LOCATED_AT` claim per sighting at the confidence the sensor reported, 0.95 when a logical sensor reports none, and, when the target is not among them, an `ABSENT_FROM` claim at 0.90, a fixed sensor-model constant of that node; a detector’s own confidence therefore reaches the ledger through the *Sense* answer itself, through *IngestClaim*, or through the vision seam, and a find is settled only when the engine’s own verdict resolves the target where the robot stands); and an *executor* for the action vocabulary that moves the platform and calls *sense*. The adapter is forbidden to read the target’s placement and is graded by the platform’s own signals; every adapter in this paper obeys that rule and `docs/EMBODIED.md` records each one’s sensing model beside its score. Teleport locomotion and simulator-state sensing in the AI2-THOR and gz-sim evaluations are properties of those adapters, chosen to isolate the search policy; they are not requirements of the engine, and the ROS 2 stack replaces both with a navigation server and a sensor node without touching the engine.

The ROS 2 shape. One managed lifecycle node, `cognition_node`, wraps the unchanged `EpisodeRunner`: `configure` reads parameters and registers locations, `activate` accepts goals, `deactivate` refuses them. It exposes one action and three services, and is itself a client of whatever `NavigateToPose` server the platform runs (Nav2’s `bt_navigator` on a robot; a demo driver in the bare sim):

action	/cognition/find_object	robot_mind_msgs/action/FindObject
srv	/cognition/ingest_claim	robot_mind_msgs/srv/IngestClaim
srv	/cognition/query_belief	robot_mind_msgs/srv/QueryBelief
client	/cognition/sense	robot_mind_msgs/srv/Sense
client	/navigate_to_pose	nav2_msgs/action/NavigateToPose

`FindObject` takes a target (entity id or class), a visit budget and whether a grasp pose is wanted, and returns found, the resolved entity and location, the inspections spent, the proof path, and the pose for a

manipulation pipeline; its feedback carries the current status, location and best hypothesis at every step. IngestClaim is the tips protocol over ROS: one typed claim into the live episode's ledger, from any source (a second perception stack, a human), queued by the service thread and drained by the episode thread between actions because the ledger's SQLite connection is thread-bound. QueryBelief is the audit read: the resolved location, ranked hypotheses with their belief breakdowns, and the missing variables the engine would seek next.

A minimal integration. Parameters of the cognition node, then one goal:

```
cognition_node:
  ros__parameters:
    robot_mind_src: /opt/robot_mind/src # the engine, imported unchanged
    location_names: [bay_1, bay_2, bay_3, bay_4]
    location_x:     [ 6.0,  6.0, -6.0, -6.0]
    location_y:     [ 6.0, -6.0,  6.0, -6.0]
    view_dist:      2.2 # stand-off from a location, metres
    goal_frame:     map # odom for the demo driver, map on Nav2

ros2 action send_goal /cognition/find_object \
  robot_mind_msgs/action/FindObject \
  "{target: target_box, max_visits: 12, want_grasp_pose: true}"
```

A sensor node answers Sense with the entity ids it sees and its confidence in each: the logical frustum sensor of the first two lanes (3.0 m, 0.7 rad half-angle, a frustum test over bridged physics poses, no confidence field), or the camera lane's yolo_sense_node (YOLO-World on the bridged RGB frame, detections bound to entities by projecting their poses through the camera intrinsics, the detector's score returned per id); on a robot it is the platform's detector and tracker, through the same seam the AI2-THOR real-detector battery exercises (ingestion/vision_adapter.py: a DetectionEvent with label, camera id, confidence and time becomes an OBSERVED claim at the detector's confidence, and the association between detections and entities is the tracker, whose failure modes the engine's verify loop exists to catch). Nothing else changes between the demo driver, stock Nav2 in simulation and the camera lane; the same client would run the same list on hardware. The integration checks are one list (20 checks; 21 with Nav2 readiness; 26 with the camera lane's detector facts, a frame per inspection, two boxes in view, verdict-gated confirmations and no wrong-instance commitment), and the numbers in §4.5 are what it read.

The Nav2 stack behind the 21/21, and one caveat. Lane two of §4.5 ran stock Nav2 1.3.12 (bt_navigator, the NavFn planner, the DWB controller, the behavior server and the velocity smoother), with localization from SLAM Toolbox over a roof lidar. One caveat bounds the 21/21. Under Nav2 the frustum sensor (3.0 m range, 0.7 rad half-angle, 2.2 m stand-off) looks from the robot body's physics pose, a stand-in for a localized robot. The search is what is graded, and the localization columns of Table 17 say what the stand-in cost. A sensor at the SLAM pose would have stood within 9 cm of the vantage used, against a 3 m sensing range. Nav2's parameters were changed only for this body and arena. The inventory, the plugin names read from the running servers, and the four bring-up runs are in Appendix B and in ros2/README.md.

Operating the engine. An operator audits a decision through the append-only ledger, proof replay by the checker, the QueryBelief query over ROS 2 and the whatif counterfactual re-run. Each returns the following. The append-only ledger holds every claim with its source, two clocks and its status history. Over ROS 2, QueryBelief returns the resolved location, the ranked hypotheses with their belief breakdowns, and the missing variables the engine would seek next. The whatif command re-runs an episode with a claim

removed and reports what changed. The `FindObject` action’s feedback carries the current best hypothesis and status at every step. Uncertainty is signalled as the ranked beam and its missing variables rather than a single answer. Learned rules reach deployment only through the candidate $\rightarrow$ provisional $\rightarrow$ validated lifecycle, with held-out precision recorded, and even then act as advisory priors that an observation overrides. No path lets a learned rule create an OBSERVED fact.

What is not built. A human-in-the-loop review step for rule promotion, a signalling policy for a particular high-stakes setting, MoveIt 2 beyond the grasp-pose hand-off, and any run on hardware.

B Development record: what was tried, what failed, what changed

This appendix carries the chronology that the main text summarises: the development history of each battery, the runs that failed and what was changed, with every number that left the main text.

Architecture: module inventory and the capabilities added later

§2 describes only the loop that runs on every decision; this block lists the modules and the capabilities added after v0.

Modules. The single Python application (74 source files) has ten modules. Five handle input and memory: ingestion and normalization, entity and time grounding, the SQLite evidence ledger, the belief-graph view over trusted claims, and salience. Grounding uses weighted identity that preserves ambiguity below threshold, and salience detects contradictions and gaps. Five reason and act: the hypothesis engine, the relational reasoner over carrier chains, the information-gain planner, the simulated-warehouse executor with verified postconditions, and the evaluator and learner. Version v0 shipped the deterministic template implementation of the `CognitiveProposer` protocol, with no model dependency.

The modules of a persistent mind. Later versions added temporal decay of evidence, run-compression of repeated observations, drives with utility arbitration, staleness-sized patrols, declarative plans with fallbacks and a calibration meter. A metacognitive layer keeps expectation records that settle every module’s predictions, diagnoses typed violations, and drives the gated self-repair loop of §4.8.

The learned rule and the three guarantees of a valid proof

The transport rule of §4.1 is mined from the engine’s own episodes with support 450, training precision 0.71 and held-out precision 0.77; its deployed reliability is 0.71, the lower of the two. It is promoted through a candidate $\rightarrow$ provisional $\rightarrow$ validated lifecycle and deployed only after validation, after which proof paths cite the learned rule and its measured reliability. The contextual bandit over action types is trained with ϵ -greedy exploration and evaluated frozen. The phrase “valid proof” covers three separate guarantees, of which the paper certifies the first in full. *Provenance integrity*: every cited claim and rule exists in the ledger and the stated time constraints hold. The checker verifies this by replay, and it is 1.000 in every reported run. *Logical consistency*: the conclusion follows from the declared rules. The checker confirms the rule ids and their arity but does not re-derive the conclusion, so this is checked structurally, not semantically. *Empirical correctness*: the conclusion matches independent ground truth. This is measured separately, as the correct-commitment rates reported with each result, and never inferred from proof validity. A valid trail says the decision was made from these inputs by these rules. It never says the inputs were true.

Scripted baseline definitions

The scripted policies of Table 7 run 12 seeds per level on levels 2–6, under the same contract as the LLM agents: same generator, executor, 8-action budget, success rule and hidden-truth grading. The *sweep* checks all locations, then all carts, in sorted order, and ignores the evidence. The *random-order* walk visits the same places in a seeded random order. The *last-seen-first* heuristic checks the target’s most recent sighting, then sweeps. The *Bayesian belief filter* keeps a posterior over holders (locations, carts, persons) and checks the argmax. It has the belief updating any POMDP-lite tracker provides, given the same evidence, and none of the engine’s machinery: no ledger, no epistemic statuses, no contradiction bookkeeping, no proof paths, no multi-anchor hypotheses, no transport handoff. Scripted baselines were run before the engine’s number was read, because they show what a benchmark measures; the ALFWorld ablation of §4.4 is the case in point. On the gaslight level the random order cannot be deceived and has no fixed order to be unlucky in, which is why it pays the least of the scripted policies. Naive evidence use is worse than no evidence use, and disciplined evidence use beats both. The LLM rows of Table 11 read in this light: the 7–8B models fail episodes that a for-loop passes, so their deficit is sustained systematicity, not search skill.

Budget sweep configuration

The budget sweep changes one field of the benchmark’s configuration (`experiments/budget_sweep.py`). The action budget is the only change; generator, executor, success rule and grading are those of the LLM contract. At eight actions every policy is at 100%, and the 12-seed subset of the sweep reproduces the scripted and planner rows of Table 7. The engine’s step count differs by build and by seed family, and Table 23 puts every figure this paper quotes in one place. Zero wrong commitments is a structural property of the success rule, shared by the random walk; what a smaller budget removes is the chance to inspect everything. Two shapes in Figure 2 carry the mechanism. The random walk climbs steadily with budget, as coverage should. The belief filter stalls at 42–44% from four actions to six, because argmax-posterior over the same evidence locks onto the wrong branch of a transport chain and keeps confirming it. The engine’s multi-anchor hypotheses and transport handoff turn that same evidence into the right first look.

build and seed family	L2–5	L2–6
original campaign build, 12 head-to-head seeds	1.75	2.05
current build with its learned rule, the same 12 seeds	1.83	2.10
current build with its learned rule, sweep seeds 20000–20099	1.80	2.05
learned layer, 300 held-out episodes, seeds 10000–10074	1.47	—

Table 23: Mean actions to success at the eight-action budget, by build and seed family. Every step figure quoted in §4.1, Table 7, Table 6 and Table 8 is a row of this table. The 1.75 of the abstract is the first row.

The exact belief-space planner

The planner of §4.1 is solved to optimality rather than approximated, because the benchmark’s search problem has a closed structure. The world is static during a search. A check is a Bernoulli sensor with a known hit rate (0.97 inspect, 0.98 scan, the executor’s own constants) over the holders it can reveal. A hit ends the episode, and the Bayes updates for misses commute. The finite-horizon value $V(S, k) = \max_{a \notin S} [p_a(b_S) + (1 - p_a(b_S))V(S \cup \{a\}, k - 1)]$ is therefore a dynamic programme over subsets of the eight-action menu (`experiments/belief_planner.py`). It starts from the belief filter’s reading of the briefing evidence and uses the true sensor model, so it is the optimal policy for the POMDP the filter’s model defines. At the full budget it finds everything in 3.12 actions to the filter’s 4.80. Table 24 breaks the

four-action cell of Table 9 down by level, which is where the arms separate: the carrier and cart-chain levels are solved by everything, and the deceptive levels are the whole of the difference.

arm, at four actions	L2	L3	L4	L5	L6
engine	1.00	1.00	0.98	0.99	0.95
engine without multi-anchor hypotheses	1.00	1.00	0.98	0.59	0.07
exact planner over the plain belief	1.00	1.00	0.73	0.57	0.37
exact planner over the matched belief	1.00	1.00	1.00	1.00	0.88

Table 24: The four-action cell of Table 9 by level, 100 seeds per cell. L4 is stale claims and decoys, L5 contradictions and occlusion, L6 the adversarial planted report. The plain filter finds 42.6% overall at this budget.

Calibration methodology

The calibration numbers of §4.1 pool every final-beam hypothesis against hidden truth, 25 seeds per level. Levels 1–5 give ECE 0.047 (Brier 0.097); the gaslight level gives ECE 0.065 (Brier 0.116). The reliability diagrams (Figure 3) show a conservative signature: beliefs above 0.7 are empirically right 100% of the time, and the only material overconfidence sits in the 0.1–0.3 bins. Adversarial evidence degrades calibration slightly and never inverts it.

Transfer ladder and Wizard-of-Oz ladder

Beyond the static benchmark, a transfer ladder drove the engine’s growth: each rung exposed a missing capability, each capability was built as a general module, and the warehouse battery was held at 100% throughout. A home world (dishwashers that hide items, wandering people) tests transfer; Table 19 gives every rung’s score. The rung-3 procedure also runs as data, a declarative plan with dependency edges and fallback branches interpreted by a generic plan runner, with measured pass/fail parity against the hand-coded sequencer on every seed (`battery_plans.json`). Procedurally generated houses entered with an empty ledger failed at first (W2 5/25, W3 0/25); the missing capability was an exploration drive. The metacognitive supervisor clustered its failures into machine-readable capability requests and the requested modules were built, which is the middle column of that table’s upper block; the v0.3 memory modules are its right-hand column.

Re-taking the same house daily with memory persisting separates two claims. On the pre-drive engine the ten-attempt curve was flat, 4/15 on every attempt, a deterministic fixpoint: a house whose rooms were all known was still never entered beyond its last sighting, because nothing connected an unseen goal to an unvisited room. Tuning choices among conceivable actions cannot conceive new ones. The drive and fallback modules did, and the battery’s self-improvement test rose across the three arms of Table 19’s last ladder row, passing on the first attempt (attempt 1 = attempt 6, slope 0; `woz_selfimprove.json`, `battery_final.json`).

The dynamic house

The dynamic-house protocol separates memory from learning: twelve houses, six days each, 1–2 items moved nightly, the mind persisting. The middle block of Table 19 gives the day-by-day counts for every arm. Persistent memory without forgetting was a liability in a changing world; decay plus refutation-driven search turned it into an asset, and that is the whole of the reading.

Head-to-head: the reasoning model, the scaffold, the adversary and the hybrid seam

These details from §4.3 do not change its numbers. Two local models were run: an older instruct-only 7B and a newer ~8B reasoning model whose explicit chain of thought is the belief tracking at issue. The reasoning model scores 45/48 against the 7B’s 40/48 at roughly $8\times$ the latency, so the accuracy gap narrows as the model is allowed to reason. The strong scaffold’s strategy is: enumerate competing explanations, including “the cart left empty”; choose the discriminating test; never repeat an empty check. Under it the 7B model lifted L3 to 12/12 but lost ground at L5, and its failures still concentrate on the deceptive branches. The model can recite the hypothesis list and still follow the salient cart trail into a dead end, because nothing in its context durably tracks what its own failed checks have ruled out.

The L6 adversary’s further moves, beyond the confident fake sighting, are sometimes a second contradictory sighting, a staged same-type twin object at the fake location, and occasionally a false absence report at the true location. The engine’s multi-anchor hypothesis generation keeps the true carrier chain alive regardless of what the liar says. In the CognitiveProposer seam, ambiguous names in a natural request are resolved against the entity registry rather than guessed.

ALFWorld: what the benchmark tests, and the seen-split defect

The scripted ablations of §4.4 support one reading of the benchmark, and the second run followed one engine fix. ALFWorld’s difficulty for LLM agents is sustained systematicity, durable state tracking and never rechecking. A ledger-based mind, a for-loop and a random walk all possess those, and current LLM agents do not. The seen-split defect was a “ghost anchor”: after the first instance of a two-object task was collected, a lingering class-level sighting let the refuted-fallback resurrect already-checked rooms. The fix is belief hygiene. The stale sighting is retired through the ledger’s own append-only supersede mechanism, and the give-up condition was corrected. Both were developed and validated on the seen split alone (134→138/140) before the unseen split was run a second time.

The reduced-allowance diagnosis of §4.4 came from replaying all 134 unseen games under each policy and recording the ranking in front of the engine at every decision (`experiments/alfworld_trace.py`; the replays reproduce the committed per-game rows on both `won` and `env_steps`). Of 1,712 decisions on that split, 1,411 were taken with every remaining candidate carrying the identical value, 166 with one place already settled by evidence and 135 with the candidates’ values actually differing. The adapter registers every receptacle at the same coordinate, so the planner’s cost term is constant and cannot break the tie either. The four unseen households are a kitchen (77 tasks), a bedroom (27), a bathroom (19) and a living room (11); the two splits disagree about the fixed alphabetical sweep because a seen kitchen holds 14.1 cabinets on average and the one unseen kitchen holds six, so the forward order opens far fewer doors before it reaches a countertop.

AI2-THOR: planner, verification seeds and the fix campaign

The 95/96 battery of §4.4 carried two verification seeds, and its planner has a classical form. The next-best-view planner is active perception in its classic set-cover formulation [12, 42]. Seeds 137 and 314 were never used to diagnose a fix; 137 ran 12/12, and the campaign’s one miss fell on 314. The engine was untouched throughout. The planner’s construction and the adapter-side fix campaign, including the two vantage strategies that were tested and rejected, are documented in `docs/EMBODIED.md`. The text-world result of §4.4 carries over to the 3D and physics simulators it abstracts.

SubT: stack status and visit-count audit

Three facts behind the 12/12 SubT result. The worlds load content-unmodified in gz-sim Harmonic, headless and CPU-only. The competition’s cloudsim stack is end-of-life, so what runs is DARPA’s worlds and DARPA’s scoring rule, not the competition infrastructure. An internal audit re-derived every visit count from world geometry alone, and the per-episode counts (2–87 of a 250-visit budget) match that derivation.

Hospital campaign: protocol rules, seeds, artifacts, reproduction and the adapter fix

These are the protocol rules and run-level facts behind the hospital numbers of §4.4. In the plain fetch protocol, targets come only from classes with one instance in the world, since a resident sibling would satisfy a class-level find while grading demands the relocated item. Two episodes drew truth spots inside the first visit’s sensing bubble, the deepest episode worked through 21 of the 102 spots, and per-episode visit counts keep that 1–21 spread visible. The evaluator restores world state after each episode, so the simulator is as fresh per episode as the engine’s ledger.

In the planted-tip protocol the tip, when false, names a decoy spot. The three deciders share one bridge, budget and grading, and the naive tip-follower visits the tip spot first and then falls back to sweep order. The standard eight seeds drew six true and two false tips. The four balancing seeds were chosen after that split was known, by their precomputed tip coin alone and before any of them ran; the eight-seed and four-seed artifacts ship separately. Every policy’s 12/12 follows from the protocol, which guarantees it for any complete searcher that commits only on sightings. The tip-follower’s true-tip visit count is its policy by construction; the engine’s matching count is measured behavior, the reported spot ranked first in all six episodes. The false-tip permutation result of Table 15 has a plain cause: one debunked tip is too little evidence structure for belief arithmetic to beat a fixed fallback order. The claims that carry weight are the true-tip collapse to 1.00 visits, the zero wrong commitments, and the ledger trail that shows each debunking. The protocol is the warehouse verify-then-redirect pattern (§4.1) reproduced in a physics simulator on the AWS RoboMaker hospital world’s floor plan.

In the decoy round the look-alike is a spawned twin instance. The class-matching tip-follower’s five-of-six wrong commitments come at one visit each, fast and wrong, and the sweep is fooled in four of six because its fixed route reaches the staged twin before the true item. The engine is not fooled because the look-alike’s sighting enters the ledger under its own instance id, the same read records the named target’s absence, the tip’s belief collapses, and the search redirects. That identity check is not a bolted-on rule; it is carried in the entity bookkeeping. The thirty-seed power run takes the first thirty integer seeds whose tip coin is false, declared in docs/EMBODIED.md before running. Its artifact (`hospital_tips_twin_30.json`) was assembled from three launches after two Gazebo transport stalls, with every launch and log recorded there. Of the six seeds shared with the six-episode round, four reproduced the engine’s visit count and two moved: seed 3 from 22 to 36 visits and seed 5 from 24 to 25, while every scripted arm reproduced. The code path is identical. The likeliest cause is that the engine arm runs first after the target and the look-alike are placed, and sensed before the placed bodies had settled in physics. The counts are reported as measured.

Two facts about the campaign build. A probe of the slowest false-tip episode found the adapter under-reporting its own sensor: absence was recorded for the visited spot alone, at lower confidence than the positive channel of the same read. With the sensor reported faithfully that episode fell from 96 to 24 visits. The whole hospital campaign, plain fetch included, was re-run on the fixed build, so every hospital number in the paper and in Table 13 is post-fix. The engine also re-verifies a debunked tip up to three times across its seek restarts, because repetition-decay state resets per seek and, with no positive evidence anywhere, the debunked tip briefly returns to the argmax. Those checks are counted in the 14.67 and cost no commitment.

Mars and seabed: worlds, placements, visit counts and what is not claimed

The two field-geometry batteries of §4.4 are specified here in full. The Space ROS Mars rover demonstration’s Curiosity terrain runs from the demo packages built on ROS 2 Jazzy with gz-sim Harmonic. The terrain is a 304 m square with 28 m of relief, of which a 72 m square is gridded into 169 survey stations at a 6 m pitch, sensed by a logical 8 m mast sensor. Project DAVE’s ocean-objects world holds 11 unique-instance seabed objects within 26 models, a flight data recorder, unexploded ordnance and hard hats among them, over 70 survey stations. Its sensor is a BlueView P900 sonar footprint with a declared range-to-confidence curve and fan-shaped absence bookkeeping. Placements are pre-registered per seed and confirmed by reading the entity state back before the first sense, and truth is the station nearest the read-back pose.

Visit counts, four seeds per arm with engine, sweep and random-order arms sharing the executor, budget and sensor, are in Table 25. Every arm is 4/4 in both worlds with zero wrong commitments in all 24 episodes. Until evidence arrives the engine’s order is a nearest-first sweep, which is why a random order can beat it by luck at $n = 4$. The planted-tip and staged-decoy variants for both worlds are pre-registered with tip coins drawn and seeds fixed, and have not been run. Not claimed: driving the rover or the vehicle, sonar perception, Space ROS itself (the demo packages run on vanilla Jazzy), or lunar worlds. HoloOcean is blocked on WSL2 and Stonefish is the viable fallback (docs/SPACE_MARINE.md).

world	stations	pitch	mean visits			engine range
			engine	sweep	random	
Mars terrain (Curiosity demo)	169	6 m	31.0	45.0	26.0	21–58
Seabed (Project DAVE)	70	5 m	7.75	21.25	4.25	2–12

Table 25: The two field-geometry batteries, four seeds per arm, all three arms sharing executor, budget and sensor. Every arm is 4/4 correct in both worlds with 0 wrong commitments in all 24 episodes. Under uniform priors a random order can beat the engine by luck at $n = 4$, so no efficiency claim rests on these columns.

The AI2-THOR detector battery: history, misses and operating point

This block carries the development history, the per-miss detector scores and the operating point of the real-detector battery in §4.5.

- *Fresh draws.* AI2-THOR redraws placements per launch. The eight seeds are therefore fresh draws of each configuration, not replays of the oracle-sensing episodes.
- *First campaign at 480².* A seed-7 smoke went from 5/12 to 10/12 with six adapter-side, target-blind fixes. The campaign scored 27/36 on seeds 7, 11 and 23. Rendering at 1080² recovered three of the remaining episodes. The confirmation-ordering fix recovered a fourth.
- *Which seed produced which fix.* The synonym table was built from seed-7 misses. The confirmation-ordering fix was diagnosed on seed 11.
- *The one adapter defect.* The campaign exposed an ordering bug in the session’s give-up test. It was fixed so that a sighting between the claim gate and the confirmation bar resolves by the engine’s own verdict. The engine itself was not touched.
- *Crash reruns.* Two held-out episodes (FloorPlan203 on seeds 42 and 137) crashed at simulator start-up. The cause was a Unity initialisation timeout under GPU contention, before a target had been chosen. Each was rerun alone as a fresh draw. Both fetched. The crash records ship in the artifacts, marked superseded.

- *Misses below the detector’s recall (eight).* Two forks and a butter knife inside a garbage can. A keychain inside a can. Three credit cards lying flat. An alarm clock on a shelf that the detector localized at IoU 0.97 but never named above 0.10.
- *Mislabeled (four).* Each was localized at IoU 0.84–0.93 but named as something else. A jar candle read as “Cup” at 0.54. A pepper shaker read as “SoapBottle” at 0.92. A butter knife on a dining table read as “Pen” at 0.27. A plunger read as “Spatula” or “BaseballBat” at 0.15. None was added to the synonym table, because each is a single spawn under the two-spawn rule.
- *Never rendered (one).* A CD inside a box on a bed, a container the adapter does not open.
- *Probing.* Every miss was probed from the receptacle’s planned vantages and from approach poses (`scripts/thor_detector_probe.py`). Each was reported not-found after the full budget (`docs/EMBODIED.md`).
- *Operating point.* Weights `yolov8x-worldv2`. Inference at `imgsz 1088` on the 1080^2 render. A raw detector floor of 0.02, so that weak candidates reach the associator. Association by $\text{IoU} \geq 0.35$ against the instance’s own rendered box. The label must match the instance’s class or one of eleven fixed synonym groups: `Bottle/WineBottle/SoapBottle/SprayBottle/WateringCan`, `Cup/Mug`, `Towel/HandTowel/Cloth`, `Pot/Pan/Kettle/Bowl`, `Pen/Pencil`, `DishSponge/SoapBar/Cloth/Book`, and five more in `adapters/thor_yolo.py`. A 0.05 confidence gate at the ledger. The detector is prompted once per process with the 62-type vocabulary.
- *Cost.* A fetched episode consumed a median of 10 detector frames (1,766 over the 36 episodes). Inference costs about 42 ms per frame on the laptop GPU used (`docs/EMBODIED.md`). The software-rendered simulator costs seconds per frame. The detector is therefore not where the 259 s mean wall time per fetched episode goes.
- *The seam’s event type.* The `DetectionEvent` the seam accepts carries a label, camera id, confidence, time and an optional `track_id`. On a platform with a tracker that field is filled. The track is then linked to an entity through the same weighted resolver the warehouse benchmark uses, and the association step described above does not exist. That configuration is the next build (Table 22, last row).

ROS 2 bring-up: bridge runs, the packaged node, Nav2 parameters, plugin names, the four diagnostic runs, and the camera lane’s sensor node

This block is the bring-up record behind the ROS 2 lanes of §4.5; `ros2/README.md` annotates every parameter change.

- *Bridge runs before the packaged node.* A logical-camera scout hunted a relocated box over the `gz.transport13` bridge to Gazebo Harmonic, 4/4 episodes correct. The same loop then ran over ROS 2 topics (Jazzy, `ros_gz`). It drove a diff-drive base through `/cmd_vel` and odometry with no teleports, 4/4.
- *The packaged node.* Beside the `FindObject` action and the `NavigateToPose` client, the node exposes claim-ingest and belief-query services. The false tip of the demo-driver lane was injected mid-episode through the ingest service. It was verified and abandoned over ROS: `bay_4`, then `bay_3`, then `bay_4` re-verified, then `bay_2`, the truth. Under uniform priors the node inspects in the fixed order `bay_4`, `bay_3`, `bay_2`, `bay_1`, so the find counts of 2, 4, 1 and 3 inspections are the truth’s position in that sweep, and the tip is the discriminating lane.

- *Parameters.* The DWB local planner runs at 0.8 m/s and 0.5 rad/s, with 0.5 m/s² and 0.5 rad/s² ramps and a 0.6 velocity-smoother backstop. The ramps keep the sphere-wheeled body’s odometry accurate, because ramped commands do not slip the wheels. MPPI at the Jazzy default crawled at 0.09 m/s on this body. Regulated Pure Pursuit stalled on the 180° bay-to-bay turns once angular acceleration was capped. NavFn plans over a global costmap built from live scans rather than the SLAM map, with 0.1 m costmaps and a 0.25 m goal tolerance. SLAM Toolbox adds a graph node per 1.0 m or 0.5 rad and publishes a 0.1 m map every 5 s. Transform tolerance is 0.5 s throughout. Wall time per inspection is almost all driving between bay stand-off poses.
- *Plugin names.* The artifacts name the plugins as read from the running servers (nav2_facts: dwb_core::DWBLocalPlanner, nav2_navfn_planner::NavfnPlanner, Nav2 1.3.12). The Nav2-lane artifact’s controller query timed out during bring-up. It records an empty controller_plugin, with controller_server (unknown) in nav2_nodes, rather than a typed-in name. The sensor_vantage, sensor_vantage: physics_pose, is read from the running sensor node in the same way.
- *Worlds.* The artifacts were regenerated on the final world, whose bay boxes are the cardboard cubes of the camera lane. The first run of the checks, on 1.0 m posts, read 0.093 m of odometry drift and 0.047 m of SLAM error with the same inspection sequences. The demo-driver checks, regenerated on the same final world, pass 20/20 with identical sequences.
- *Four diagnostic runs.* Bringing Nav2 up took four runs. Each was resolved in the simulated body or in Nav2/SLAM configuration, and none in cognition: odometry-frame sensing, an axle/origin offset, an intermittent boot failure, and SLAM starving the navigator. Run 4 is the other end of the navigation-failure path. With SLAM starving the navigator, eight navigation attempts, five of them aborting within 60 ms, ended in not-found rather than a guess.
- *The camera lane’s sensor node.* The yolo_sense_node runs YOLO-World on each frame with a fixed generic vocabulary (“cardboard box”, “crate”, “barrel”, “pillar”, ...; never an entity id). It binds each box detection to a known entity by projecting that entity’s physics pose through the camera intrinsics (IoU $\geq$ 0.35, each entity bound once). The AI2-THOR battery’s 0.02 floor, 0.05 claim gate and 0.35 gate are untouched for this world. The node answers with the entity ids and the detector’s own confidence. The cognition node writes that number into the LOCATED_AT claim. Across the five episodes 11 frames held two or more box detections and every one was bound to its own projected entity, with all 5 confirmations following the engine’s verdict. Detector time was 49–403 ms per frame after a 2.0 s first-frame warm-up, measured on the laptop GPU at a load average of 0.86, and the annotated frames ship in the artifact bundle. Odometry drift stayed $\leq$ 0.099 m and SLAM error $\leq$ 0.048 m in this lane, against $\leq$ 0.099 m and $\leq$ 0.082 m under the frustum sensor; each inspection cost 24–38 s of wall time, almost all of it driving at no more than 0.8 m/s.
- *The camera lane’s confirmation rule.* The episode runner settles a find only on an observation at $\geq$ 0.9. A detector score of 0.3–0.6 never is, and must not be inflated to it. The executor therefore writes the raw sighting at the detector’s number. It adds a single inspection-confirmation claim (0.97, provenance confirmed_by: engine_verdict and detector_confidence) only when the engine’s own verdict resolves the target at the bay the robot is standing at. The artifact lists every such confirmation with the verdict that opened it.
- *The camera-lane timeout.* In the tip episode of the YOLO-World lane, Nav2 reported “Failed to make progress” on the 135° reversal out of the bay_2 stand-off. It cleared its costmap and recovered, and the retry outran the node’s 120 s result wait. The engine took the failed action and re-planned, which is what reordered the inspections: bay_3, bay_4, bay_4, bay_2 against bay_4, bay_3, bay_4, bay_2

under the frustum sensor, in five attempts for four inspections. The decisions were the same as under the frustum sensor: verify the tip twice, then redirect. The tip episode under the frustum sensor cost 246 s against 56 s under the demo driver, which includes its own 120 s timeout.

The phone battery: sittings one to four

This block records the four phone sittings of §4.5 in order, with the three fixes made between the first graded sitting and the pre-registered fourth.

1. *Sitting one.* Filmed off its manifest. Kept as a pilot and not graded.
2. *Sitting two, the first graded.* Pre-registered as a manifest with the engine and sense model fixed before the clips existed (docs/PHONE_PROTOCOL.md). The engine scored 12/36, level with the sweep (12/36) and the random order (13/36). Raw detector output had found the one place where the loop's resolve-on-sighting rule is wrong. The rule is right wherever an inspection can confirm a sighting. A clip cannot, because the same clip returns the same detection. Three causes were fixed. First, the phone adapter now sets a sighting bar of 0.9, the loop's own confirmation confidence; the engine default is unchanged and every native suite byte-identical. Below the bar a sighting is a hypothesis, the search goes on, and an exhausted search commits to its uniquely best-supported zone. Second, a coverage-bookkeeping fault in the harness's search loop. Third, a sense-model vocabulary (aliases, distractor prompts) for a detector prompted with household words. Re-run on the same clips, the fixed engine scored 29/36. Every one of the seven wrong is a perception error named in the artifact. A power bank and a photo frame read as the phone. A white mug went unseen on white marble.
3. *Sitting three.* Pre-registered on the fixed engine. It scored 33/36, with 15/18 true tips followed and 18/18 false tips debunked. The three wrong were again perception. The remote read as a phone above a phone lying on a dark bench without its placemat. The tumbler read as a soap bottle.
4. *Sense model v3.* One label per object across every class, with synonym folding, on the same weights and prompt list. It was chosen on the graded footage of sittings two and three (29/36 unchanged; 33/36 to 35/36), then pre-registered for sitting four.
5. *Sitting four.* One run, nothing changed after seeing it: 36/36 finds, 18/18 true tips followed, 18/18 false tips debunked (§4.5).

The lifelong curve: two protocol facts

Two protocol facts bound the reading of the 95-to-10 habit result in §4.6.

- The seeded coin drew habit episodes for all ten post-validation episodes. The validated habit's off-habit cost therefore rests on the four-episode wrong-habit run (+1, −13, +7, −15 actions).
- A first curve on this protocol validated the habit and saved nothing, because the class rule had been instantiated on a single countertop. The defect was fixed before the reported curve. Both curves ship.

Self-repair exams v1 and v2

This block carries the two sealed fault-injection exams behind the self-repair paragraph of §4.8, and the review that turned the first into the second.

Table 26 gives exam v1. Exam v2 jitters every numeric field (46 differ from the archive) before one is sabotaged hard, and adoption uses only the engine's own success signal. Both diff-defeating lanes recover

sabotage	damage	diagnosis	repair adopted	recovery
memory decay constant cut 40×	0.965 → 0.256	SEARCH_EXHAUSTION	sabotaged field, restored	100%
eyes tp 0.97 → 0.01	0.965 → 0.790	FRUITLESS_ACTIONS	sabotaged field, restored	100%
scanner tp 0.98 → 0.01	0.965 → 0.026	FRUITLESS_ACTIONS	sabotaged field, restored	100%
repetition-decay constant 4× weaker	0.965 → 0.826	(none — below threshold)	(none, correctly)	0%
nothing (healthy)	—	(none)	(none, correctly)	—

Table 26: Self-repair exam v1: the three detectable single-field sabotages are diagnosed and repaired to 100% recovery, the sub-threshold sabotage and the healthy control are correctly left untouched, and the placebo passes in 0% of raced lanes. “tp” is a sensor’s true-positive rate.

100%/98% by touching only the sabotaged field among 46 candidates. A no-archive lane recovers 99% by pure factor-climbing, and the drift-only control rests with zero false repairs. One lane fails: the no-archive scanner catastrophe stays broken, because greedy one-edit repair cannot cross score-flat valleys without an archive. Sabotages come from the v1 detectable set, so the capability shown is recovery of pre-screened, detectable, single-field faults under drift. Self-repair here means configuration repair behind an expectation monitor, with four ceilings (limitation 9). The loop cannot invent a missing representation, fix a wrong objective, repair its own diagnostic machinery, or see a failure it holds no expectation about.

The review of exam v1, and the lanes that adopted nothing. These are the reasons exam v2 was built, and what the lanes that adopted nothing, or adopted the wrong thing, did. An internal review of exam v1 (Table 26), run as a separate session of the same assistant, raised four objections. The sabotage roster was screened for detectability. A one-line config diff against the archive would solve every v1 lane. The v1 candidate repairs were selected by the exam’s hidden-truth score. The violation vocabulary names ~25% of the numeric config surface. Exam v2 was built to answer them by construction (§4.8). In exam v1 the sub-threshold lane weakened the repetition-decay constant fourfold, which cut success from 0.965 to 0.826. That damage sits under the violation threshold, so no violation was raised, no repair was proposed, and recovery is 0%, the correct outcome. The healthy control likewise produced no violation and no repair. In exam v2 the failing no-archive scanner lane did adopt one non-sabotaged-field edit that passed the gates without helping, recorded in its artifact. The 120-corruption sweep of the stress battery (§4.8) shows the loop’s restraint at scale.

The absent-target battery: verification of the manipulation, gate counts and review history

This block carries the detail that left §4.7, with every number. The sweep labels every arm with the bar it ran at and every verdict with the path, direct or exhaustion, that produced it. The absence manipulation was verified rather than assumed: every write path into world state was enumerated, world truth reads *unknown* in 300/300 episodes, no episode materialises the target at run time over 1,800 episodes, and the seed streams are byte-identical to the controls. In the absent arm at bar 0.9 the weak-sighting branch of the instrumented gate is entered 1,210 times across the arm’s 3,000 reasoning calls. The exhaustion verdict fires 799 times; 678 of those are retracted by the inspection they trigger, and 121 are still held when the budget ends. In the length-matched control at full budget the direct path names no wrong room, and with a strong sighting in the ledger the exhaustion rule never decides the episode.

Three separate-session reviews of the harness preceded acceptance of its numbers. Each began from the assumption that the headline bucket was unreachable by construction; that failure mode, a headline bucket no episode could reach, had occurred twice earlier in the project. The accepted figures were re-derived by a classifier route that never touches the harness’s reporting code. Of the two diagnostic arms, the bar-0.0 absent arm shows 36/300 transient direct-path verdicts and 0/300 held at the end. They rest on a level-5

low-confidence camera artefact (0.45–0.62) that passes the direct path’s freshness test while the confidence test is vacuous at a bar of zero. They occur in the same episodes where the present-target control holds a transient verdict, and the first inspection retracts them, so the 0/300 is an anchor-freshness window and not restraint. The evidence-stripped arm returns the same uniform belief vector in 300/300 episodes, a wiring check that no verdict is invented from nothing.

Detail moved from the numbered limitations

Each paragraph below carries the material cut from the limitation it names, with every number.

Limitation 4, the probes behind rule set v0.2.0. The probes that produced the current rule set (v0.2.0) were hand-built temporal scenarios run after the first campaign. They exposed two defects the benchmark had not surfaced: rooms never seen could outrank the last-seen room on staleness alone, and containers could not be located. Both were fixed before the budget sweep and the planner race. Original-campaign and post-fix engine figures are both reported, in Table 23.

Limitation 8, the Wilson 95% intervals. Table 27 collects every interval the paper quotes for a k/n count in one place. The embodied intervals are wide at these sample sizes, which is the limitation.

what it counts	k/n	Wilson 95%
AI2-THOR fetches, oracle sensing	95/96	[0.94, 1.00]
AI2-THOR fetches, real detector	83/96	[0.78, 0.92]
held-out seeds alone	52/60	[0.76, 0.93]
SubT reports scored	12/12	[0.76, 1.00]
hospital fetches	8/8	[0.68, 1.00]
staged decoys, first round: not fooled	6/6	[0.61, 1.00]
staged decoys, pre-registered run: fooled	0/30	[0.00, 0.11]
Mars and seabed, per arm	4/4	[0.51, 1.00]
ROS 2 packaged-stack checks	20/20	[0.84, 1.00]
real-pixels hunts, fourth sitting	36/36	lower bound 0.90
head-to-head, engine and both hosted models	48/48	[0.93, 1.00]
the 8B reasoning model	45/48	[0.83, 0.98]
the 7B instruct model	40/48	[0.70, 0.91]

Table 27: Every Wilson 95% interval quoted in this paper for a k/n count, in one place. The warehouse and held-out sweeps carry their intervals in their own tables, where the cell counts are 500 and 600 episodes rather than single digits.

Limitation 8, the decoy power run. The six-seed decoy round is directional rather than powered: 0 of 6 fooled has a Wilson upper bound of 0.39, which excludes a policy fooled most of the time but not one fooled a third of the time. The 30-seed powered run used the first thirty integer seeds whose tip coin is false. It was declared in docs/EMBODIED.md before running, and its launches are recorded there with their logs. Two launches aborted, one by a process-cleanup pattern and one by machine load. A third and a fourth completed eleven and eighteen seeds before stalling on a Gazebo transport fault, and their rows were recovered from the console logs. A fifth ran the last seed. Its counts are in Table 15 and its interval in Table 27.

Limitation 8, seed-family variance. Two visible instances. The learned build’s 1.47 actions on the held-out family 10000–10074 against 1.80 on the budget-sweep family 20000–20099, both on levels 2–5. And the

gaslight action average of §4.3: 3.1 on the 40-seed L6 battery, 2.76 on the 25-seed battery artifact, 3.25 on the 12 head-to-head seeds, against ~ 1.9 undeceived on the same seeds. Variance across seed families is otherwise unquantified and single-machine.

Limitation 10, SLAM and the SubT score. SLAM runs only in the Nav2 integration checks, where the sensor vantage still comes from physics. In the SubT protocol the position half of the scoring rule is inherited from the sensor by construction.

Limitation 12, the two real components and the real-pixels sitting. The evaluations invest in simulator fidelity, a real detector on the perception seam, a real navigation stack on the navigation seam, and artifact reproducibility. The detector ran in AI2-THOR and Nav2 in the gz-sim arena. The combined lane, YOLO-World in the Sense slot of the Nav2 robot, passed 26/26 checks in a four-bay arena with same-mesh boxes and an association step that knows the entities’ physics poses, a stand-in for a tracker. The real-pixels battery removes the simulator and the association step from sensing but not the robot: the camera was carried by hand, the six zones were declared rather than mapped, and locomotion was a person walking. Its 36 hunts are one house, one camera and one sitting, so its interval is wide (Wilson lower bound 0.90), and one of the 36 was decided by a belief margin of 0.02 over a look-alike reading. A simulator is also where hidden truth, stale and adversarial claims, action budgets, thousands of seeds and exact replay of every failure can be controlled at once, which a single physical platform cannot offer.

C Glossary

- ECE: the expected calibration error, the mean absolute gap between stated belief and observed frequency over confidence bins.
- Brier score: the mean squared error of the stated belief.
- IoU: intersection over union between a detector box and an instance mask.
- NBV: next-best view, the vantage that uncovers the most unseen volume.
- POMDP: partially observable Markov decision process.
- POUCT: partially observable UCT, the sampled planner `pomdp_py` ships.
- SLAM: simultaneous localization and mapping.
- ROS 2: the Robot Operating System 2. Nav2 is its navigation stack, whose DWB and MPPI controllers are its dynamic-window and model-predictive path-integral local planners.
- WSL 2: the Windows Subsystem for Linux.
- EGL: the headless OpenGL interface some simulators require.
- EOL: end of life.
- tp: a sensor’s true-positive rate, in the self-repair tables.

References

- [1] Ashutosh Adhikari, Xingdi Yuan, Marc-Alexandre Côté, Mikuláš Zelinka, Marc-Antoine Rondeau, Romain Laroche, Pascal Poupart, Jian Tang, Adam Trischler, and William L. Hamilton. Learning dynamic belief graphs to generalize on text-based games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2002.09127.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [3] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.
- [4] Francesco Argenziano, John Mark Alexis Marcelo, Michele Brienza, Abdel Hakim Drid, Emanuele Musumeci, Daniele Nardi, Domenico D. Bloisi, and Vincenzo Suriani. R2F: Repurposing ray frontiers for LLM-free object navigation. *arXiv preprint arXiv:2603.08475*, 2026.
- [5] Dhruv Batra, Aaron Gokaslan, Aniruddha Kembhavi, Oleksandr Maksymets, Roozbeh Mottaghi, Manolis Savva, Alexander Toshev, and Erik Wijmans. Objectnav revisited: On evaluation of embodied agents navigating to objects. *arXiv preprint arXiv:2006.13171*, 2020.
- [6] Thomas Bolander and Mikkel Birkegaard Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics*, 21(1):9–34, 2011. doi: 10.3166/jancl.21.9-34.
- [7] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, Lecture Notes in Computer Science, pages 316–330, 2001. doi: 10.1007/3-540-44503-X_20.
- [8] Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. Automanual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.16247.
- [9] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [10] C. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1): 41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- [11] Timothy H. Chung, Viktor Orekhov, and Angela Maio. Into the robotic depths: Analysis and insights from the DARPA subterranean challenge. *Annual Review of Control, Robotics, and Autonomous Systems*, 6:477–502, 2023.
- [12] C. Connolly. The determination of next best views. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 432–435, 1985.
- [13] Michael T. Cox. Metacognition in computation: A selected research review. *Artificial Intelligence*, 169(2): 104–141, 2005.
- [14] Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.
- [15] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [16] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.
- [17] Salomé Eriksson, Gabriele Röger, and Malte Helmert. Unsolvability certificates for classical planning. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 27, 2017. doi: 10.1609/icaps.v27i1.13818.

- [18] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. arXiv:1705.08500.
- [19] Google DeepMind. Gemini api model documentation: gemini-3.7-flash. <https://ai.google.dev/gemini-api/docs/models>, 2026. Accessed August 2026.
- [20] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007. doi: 10.1145/1265530.1265535.
- [21] Alasdair Hill, Ekaterina Komendantskaya, and Ronald P. A. Petrick. Proof-carrying plans: a resource logic for AI planning. In *Proceedings of the 22nd International Symposium on Principles and Practice of Declarative Programming (PPDP)*, pages 1–13, 2020. doi: 10.1145/3414080.3414094. arXiv:2008.04165.
- [22] International Organization for Standardization. ISO 13482:2014 — robots and robotic devices — safety requirements for personal care robots. Geneva, Switzerland, 2014. <https://www.iso.org/standard/53820.html>.
- [23] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9–10):1194–1227, 2013.
- [24] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *IEEE Computer*, 36(1):41–50, 2003.
- [25] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, et al. AI2-THOR: An interactive 3D environment for visual AI. *arXiv preprint arXiv:1712.05474*, 2017.
- [26] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.
- [27] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012.
- [28] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- [29] Peiqi Liu, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. OK-Robot: What really matters in integrating open-knowledge models for robotics. *arXiv preprint arXiv:2401.12202*, 2024.
- [30] Joel Loo, Zhanxin Wu, and David Hsu. Open scene graphs for open-world object-goal navigation. *arXiv preprint arXiv:2508.04678*, 2025.
- [31] Matt Luckcuck, Marie Farrell, Louise Dennis, Clare Dixon, and Michael Fisher. Formal specification and verification of autonomous robotic systems: A survey. *ACM Computing Surveys*, 52(5), 2019. arXiv:1807.00048.
- [32] Imen Mahdi, Matteo Cassinelli, Fabien Despinoy, Tim Welschehold, and Abhinav Valada. Relational semantic reasoning on 3D scene graphs for open world interactive object search. *arXiv preprint arXiv:2603.05642*, 2026.
- [33] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [34] Yu Qian, Hong Miao, Boyang Guo, Tingyi Jiang, Shan Zhao, Tianxing Le, Lintian Li, and Meng Liu. ECHO: A cognitively inspired, auditable memory plane for long-horizon agents. *arXiv preprint arXiv:2608.21755*, 2026.
- [35] Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Liangyu Li, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: Compiling trajectories into validated typed agent artifacts. *arXiv preprint arXiv:2601.22758*, 2026. Reports 100.0% (134/134) single-attempt success on ALFWorld with a GPT-5.6-terra backbone.
- [36] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [37] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- [38] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Sünderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.06135.
- [39] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [40] Donald B. Reid. An algorithm for tracking multiple targets. *IEEE Transactions on Automatic Control*, 24(6): 843–854, 1979.
- [41] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. Robots that ask for help: Uncertainty alignment for large language model planners. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.01928.
- [42] William R. Scott, Gerhard Roth, and Jean-François Rivest. View planning for automated three-dimensional object reconstruction and inspection. *ACM Computing Surveys*, 35(1):64–96, 2003.
- [43] Hardik Shah, Erica Tevere, Deegan Atha, Marcel Kaufmann, Shehryar Khattak, Manthan Patel, Marco Hutter, Jonas Frey, and Patrick Spieler. WildOS: Open-vocabulary object search in the wild. *arXiv preprint arXiv:2602.19308*, 2026.
- [44] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [45] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. arXiv:1912.01734.
- [46] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.03768.
- [47] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. LLM-Planner: Few-shot grounded planning for embodied agents with large language models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. arXiv:2212.04088.
- [48] David Smith Sundarsingh, Yifei Li, Tianji Tang, George J. Pappas, Nikolay Atanasov, and Yiannis Kantaros. Safe planning in unknown environments using conformalized semantic maps. *arXiv preprint arXiv:2509.25124*, 2025.
- [49] Wenjing Tang, Xinyu He, Yongxi Huang, Yunxiao Xiao, Cewu Lu, and Panpan Cai. Tru-POMDP: Task planning under uncertainty via tree of hypotheses and open-ended POMDPs. *arXiv preprint arXiv:2506.02860*, 2025.
- [50] Cagri Temel. Towards trustworthy autonomous robots: An explainable AI-based decision framework. *arXiv preprint arXiv:2609.02861*, 2026.
- [51] Marco Tranzatto, Takahiro Miki, Mihir Dharmadhikari, Lukas Bernreiter, Mihir Kulkarni, Frank Mascarich, Olov Andersson, Shehryar Khattak, Marco Hutter, Roland Siegwart, and Kostas Alexis. CERBERUS in the DARPA subterranean challenge. *Science Robotics*, 7(66):eabp9742, 2022.
- [52] Arthur Wandzel, Yoonseon Oh, Michael Fishman, Nishanth Kumar, Lawson L. S. Wong, and Stefanie Tellex. Multi-object search using object-oriented POMDPs. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019. doi: 10.1109/ICRA.2019.8793888.
- [53] Yuchen Xiao, Sammie Katt, Andreas ten Pas, Shengjian Chen, and Christopher Amato. Online planning for target object search in clutter under partial observability. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8241–8247, 2019.

- [54] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. PAST-Bench: Benchmarking the foundations of recursive self-improvement in personal agents. *arXiv preprint arXiv:2608.04003*, 2026.
- [55] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [56] Naoki Yokoyama, Sehoon Ha, Dhruv Batra, Jiuguang Wang, and Bernadette Bucher. VLFM: Vision-language frontier maps for zero-shot semantic navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. arXiv:2312.03275.
- [57] Zhaochen Yu, Yingcheng Wu, Zhenfei Yin, Kaiyuan Chen, Zhe Zhao, Mengdi Wang, Shuicheng Yan, and Ling Yang. Recursive experiential-working memory evolution for long-horizon agent harnesses. *arXiv preprint arXiv:2608.24876*, 2026. Introduces the Recuris system.
- [58] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [59] Guoyang Zhao, Yudong Li, Weiqing Qi, Kai Zhang, Bonan Liu, Kai Chen, Haoang Li, and Jun Ma. Decision-driven semantic object exploration for legged robots via confidence-calibrated perception and topological subgoal selection. *arXiv preprint arXiv:2509.20739*, 2025.
- [60] Kaiyu Zheng and Stefanie Tellex. pomdp_py: A framework to build and solve POMDP problems. *arXiv preprint arXiv:2004.10099*, 2020.
- [61] Kaiyu Zheng, Yoonchang Sung, George Konidaris, and Stefanie Tellex. Multi-resolution POMDP planning for multi-object search in 3D. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021. arXiv:2005.02878.
- [62] Kaiyu Zheng, Rohan Chitnis, Yoonchang Sung, George Konidaris, and Stefanie Tellex. Towards optimal correlational object search. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2022. arXiv:2110.09991.